

RQ3.R

Ester Maiocchi (10800862), Marianna Mazza (10838595), Arianna Perotti (10811649)

RQ3: Can we predict how a new device would perform in terms of user satisfaction? ----

```
df <- read.csv("df_clean.csv")
df <- df[,-1]
```

```
seed <- 2002
set.seed(seed)
```

Transform categorical variables into factors

```
df$Device_Name <- as.factor(df$Device_Name)
df$Brand <- as.factor(df$Brand)
df$Category <- as.factor(df$Category)
df$Model <- as.factor(df$Model)
```

Transform water resistance into a factor with ordered levels

```
unique(df$Water_Resistance_Rating)
```

```
## [1] "3ATM" "IP68" "IPX8" "5ATM" "IPX4" "IPX7" "10ATM"
```

```
df$Water_Resistance_Rating <- factor(df$Water_Resistance_Rating,
                                     levels = c("IPX4", "3ATM", "IPX7", "5ATM",
                                     "IPX8", "IP68", "10ATM"),
                                     ordered = TRUE)
```

```
df$Water_Resistance_Score <- as.numeric(df$Water_Resistance_Rating)
```

Transform connectivity into a factor with ordered levels

```
unique(df$Connectivity_Features)
```

```
## [1] "Bluetooth, WiFi" "WiFi, Bluetooth, NFC"
## [3] "Bluetooth" "WiFi, Bluetooth, NFC, LTE"
```

```
df$Connectivity_Features <- factor(
  df$Connectivity_Features,
  levels = c("Bluetooth", "Bluetooth, WiFi", "WiFi, Bluetooth, NFC", "WiFi,
  Bluetooth, NFC, LTE"),
  ordered = TRUE)
```

```
df$Connectivity_Features_Score <- as.numeric(df$Connectivity_Features)
```

Transform app ecosystem into a factor with ordered levels

```
unique(df$App_Ecosystem_Support)
```

```
## [1] "Cross-platform" "iOS" "Android/iOS"
```

```
df$App_Ecosystem_Support <- factor(
  df$App_Ecosystem_Support,
```

```

  levels = c("iOS", "Android/iOS", "Cross-platform"),
  ordered = TRUE)
df$App_Ecosystem_Support_Score <- as.numeric(df$App_Ecosystem_Support)

library(tidyverse)

## — Attaching core tidyverse packages ————— tidyverse 2.0.0
—
## ✓ dplyr      1.1.4      ✓ readr      2.1.6
## ✓ forcats   1.0.1      ✓ stringr   1.6.0
## ✓ ggplot2   4.0.1      ✓ tibble    3.3.0
## ✓ lubridate 1.9.4      ✓ tidyr     1.3.1
## ✓ purrr     1.2.0
## — Conflicts ————— tidyverse_conflicts()
—
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
  conflicts to become errors

library(mgcv)

## Caricamento del pacchetto richiesto: nlme
##
## Caricamento pacchetto: 'nlme'
##
## Il seguente oggetto è mascherato da 'package:dplyr':
##
##      collapse
##
## This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.

library(pbapply)

# Conformal prediction intervals for User satisfaction ----
library(mgcv)
library(conformalInference)
library(rgl)
library(dbscan)

##
## Caricamento pacchetto: 'dbscan'
##
## Il seguente oggetto è mascherato da 'package:stats':
##
##      as.dendrogram

```

```

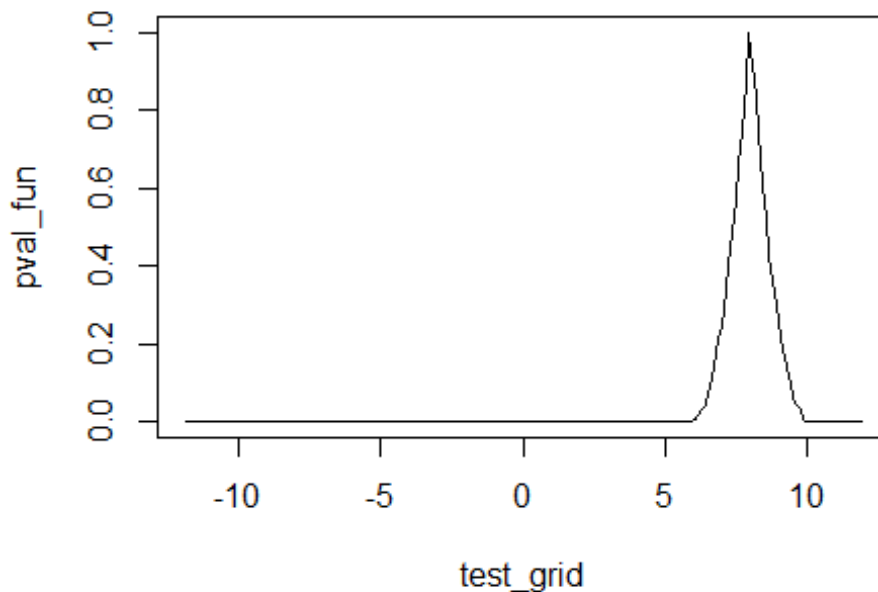
library(pbapply)
library(beadplexr)

n_grid <- 200
grid_factor <- 1.25
alpha <- 0.1
y <- df$User_Satisfaction_Rating
n <- length(y)

# T prediction interval
wrapper_full <- function(grid_point, y){
  aug_y <- c(grid_point,y)
  ncm <- numeric(length(aug_y))
  for (i in 1:length(aug_y)) {
    ncm[i] <- abs(aug_y[i] - mean(aug_y[-i]))
  }
  sum(ncm>=ncm[1])/(length(aug_y))
}

test_grid <- seq(-
grid_factor*max(abs(y)),+grid_factor*max(abs(y)),length.out=n_grid)
pval_fun <- sapply(test_grid, wrapper_full, y)
plot(test_grid,pval_fun,type='l')

```



```

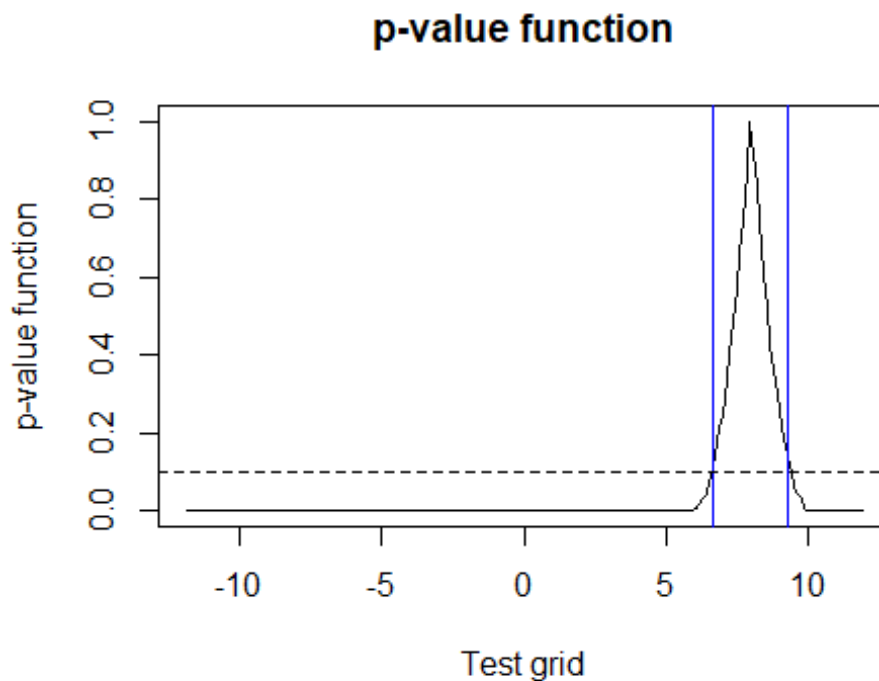
index_in <- pval_fun>alpha
pred_t_interval <- range(test_grid[index_in])

```

```

plot_pval = function(test_grid, pval_fun, pred, alpha) {
  plot(
    test_grid,
    pval_fun,
    type = 'l',
    main = "p-value function",
    xlab = "Test grid",
    ylab = "p-value function"
  )
  abline(v = pred, col = 'blue')
  abline(h = alpha, lty = 2)
}
plot_pval(test_grid, pval_fun, pred_t_interval, alpha)

```



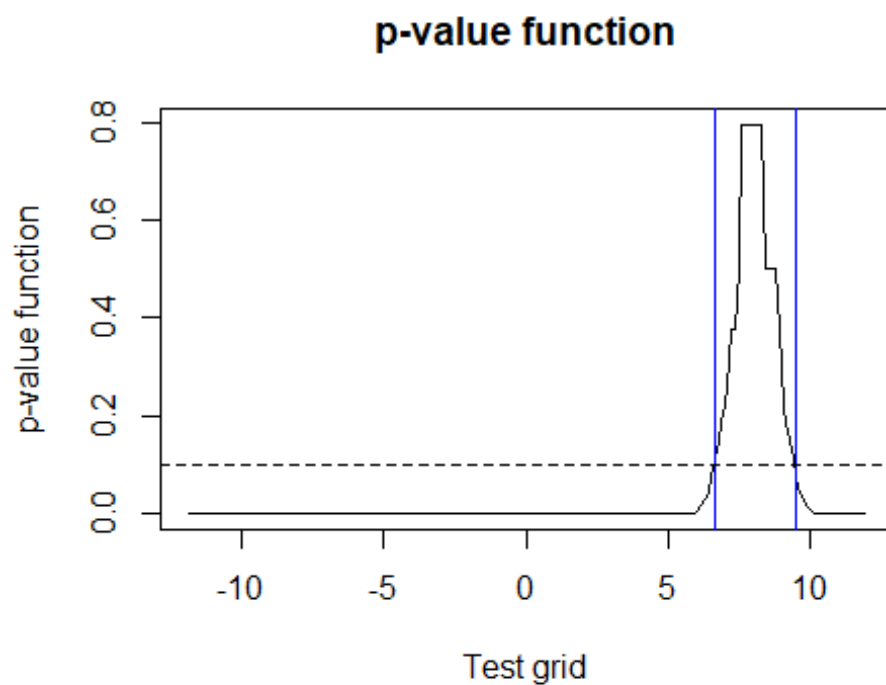
```

# KNN distance
pval_fun = numeric(n_grid)
k_s = 0.45
wrapper_knn = function(grid_point, y) {
  aug_y = c(grid_point, y)
  ncm = kNNdist(matrix(aug_y), k_s * n)
  sum((ncm[-1] >= ncm[1])) / (n + 1)
}

pval_fun = sapply(test_grid, wrapper_knn, y)
index_in = pval_fun > alpha
pred_knn = test_grid[as.logical(c(0, abs(diff(index_in))))]

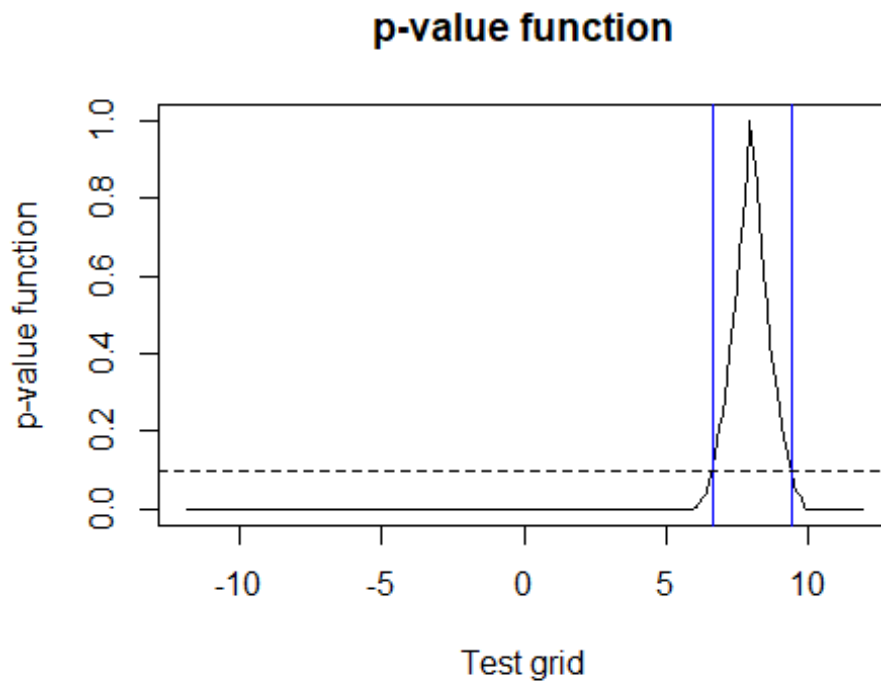
plot_pval(test_grid, pval_fun, pred_knn, alpha)

```



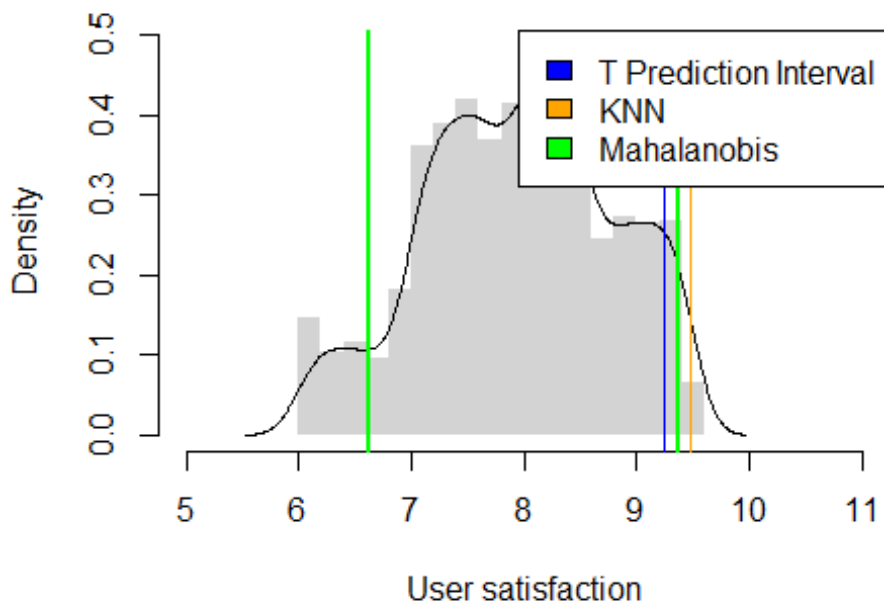
```
# Mahalanobis distance
pval_fun = numeric(n_grid)
wrapper_mal = function(grid_point) {
  aug_y = c(grid_point, y)
  ncm = mahalanobis(matrix(aug_y), colMeans(matrix(aug_y)), cov(matrix(aug_y)))
  sum((ncm[-1] >= ncm[1])) / (n + 1)
}
pval_fun = sapply(test_grid, wrapper_mal)
index_in = pval_fun > alpha
pred_mahalanobis = test_grid[as.logical(c(0, abs(diff(index_in))))]

plot_pval(test_grid, pval_fun, pred_mahalanobis, alpha)
```



```
# Show results
hist(
  y,
  breaks = 20,
  freq = FALSE,
  main = 'Histogram of User satisfaction',
  xlab = 'User satisfaction',
  xlim = c(5, 11),
  border = NA
)
lines(density(y))
abline(v = jitter(pred_t_interval, amount=0.003), col = 'blue', lwd = 1)
abline(v = jitter(pred_knn, amount=0.003), col = 'orange', lwd = 1)
abline(v = jitter(pred_mahalanobis, amount=0.003), col = 'green', lwd = 2)
legend("topright",
  legend = c("T Prediction Interval", "KNN", "Mahalanobis"),
  fill = c("blue", "orange", "green"))
```

Histogram of User satisfaction



```
result = data.frame(
  rbind(
    "T Prediction Interval"=pred_t_interval,
    "KNN"=pred_knn,
    "Mahalanobis"=pred_mahalanobis
  )
)
names(result) = c("LOWER", "UPPER")
print(result)

##              LOWER      UPPER
## T Prediction Interval 6.623744 9.249372
## KNN                   6.623744 9.488065
## Mahalanobis           6.623744 9.368719

# GAM model for User Satisfaction ----

# Select relevant variables
vars <- c(
  "User_Satisfaction",
  "Battery_Life_Hours",
  "Heart_Rate_Accuracy_Percent",
  "Step_Count_Accuracy_Percent",
  "Sleep_Tracking_Accuracy_Percent",
  "GPS_Quality",
  "Health_Sensors_Count",
  "Water_Resistance_Rating",
  "Connectivity_Features",
```

```

    "App_Ecosystem_Support",
    "Price_USD",
    "Category",
    "Brand"
)

# Train / calibration split
df_model <- df
n <- nrow(df_model)
cal_idx <- sample(1:n, size = floor(0.25 * n))

df_cal <- df_model[cal_idx, ]
df_train <- df_model[-cal_idx, ]
n_train <- nrow(df_train)

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Battery_Life_Hours, bs="cr") +
    s(Heart_Rate_Accuracy_Percent, bs="cr") +
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Sleep_Tracking_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    s(Health_Sensors_Count, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

summary(gam_fit)

##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Battery_Life_Hours, bs = "cr") +
##   s(Heart_Rate_Accuracy_Percent, bs = "cr") + s(Step_Count_Accuracy_Percent,
##   bs = "cr") + s(Sleep_Tracking_Accuracy_Percent, bs = "cr") +
##   s(GPS_Accuracy_Meters, bs = "cr") + s(Health_Sensors_Count,
##   bs = "cr") + Water_Resistance_Rating + Connectivity_Features +
##   App_Ecosystem_Support + s(Price_USD, bs = "cr") + Category +
##   Brand
##

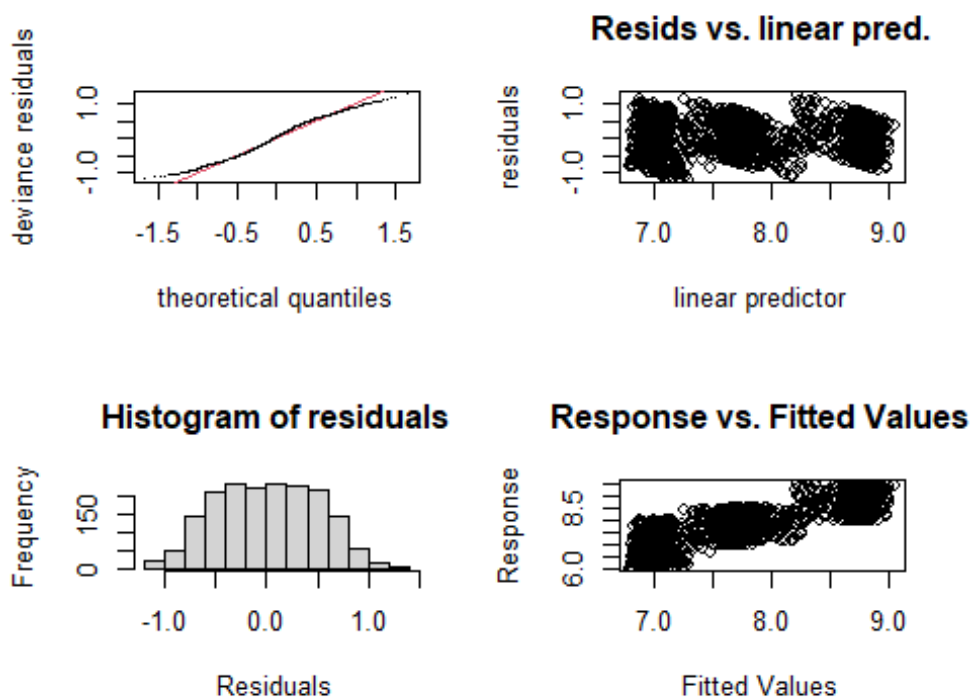
```



```

## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)      5.891211    0.026269 224.264 < 2e-16 ***
## Water_Resistance_Rating.L -0.179686    0.121706  -1.476    0.140
## Water_Resistance_Rating.Q  0.092541    0.072554   1.275    0.202
## Water_Resistance_Rating.C  0.018384    0.061787   0.298    0.766
## Water_Resistance_Rating^4 -0.080928    0.050695  -1.596    0.111
## Water_Resistance_Rating^5  0.032263    0.036531   0.883    0.377
## Water_Resistance_Rating^6 -0.031605    0.037798  -0.836    0.403
## Connectivity_Features.L -0.620624    0.025346 -24.486 < 2e-16 ***
## Connectivity_Features.Q  0.001118    0.027962   0.040    0.968
## Connectivity_Features.C  0.382360    0.029562  12.934 < 2e-16 ***
## App_Ecosystem_Support.L  0.812186    0.034461  23.568 < 2e-16 ***
## App_Ecosystem_Support.Q -0.508567    0.028147 -18.068 < 2e-16 ***
## CategoryFitness Tracker  0.983483    0.064322  15.290 < 2e-16 ***
## CategorySmart Ring      0.520747    0.045484  11.449 < 2e-16 ***
## CategorySmartwatch      2.219506    0.027767  79.933 < 2e-16 ***
## CategorySports Watch    1.471909    0.044169  33.324 < 2e-16 ***
## BrandApple              1.181813    0.026775  44.138 < 2e-16 ***
## BrandFitbit              0.060865    0.056939   1.069    0.285
## BrandGarmin              0.172208    0.092196   1.868    0.062 .
## BrandHuawei              -0.039082    0.066186  -0.590    0.555
## BrandOura                0.520747    0.045484  11.449 < 2e-16 ***
## BrandPolar               0.023483    0.064058   0.367    0.714
## BrandSamsung             0.033569    0.073827   0.455    0.649
## BrandWHOO                0.695566    0.110973   6.268 4.6e-10 ***
## BrandWithings            0.000285    0.064301   0.004    0.996
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(Battery_Life_Hours)      1.0000 1.0000   0.176 0.6748
## s(Heart_Rate_Accuracy_Percent) 3.0992 3.8862   1.436 0.2295
## s(Step_Count_Accuracy_Percent) 1.0225 1.0447   4.408 0.0392 *
## s(Sleep_Tracking_Accuracy_Percent) 1.0000 1.0000   1.524 0.2172
## s(GPS_Accuracy_Meters)      0.4458 0.4458 204.790 <2e-16 ***
## s(Health_Sensors_Count)      1.0000 1.0000   0.208 0.6481
## s(Price_USD)                8.6581 8.9435 185.020 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 83/88
## R-sq.(adj) = 0.665   Deviance explained = 67.2%
## GCV = 0.23954   Scale est. = 0.2346     n = 1782
gam.check(gam_fit)

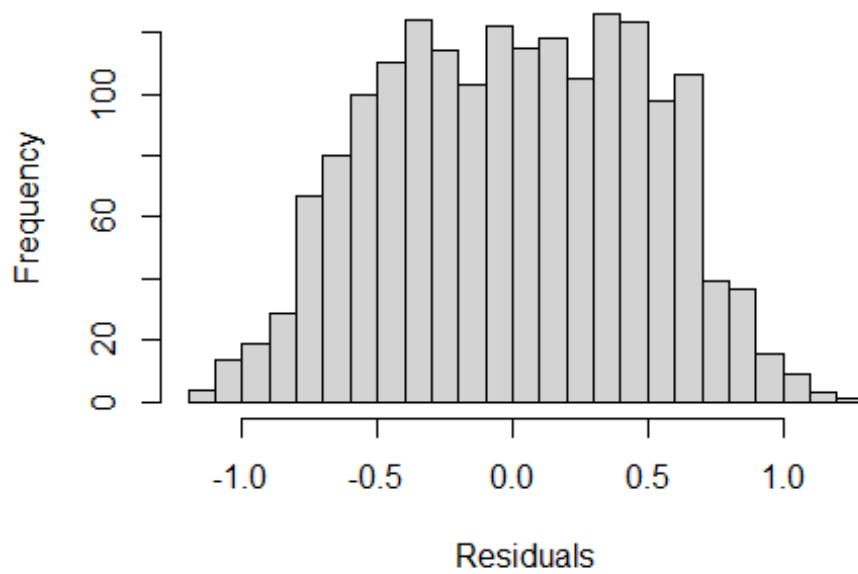
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 21 iterations.
## The RMS GCV score gradient at convergence was 1.724066e-08 .
## The Hessian was positive definite.
## Model rank = 83 / 88
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##          k'   edf k-index p-value
## s(Battery_Life_Hours)      9.000 1.000    1.01  0.595
## s(Heart_Rate_Accuracy_Percent) 9.000 3.099    0.99  0.410
## s(Step_Count_Accuracy_Percent) 9.000 1.023    0.99  0.285
## s(Sleep_Tracking_Accuracy_Percent) 9.000 1.000    1.01  0.660
## s(GPS_Accuracy_Meters)      9.000 0.446    1.01  0.680
## s(Health_Sensors_Count)      9.000 1.000    1.02  0.795
## s(Price_USD)                9.000 8.658    0.96  0.035 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

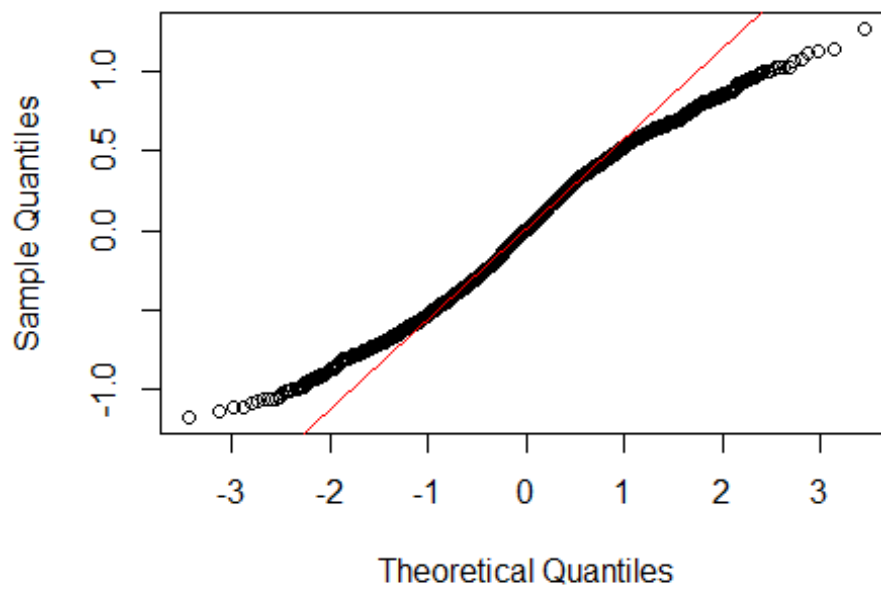
# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```

Histogram of GAM residuals



```
qqnorm(res)
qqline(res, col = "red")
```

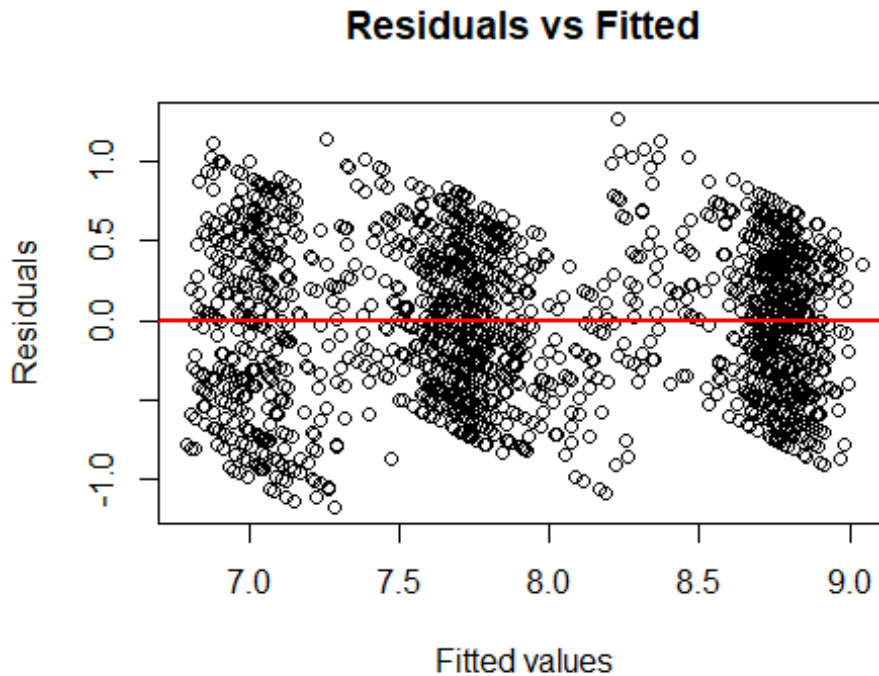
Normal Q-Q Plot



```
fit <- fitted(gam_fit)
```

```
plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# Permutation tests for GAM
B <- 1000

# H0: Battery_Life_Hours = 0

# Observed test statistic for Battery_Life_Hours
T0 = abs(summary(gam_fit)$s.table[1, 3])

# Fit null model excluding Battery_Life_Hours
gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Heart_Rate_Accuracy_Percent, bs="cr") +
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Sleep_Tracking_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    s(Health_Sensors_Count, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
```

```

    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
    data = df_train
)

res.H0 = gam_H0$residuals # residuals under H0

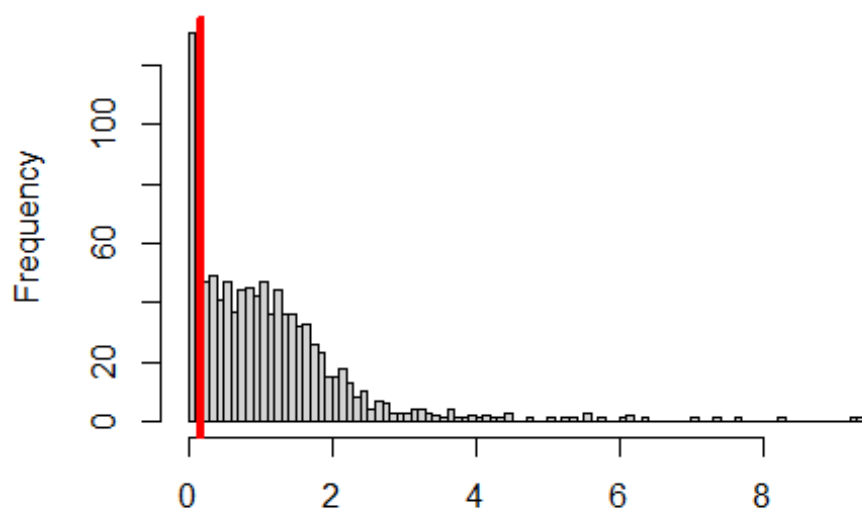
# Wrapper function to compute permuted test statistic
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~ s(Battery_Life_Hours, bs="cr") +
    s(Heart_Rate_Accuracy_Percent, bs="cr") +
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Sleep_Tracking_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    s(Health_Sensors_Count, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
    data = df_train
  )
  return(abs(summary(gam.perm)$s.table[1, 3]))
}

# Run permutation test
T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
     breaks = 100,
     main = 'Permutational distribution of test statistics',
     xlab = '')
abline(v = T0, col = 'red', lwd = 4)

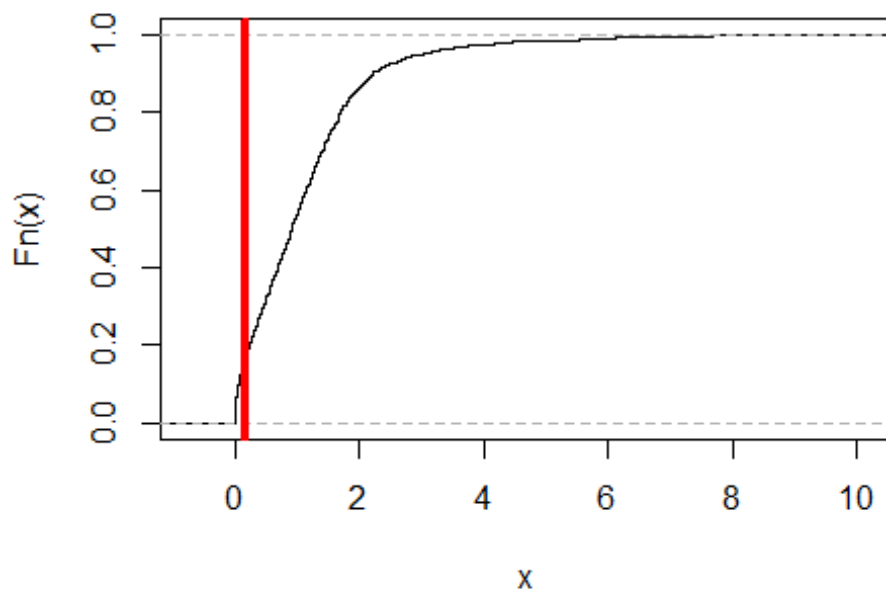
```

Permutational distribution of test statistics



```
plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)
```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P
```

```
## [1] 0.827
```

```
# we remove Battery_Life_Hours
```

```
# GAM model
```

```
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Heart_Rate_Accuracy_Percent, bs="cr") +
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Sleep_Tracking_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    s(Health_Sensors_Count, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)
```

```
summary(gam_fit)
```

```
##
```

```
## Family: gaussian
```

```
## Link function: identity
```

```
##
```

```
## Formula:
```

```
## User_Satisfaction_Rating ~ s(Heart_Rate_Accuracy_Percent, bs = "cr") +
```

```
##   s(Step_Count_Accuracy_Percent, bs = "cr") +
```

```
s(Sleep_Tracking_Accuracy_Percent,
```

```
##   bs = "cr") + s(GPS_Accuracy_Meters, bs = "cr") + s(Health_Sensors_Count,
```

```
##   bs = "cr") + Water_Resistance_Rating + Connectivity_Features +
```

```
##   App_Ecosystem_Support + s(Price_USD, bs = "cr") + Category +
```

```
##   Brand
```

```
##
```

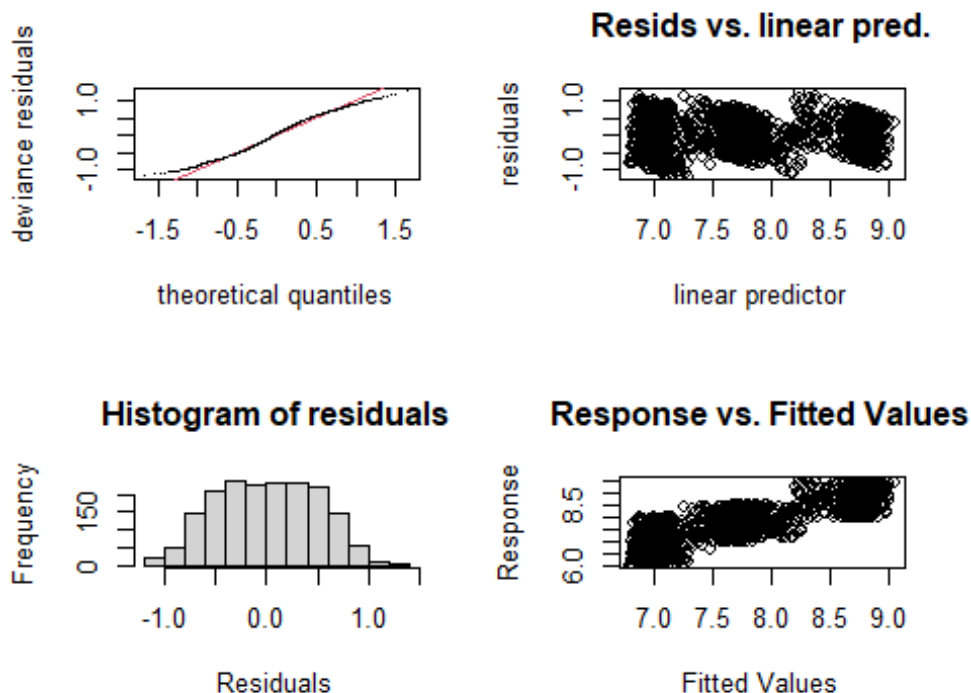
```
## Parametric coefficients:
```

##		Estimate	Std. Error	t value	Pr(> t)
##	(Intercept)	5.890785	0.026195	224.885	< 2e-16 ***
##	Water_Resistance_Rating.L	-0.188771	0.119552	-1.579	0.1145
##	Water_Resistance_Rating.Q	0.083870	0.069615	1.205	0.2285
##	Water_Resistance_Rating.C	0.011673	0.059701	0.196	0.8450
##	Water_Resistance_Rating^4	-0.085494	0.049473	-1.728	0.0841 .
##	Water_Resistance_Rating^5	0.030699	0.036301	0.846	0.3978
##	Water_Resistance_Rating^6	-0.031234	0.037775	-0.827	0.4085
##	Connectivity_Features.L	-0.619631	0.025145	-24.642	< 2e-16 ***
##	Connectivity_Features.Q	0.001998	0.027876	0.072	0.9429
##	Connectivity_Features.C	0.380945	0.029344	12.982	< 2e-16 ***

```

## App_Ecosystem_Support.L      0.812161    0.034449   23.576 < 2e-16 ***
## App_Ecosystem_Support.Q     -0.508415    0.028137  -18.069 < 2e-16 ***
## CategoryFitness Tracker      0.982254    0.064110   15.321 < 2e-16 ***
## CategorySmart Ring           0.519813    0.045409   11.447 < 2e-16 ***
## CategorySmartwatch           2.220815    0.027547   80.619 < 2e-16 ***
## CategorySports Watch         1.470246    0.044029   33.393 < 2e-16 ***
## BrandApple                   1.181751    0.026769   44.146 < 2e-16 ***
## BrandFitbit                  0.059983    0.056911    1.054  0.2920
## BrandGarmin                  0.153575    0.082035    1.872  0.0614 .
## BrandHuawei                  -0.039338    0.066161   -0.595  0.5522
## BrandOura                    0.519813    0.045409   11.447 < 2e-16 ***
## BrandPolar                   0.023519    0.064039    0.367  0.7135
## BrandSamsung                 0.033132    0.073799    0.449  0.6535
## BrandWHOOP                   0.697658    0.110438    6.317 3.37e-10 ***
## BrandWithings                0.000391    0.064283    0.006  0.9951
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Heart_Rate_Accuracy_Percent)  3.1149 3.9056   1.457  0.2216
## s(Step_Count_Accuracy_Percent)  1.0242 1.0480   4.398  0.0397 *
## s(Sleep_Tracking_Accuracy_Percent) 1.0000 1.0000   1.496  0.2214
## s(GPS_Accuracy_Meters)          0.4458 0.4459 205.568 <2e-16 ***
## s(Health_Sensors_Count)         1.0000 1.0000   0.218  0.6405
## s(Price_USD)                    8.6184 8.9413 185.463 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 74/79
## R-sq.(adj) = 0.665   Deviance explained = 67.1%
## GCV = 0.2393   Scale est. = 0.23449   n = 1782
gam.check(gam_fit)

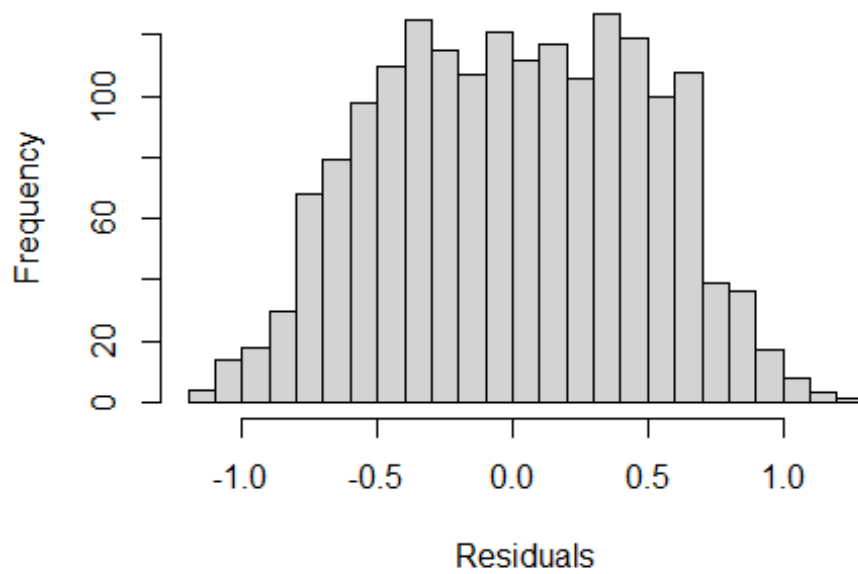
```

```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 17 iterations.
## The RMS GCV score gradient at convergence was 4.329835e-08 .
## The Hessian was positive definite.
## Model rank = 74 / 79
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##          k'   edf k-index p-value
## s(Heart_Rate_Accuracy_Percent) 9.000 3.115 0.99 0.410
## s(Step_Count_Accuracy_Percent) 9.000 1.024 0.99 0.275
## s(Sleep_Tracking_Accuracy_Percent) 9.000 1.000 1.01 0.710
## s(GPS_Accuracy_Meters) 9.000 0.446 1.01 0.600
## s(Health_Sensors_Count) 9.000 1.000 1.02 0.795
## s(Price_USD) 9.000 8.618 0.96 0.025 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

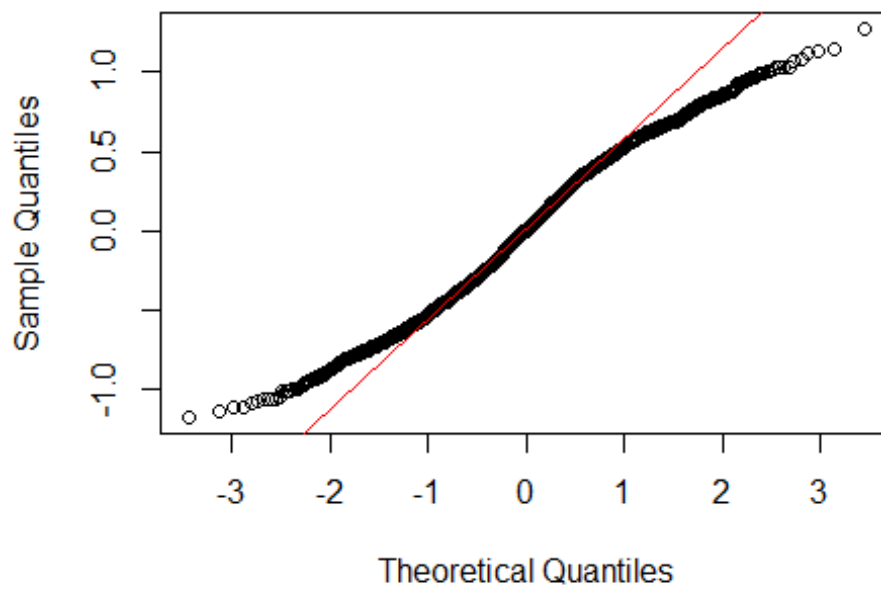
# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
     xlab = "Residuals")
```

Histogram of GAM residuals



```
qqnorm(res)  
qqline(res, col = "red")
```

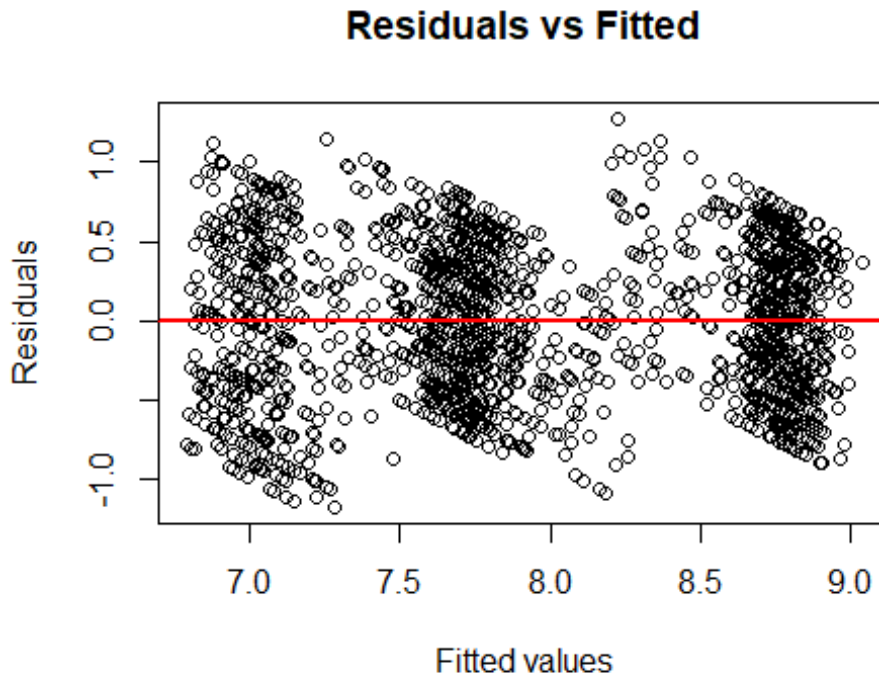
Normal Q-Q Plot



```
fit <- fitted(gam_fit)
```

```
plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# Permutation tests for GAM
# H0: Health_Sensors_Count = 0

# Observed test statistic for Health_Sensors_Count
T0 = abs(summary(gam_fit)$s.table[5, 3])

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Heart_Rate_Accuracy_Percent, bs="cr") +
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Sleep_Tracking_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
```

```

)

res.H0 = gam_H0$residuals

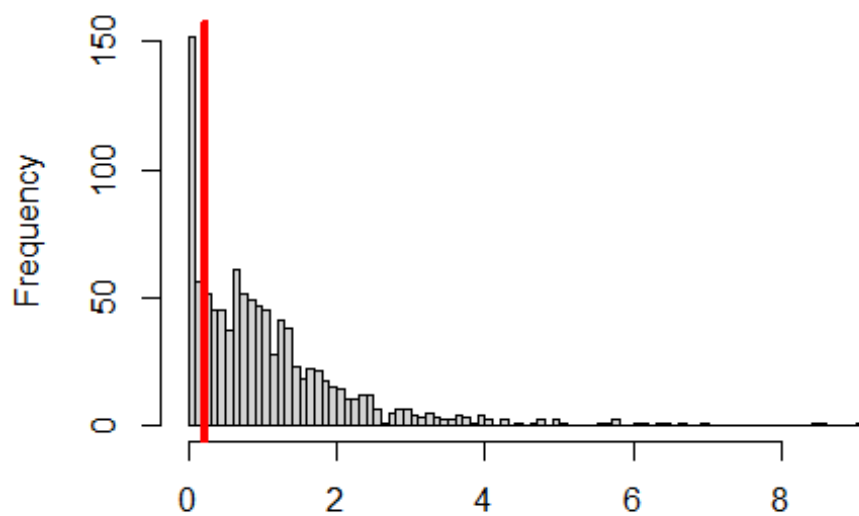
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Heart_Rate_Accuracy_Percent, bs="cr") +
      s(Step_Count_Accuracy_Percent, bs="cr") +
      s(Sleep_Tracking_Accuracy_Percent, bs="cr") +
      s(GPS_Accuracy_Meters, bs="cr") +
      s(Health_Sensors_Count, bs="cr") +
      Water_Resistance_Rating +
      Connectivity_Features +
      App_Ecosystem_Support +
      s(Price_USD, bs="cr") +
      Category +
      Brand,
    data = df_train
  )
  return(abs(summary(gam.perm)$s.table[5, 3]))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

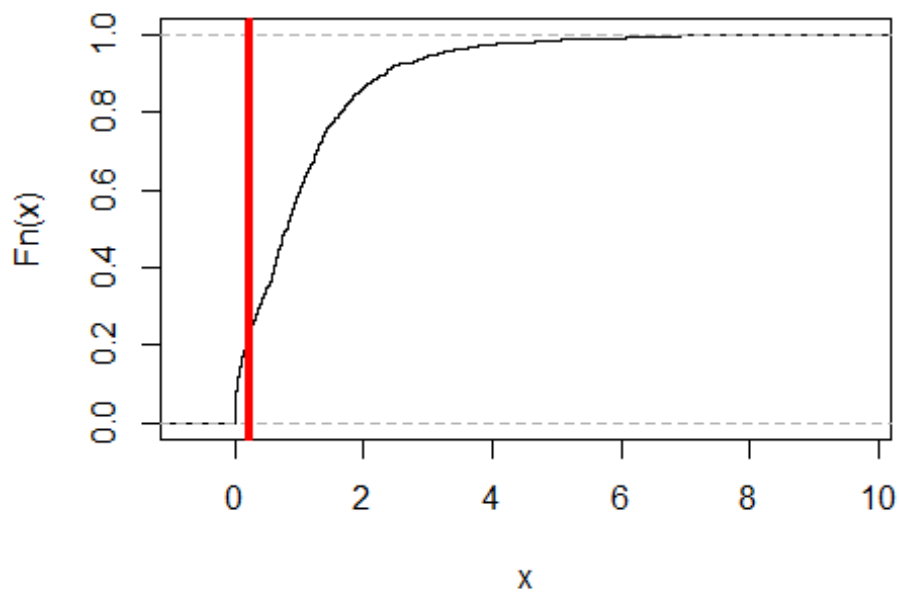
```

Permutational distribution of test statistics



```
plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)
```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P
```

```
## [1] 0.777
```

```
# we remove Health_Sensors_Count
```

```
# GAM model
```

```
gam_fit <- gam(  
  User_Satisfaction_Rating ~  
    s(Heart_Rate_Accuracy_Percent, bs="cr") +  
    s(Step_Count_Accuracy_Percent, bs="cr") +  
    s(Sleep_Tracking_Accuracy_Percent, bs="cr") +  
    s(GPS_Accuracy_Meters, bs="cr") +  
    Water_Resistance_Rating +  
    Connectivity_Features +  
    App_Ecosystem_Support +  
    s(Price_USD, bs="cr") +  
    Category +  
    Brand,  
  data = df_train  
)
```

```
summary(gam_fit)
```

```
##
```

```
## Family: gaussian
```

```
## Link function: identity
```

```
##
```

```
## Formula:
```

```
## User_Satisfaction_Rating ~ s(Heart_Rate_Accuracy_Percent, bs = "cr") +
```

```
##   s(Step_Count_Accuracy_Percent, bs = "cr") +
```

```
s(Sleep_Tracking_Accuracy_Percent,
```

```
##   bs = "cr") + s(GPS_Accuracy_Meters, bs = "cr") + Water_Resistance_Rating +
```

```
##   Connectivity_Features + App_Ecosystem_Support + s(Price_USD,
```

```
##   bs = "cr") + Category + Brand
```

```
##
```

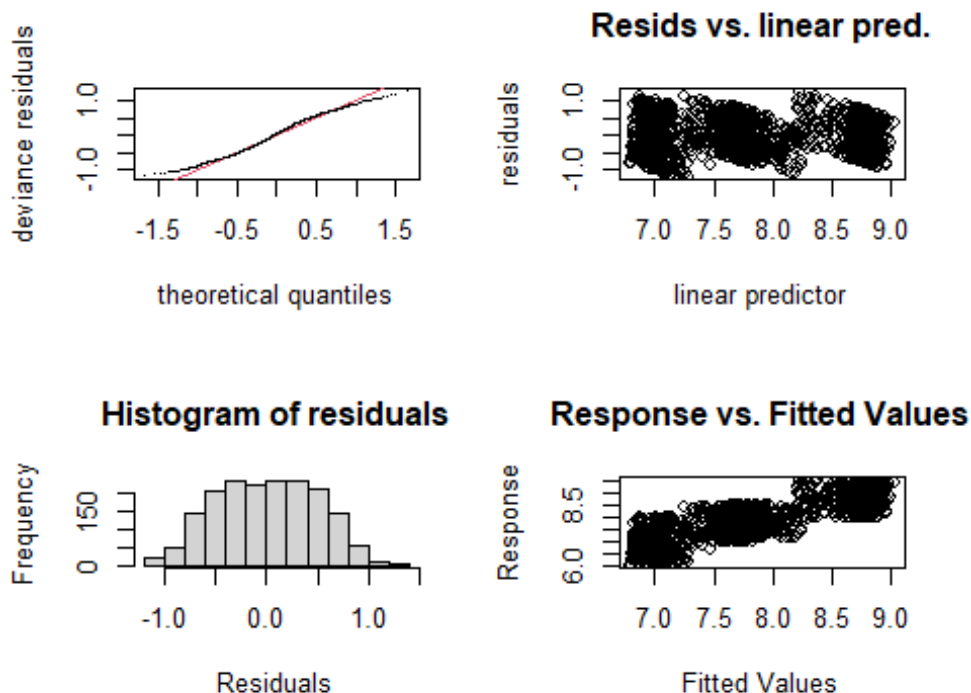
```
## Parametric coefficients:
```

```
##               Estimate Std. Error t value Pr(>|t|)  
## (Intercept)    5.889603   0.026056 226.033 < 2e-16 ***  
## Water_Resistance_Rating.L -0.188388   0.119542  -1.576   0.1152  
## Water_Resistance_Rating.Q  0.084745   0.069586   1.218   0.2234  
## Water_Resistance_Rating.C  0.012357   0.059683   0.207   0.8360  
## Water_Resistance_Rating^4 -0.084820   0.049453  -1.715   0.0865 .  
## Water_Resistance_Rating^5  0.030965   0.036298   0.853   0.3937  
## Water_Resistance_Rating^6 -0.031187   0.037778  -0.826   0.4092  
## Connectivity_Features.L  -0.614994   0.023144 -26.572 < 2e-16 ***  
## Connectivity_Features.Q   0.001750   0.027886   0.063   0.9500  
## Connectivity_Features.C   0.378193   0.028762  13.149 < 2e-16 ***  
## App_Ecosystem_Support.L   0.812522   0.034487  23.560 < 2e-16 ***  
## App_Ecosystem_Support.Q  -0.508367   0.028153 -18.057 < 2e-16 ***  
## CategoryFitness Tracker   0.985983   0.063606  15.501 < 2e-16 ***  
## CategorySmart Ring        0.517402   0.045116  11.468 < 2e-16 ***
```

```

## CategorySmartwatch      2.225601    0.025677  86.676 < 2e-16 ***
## CategorySports Watch    1.470041    0.044031  33.387 < 2e-16 ***
## BrandApple              1.181121    0.026789  44.089 < 2e-16 ***
## BrandFitbit             0.059983    0.056912   1.054  0.2920
## BrandGarmin             0.150147    0.082595   1.818  0.0693 .
## BrandHuawei             -0.039191    0.066180  -0.592  0.5538
## BrandOura               0.517402    0.045116  11.468 < 2e-16 ***
## BrandPolar              0.023591    0.064039   0.368  0.7126
## BrandSamsung            0.034264    0.073759   0.465  0.6423
## BrandWHOO              0.690576    0.109408   6.312 3.49e-10 ***
## BrandWithings          0.001083    0.064267   0.017  0.9866
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Heart_Rate_Accuracy_Percent)  3.117  3.908   1.458  0.2212
## s(Step_Count_Accuracy_Percent)  1.061  1.120   4.249  0.0502 .
## s(Sleep_Tracking_Accuracy_Percent) 1.000  1.000   1.488  0.2227
## s(GPS_Accuracy_Meters)          1.044  1.285  74.224 <2e-16 ***
## s(Price_USD)                   8.617  8.941 185.608 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 65/70
## R-sq.(adj) = 0.665   Deviance explained = 67.1%
## GCV = 0.23922   Scale est. = 0.23446   n = 1782
gam.check(gam_fit)

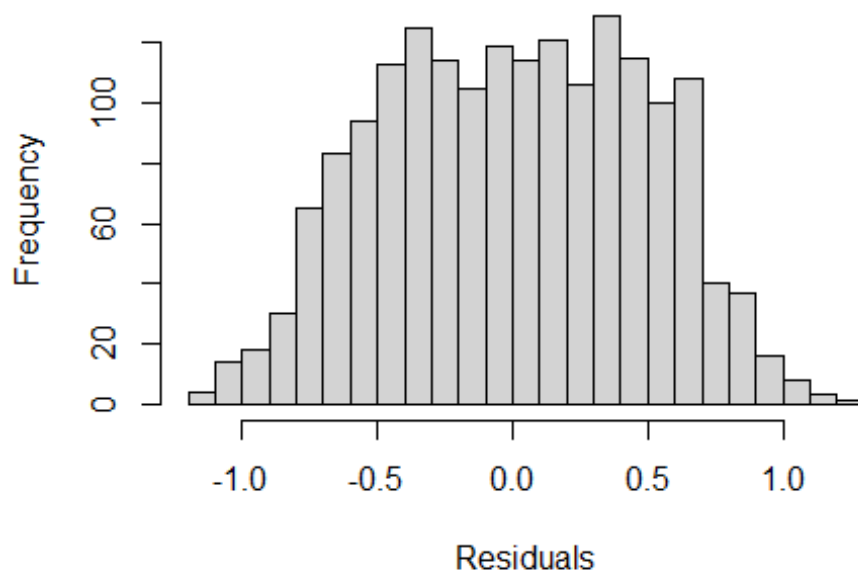
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 17 iterations.
## The RMS GCV score gradient at convergence was 6.97661e-08 .
## The Hessian was positive definite.
## Model rank = 65 / 70
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##          k'   edf k-index p-value
## s(Heart_Rate_Accuracy_Percent) 9.00 3.12    0.99  0.375
## s(Step_Count_Accuracy_Percent) 9.00 1.06    0.99  0.280
## s(Sleep_Tracking_Accuracy_Percent) 9.00 1.00    1.01  0.660
## s(GPS_Accuracy_Meters) 9.00 1.04    1.01  0.600
## s(Price_USD) 9.00 8.62    0.96  0.035 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

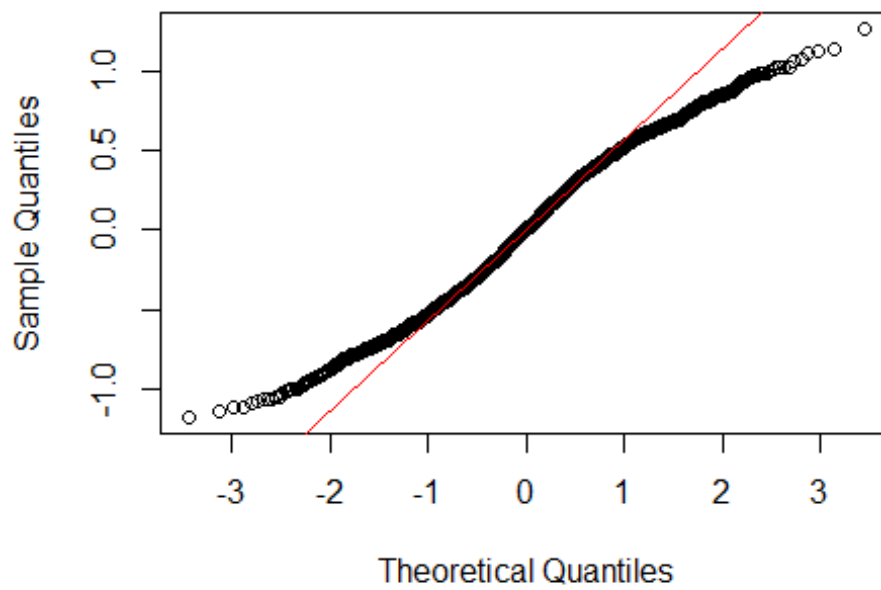
# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
     xlab = "Residuals")
```


Histogram of GAM residuals



```
qqnorm(res)  
qqline(res, col = "red")
```

Normal Q-Q Plot



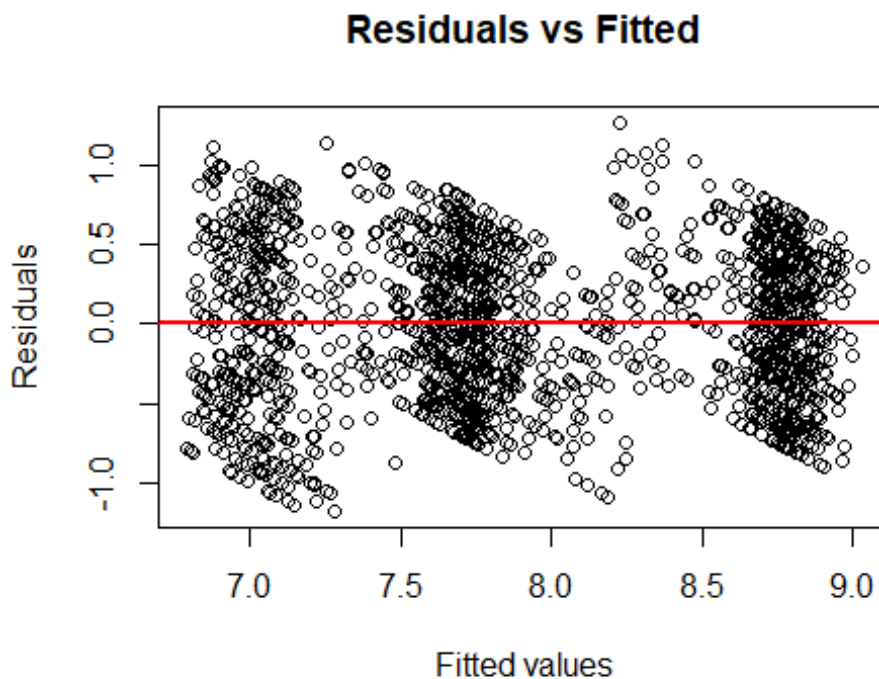
```

fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)

```



```

# Permutation tests for GAM
# H0: Sleep_Tracking_Accuracy_Percent = 0

# Observed test statistic for Sleep_Tracking_Accuracy_Percent
T0 = abs(summary(gam_fit)$s.table[3, 3])

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Heart_Rate_Accuracy_Percent, bs="cr") +
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +

```

```

    Brand,
    data = df_train
)

res.H0 = gam_H0$residuals

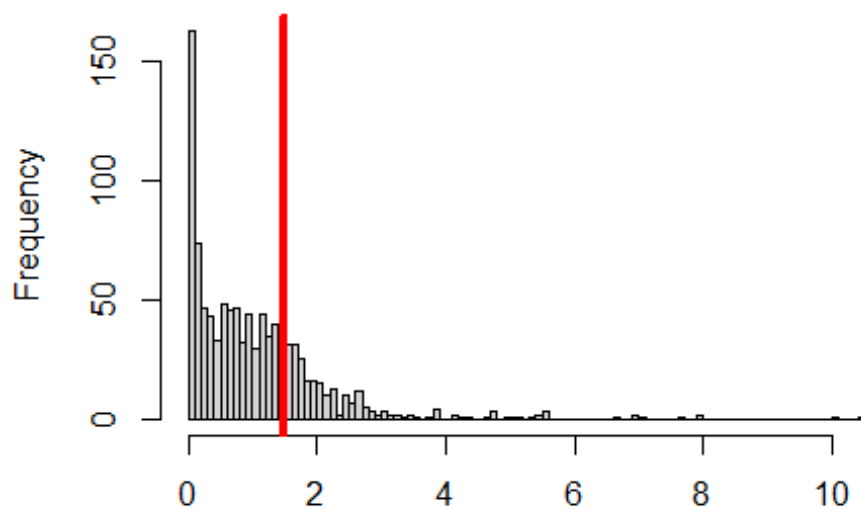
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Heart_Rate_Accuracy_Percent, bs="cr") +
      s(Step_Count_Accuracy_Percent, bs="cr") +
      s(Sleep_Tracking_Accuracy_Percent, bs="cr") +
      s(GPS_Accuracy_Meters, bs="cr") +
      Water_Resistance_Rating +
      Connectivity_Features +
      App_Ecosystem_Support +
      s(Price_USD, bs="cr") +
      Category +
      Brand,
    data = df_train
  )
  return(abs(summary(gam.perm)$s.table[3, 3]))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

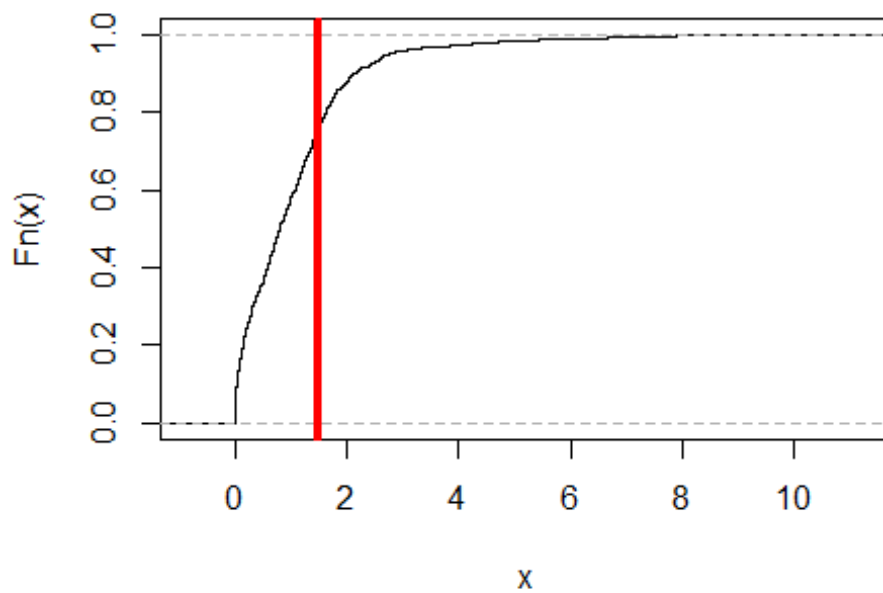
```

Permutational distribution of test statistics



```
plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)
```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P
```

```
## [1] 0.245

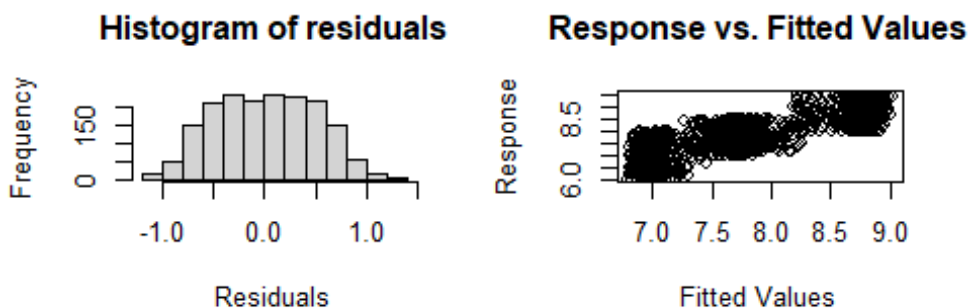
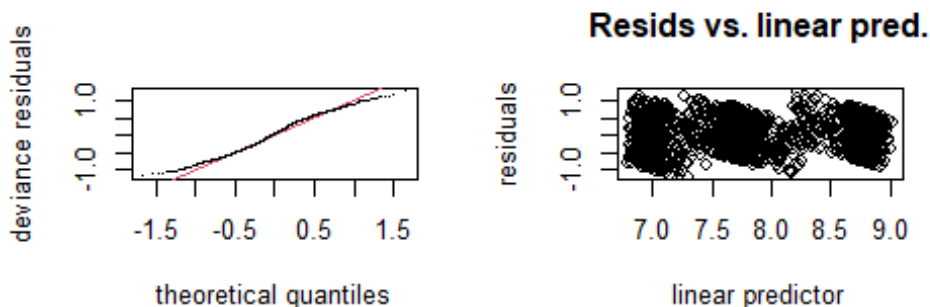
# we remove Sleep_Tracking_Accuracy_Percent

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Heart_Rate_Accuracy_Percent, bs="cr") +
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

summary(gam_fit)

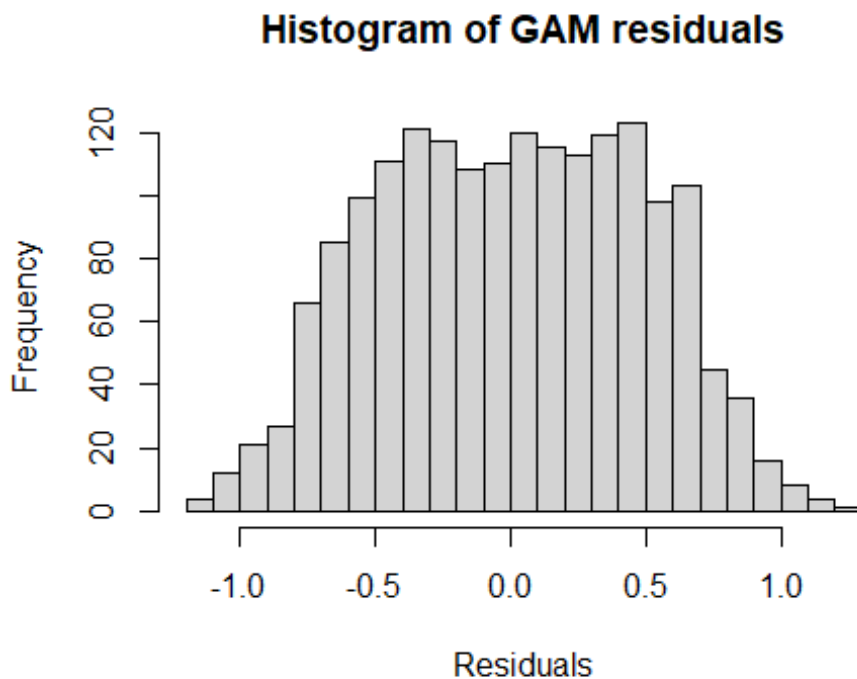
##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Heart_Rate_Accuracy_Percent, bs = "cr") +
##   s(Step_Count_Accuracy_Percent, bs = "cr") + s(GPS_Accuracy_Meters,
##   bs = "cr") + Water_Resistance_Rating + Connectivity_Features +
##   App_Ecosystem_Support + s(Price_USD, bs = "cr") + Category +
##   Brand
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   5.887512   0.025992 226.517 < 2e-16 ***
## Water_Resistance_Rating.L -0.186958   0.119535  -1.564   0.1180
## Water_Resistance_Rating.Q  0.086582   0.069569   1.245   0.2135
## Water_Resistance_Rating.C  0.011751   0.059679   0.197   0.8439
## Water_Resistance_Rating^4 -0.083333   0.049437  -1.686   0.0920 .
## Water_Resistance_Rating^5  0.031318   0.036288   0.863   0.3882
## Water_Resistance_Rating^6 -0.029879   0.037755  -0.791   0.4288
## Connectivity_Features.L  -0.607704   0.022358 -27.180 < 2e-16 ***
## Connectivity_Features.Q   0.001580   0.027871   0.057   0.9548
## Connectivity_Features.C   0.374932   0.028637  13.092 < 2e-16 ***
## App_Ecosystem_Support.L   0.814401   0.034459  23.634 < 2e-16 ***
## App_Ecosystem_Support.Q  -0.509180   0.028142 -18.093 < 2e-16 ***
## CategoryFitness Tracker   0.974555   0.062886  15.497 < 2e-16 ***
## CategorySmart Ring        0.543184   0.039787  13.652 < 2e-16 ***
## CategorySmartwatch        2.232534   0.025055  89.106 < 2e-16 ***
## CategorySports Watch      1.460085   0.043151  33.837 < 2e-16 ***
```

```
## BrandApple          1.178764    0.026727  44.104 < 2e-16 ***
## BrandFitbit         0.057769    0.056877   1.016  0.3099
## BrandGarmin         0.142414    0.082830   1.719  0.0857 .
## BrandHuawei        -0.041456    0.066125  -0.627  0.5308
## BrandOura          0.543184    0.039787  13.652 < 2e-16 ***
## BrandPolar         0.019929    0.063963   0.312  0.7554
## BrandSamsung       0.029832    0.073711   0.405  0.6857
## BrandWHOO          0.677153    0.108859   6.220 6.19e-10 ***
## BrandWithings     -0.002847    0.064177  -0.044  0.9646
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Heart_Rate_Accuracy_Percent) 3.0362 3.8078  1.400  0.2421
## s(Step_Count_Accuracy_Percent) 1.0879 1.1713  3.936  0.0665 .
## s(GPS_Accuracy_Meters)         0.4459 0.4459 210.763 <2e-16 ***
## s(Price_USD)                   8.6171 8.9409 185.823 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 56/61
## R-sq.(adj) =  0.665   Deviance explained = 67.1%
## GCV = 0.23899   Scale est. = 0.23447    n = 1782
gam.check(gam_fit)
```



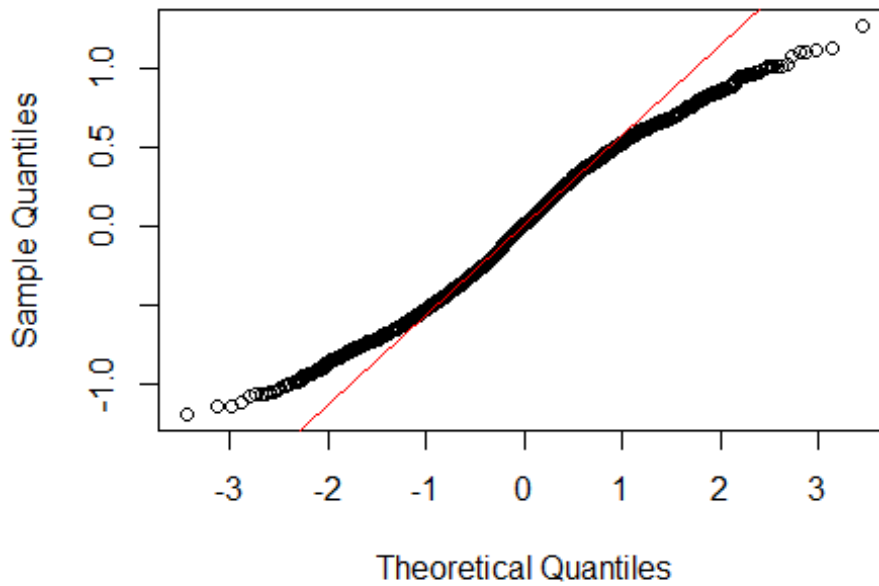
```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 16 iterations.
## The RMS GCV score gradient at convergence was 5.722106e-08 .
## The Hessian was positive definite.
## Model rank = 56 / 61
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##               k'   edf k-index p-value
## s(Heart_Rate_Accuracy_Percent) 9.000 3.036    0.99  0.315
## s(Step_Count_Accuracy_Percent) 9.000 1.088    0.99  0.250
## s(GPS_Accuracy_Meters)         9.000 0.446    1.00  0.605
## s(Price_USD)                   9.000 8.617    0.96  0.025 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```



```
qqnorm(res)
qqline(res, col = "red")
```

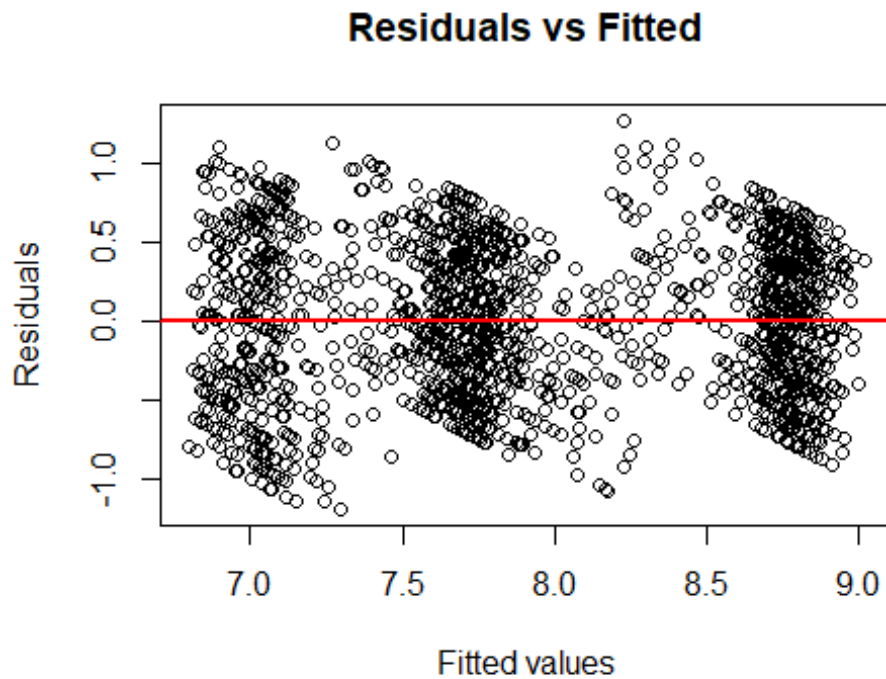
Normal Q-Q Plot



```
fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```

```
# Permutation test for GAM
# H0: Heart_Rate_Accuracy_Percent = 0

# Observed test statistic for Heart_Rate_Accuracy_Percent
T0 = abs(summary(gam_fit)$s.table[1, 3])

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

res.H0 = gam_H0$residuals

wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
```

```

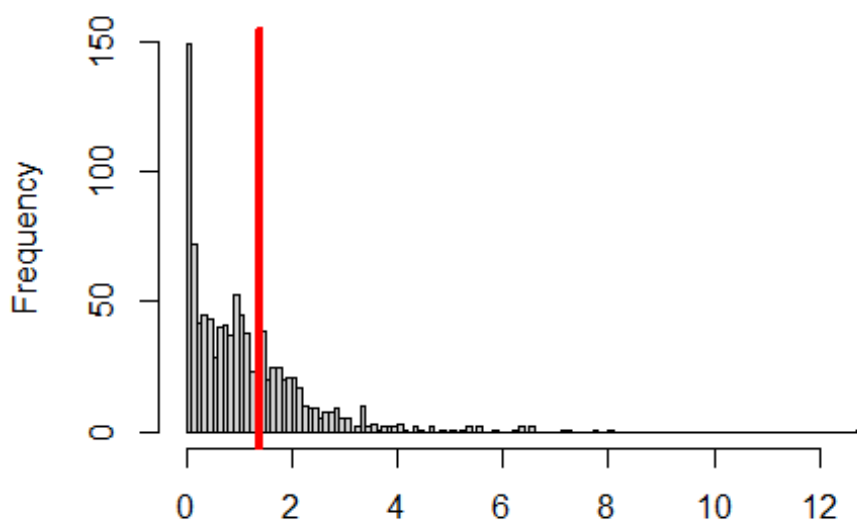
s(Heart_Rate_Accuracy_Percent, bs="cr") +
s(Step_Count_Accuracy_Percent, bs="cr") +
s(GPS_Accuracy_Meters, bs="cr") +
Water_Resistance_Rating +
Connectivity_Features +
App_Ecosystem_Support +
s(Price_USD, bs="cr") +
Category +
Brand,
data = df_train
)
return(abs(summary(gam.perm)$s.table[1, 3]))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

Permutational distribution of test statistics

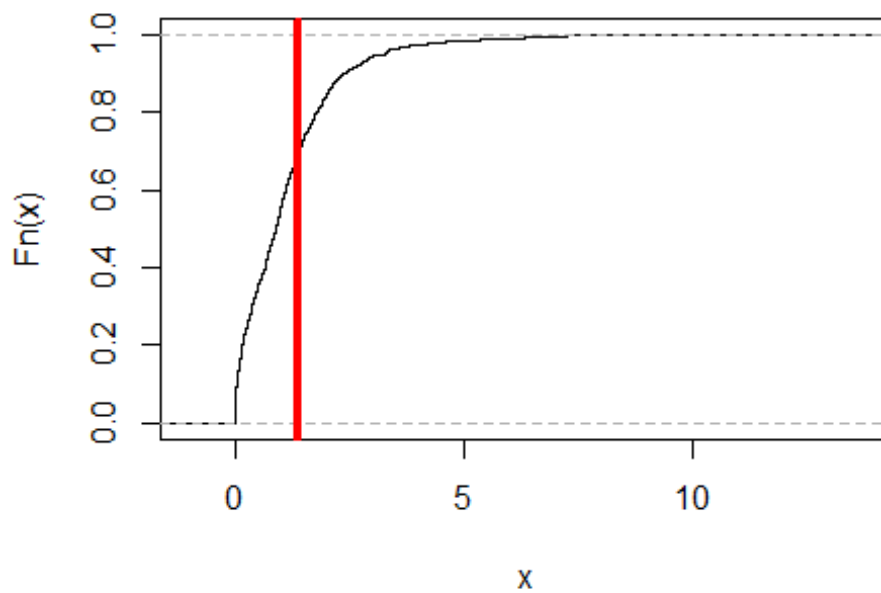


```

plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)

```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P

## [1] 0.308

# We remove Heart_Rate_Accuracy_Percent

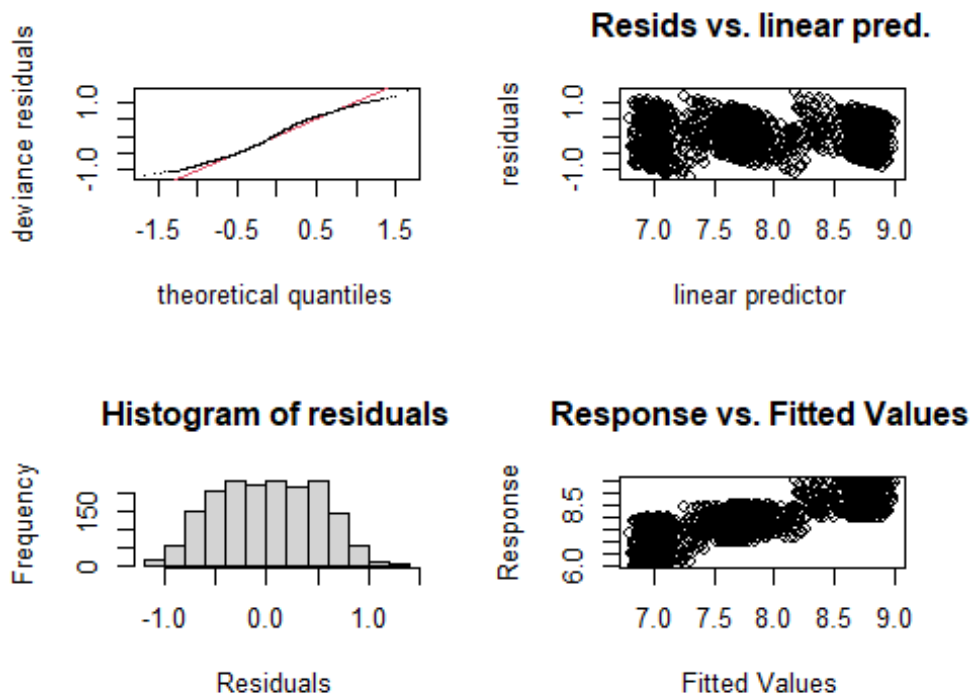
# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(GPS_Accuracy_Meters, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

summary(gam_fit)

##
## Family: gaussian
## Link function: identity
```

```
##
## Formula:
## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
##   s(GPS_Accuracy_Meters, bs = "cr") + Water_Resistance_Rating +
##   Connectivity_Features + App_Ecosystem_Support + s(Price_USD,
##   bs = "cr") + Category + Brand
##
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.8832008  0.0259144 227.024 < 2e-16 ***
## Water_Resistance_Rating.L -0.1907640  0.1195841  -1.595  0.111
## Water_Resistance_Rating.Q  0.0951247  0.0694733   1.369  0.171
## Water_Resistance_Rating.C  0.0063685  0.0595954   0.107  0.915
## Water_Resistance_Rating^4 -0.0770614  0.0493717  -1.561  0.119
## Water_Resistance_Rating^5  0.0280397  0.0362112   0.774  0.439
## Water_Resistance_Rating^6 -0.0278885  0.0377423  -0.739  0.460
## Connectivity_Features.L -0.5919549  0.0209043 -28.317 < 2e-16 ***
## Connectivity_Features.Q -0.0001108  0.0278862  -0.004  0.997
## Connectivity_Features.C  0.3681575  0.0284736  12.930 < 2e-16 ***
## App_Ecosystem_Support.L  0.8142511  0.0346131  23.524 < 2e-16 ***
## App_Ecosystem_Support.Q -0.5089225  0.0282060 -18.043 < 2e-16 ***
## CategoryFitness Tracker  0.9858156  0.0622123  15.846 < 2e-16 ***
## CategorySmart Ring      0.5248716  0.0386332  13.586 < 2e-16 ***
## CategorySmartwatch      2.2474948  0.0240168  93.580 < 2e-16 ***
## CategorySports Watch    1.4919236  0.0402016  37.111 < 2e-16 ***
## BrandApple              1.1775377  0.0268635  43.834 < 2e-16 ***
## BrandFitbit              0.0563120  0.0569255   0.989  0.323
## BrandGarmin              0.1308051  0.0859823   1.521  0.128
## BrandHuawei              -0.0399048  0.0661653  -0.603  0.547
## BrandOura                0.5248716  0.0386332  13.586 < 2e-16 ***
## BrandPolar               0.0205176  0.0639933   0.321  0.749
## BrandSamsung             0.0274948  0.0739969   0.372  0.710
## BrandWHOO                0.6330952  0.1066286   5.937 3.48e-09 ***
## BrandWithings            -0.0023681  0.0642233  -0.037  0.971
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 1.2972 1.5462   3.651  0.124
## s(GPS_Accuracy_Meters)          0.4459 0.4459 212.502 <2e-16 ***
## s(Price_USD)                    8.6117 8.9395 185.334 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 47/52
## R-sq.(adj) = 0.664 Deviance explained = 67%
## GCV = 0.2391 Scale est. = 0.23496 n = 1782
```

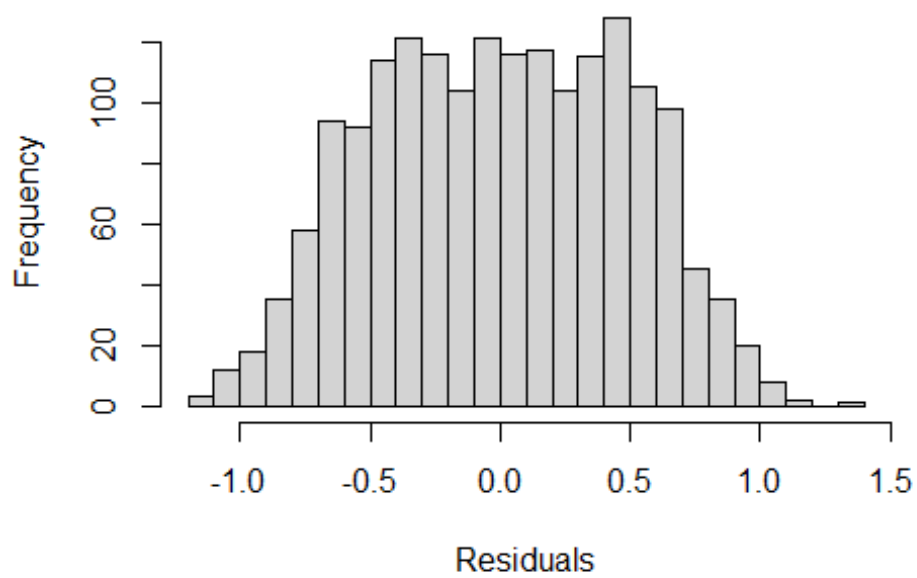
```
gam.check(gam_fit)
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 18 iterations.
## The RMS GCV score gradient at convergence was 8.792464e-08 .
## The Hessian was positive definite.
## Model rank =  47 / 52
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##          k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.000 1.297    0.99  0.300
## s(GPS_Accuracy_Meters)         9.000 0.446    1.00  0.590
## s(Price_USD)                   9.000 8.612    0.96  0.015 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

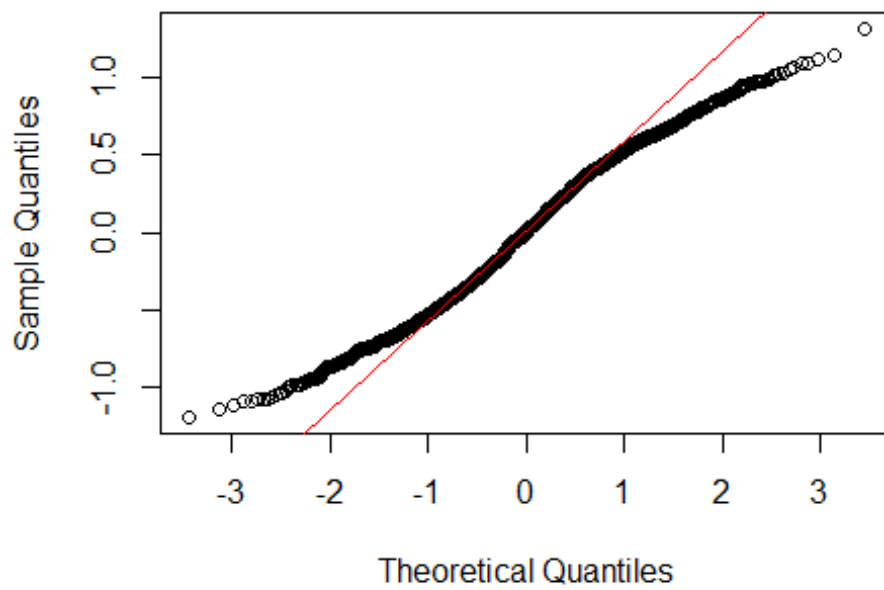
# Residual Diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```

Histogram of GAM residuals



```
qqnorm(res)  
qqline(res, col = "red")
```

Normal Q-Q Plot



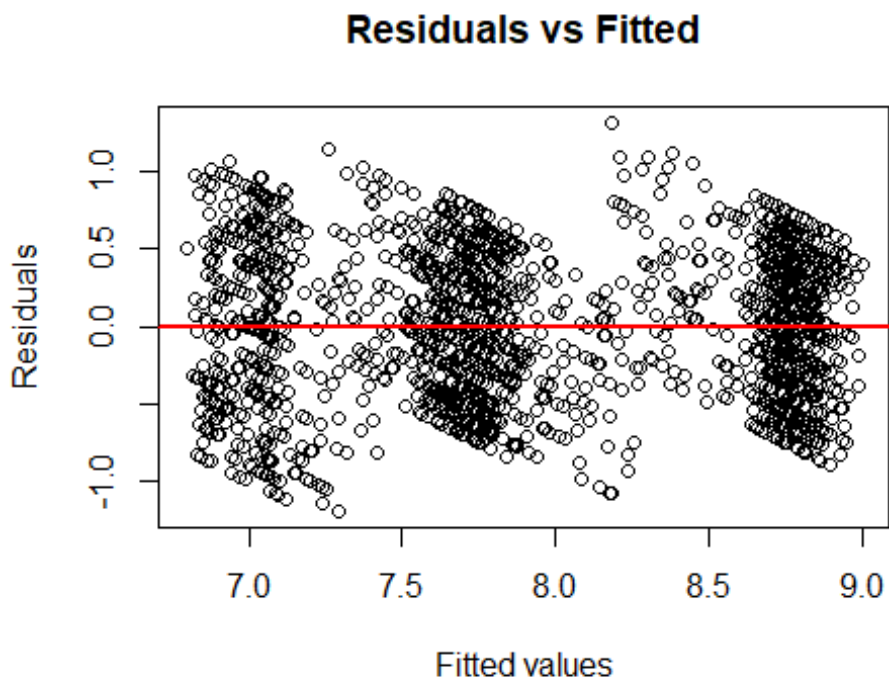
```

fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)

```



```

# Permutation test for GAM
# H0: GPS_Accuracy_Meters = 0

# Observed statistic for GPS_Accuracy_Meters
T0 = abs(summary(gam_fit)$s.table[2, 3])

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

```

```

)

res.H0 = gam_H0$residuals

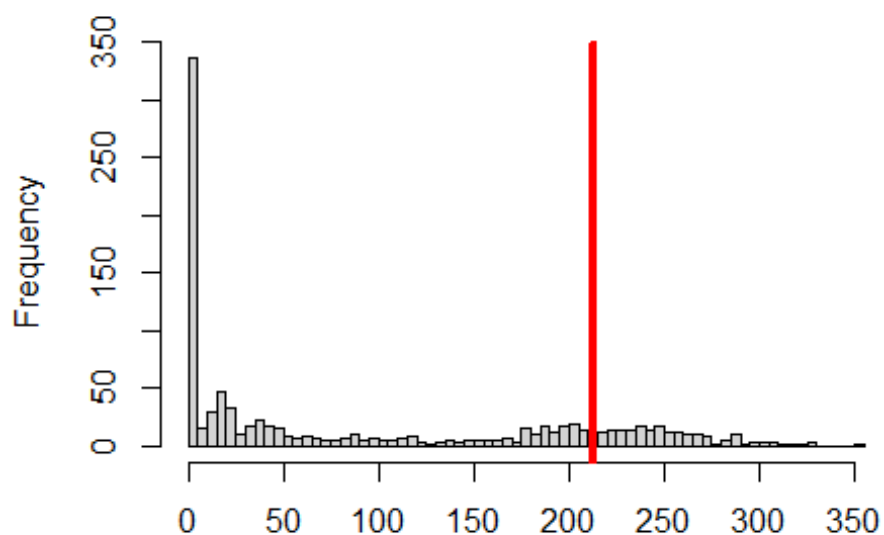
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      s(GPS_Accuracy_Meters, bs="cr") +
      Water_Resistance_Rating +
      Connectivity_Features +
      App_Ecosystem_Support +
      s(Price_USD, bs="cr") +
      Category +
      Brand,
    data = df_train
  )
  return(abs(summary(gam.perm)$s.table[2, 3]))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

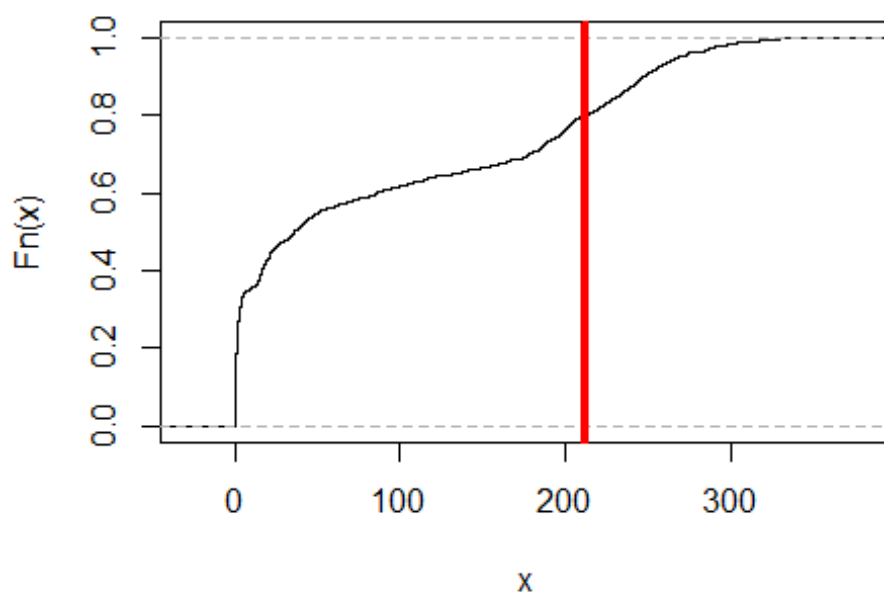
```


Permutational distribution of test statistics



```
plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')  
abline(v = T0, col = 'red', lwd = 4)
```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B  
P
```

```
## [1] 0.202

# We remove GPS_Accuracy_Meters

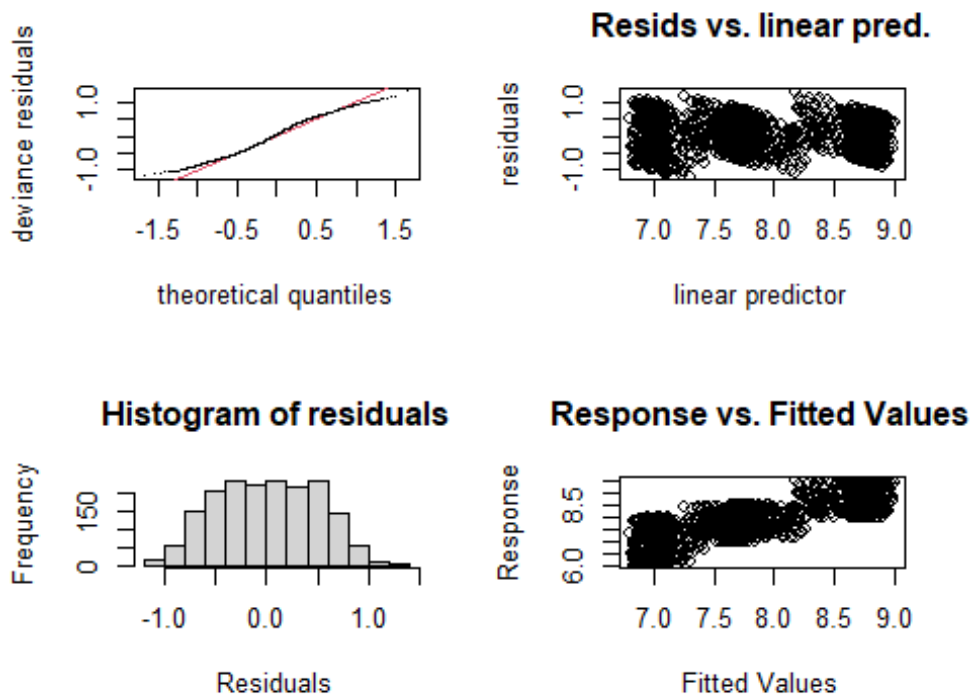
# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Water_Resistance_Rating +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

summary(gam_fit)

##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
##   Water_Resistance_Rating + Connectivity_Features + App_Ecosystem_Support +
##   s(Price_USD, bs = "cr") + Category + Brand
##
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.949e+00  2.666e-02 223.106 < 2e-16 ***
## Water_Resistance_Rating.L -1.908e-01  1.196e-01  -1.595    0.111
## Water_Resistance_Rating.Q  9.513e-02  6.947e-02   1.369    0.171
## Water_Resistance_Rating.C  6.360e-03  5.960e-02   0.107    0.915
## Water_Resistance_Rating^4 -7.708e-02  4.937e-02  -1.561    0.119
## Water_Resistance_Rating^5  2.802e-02  3.621e-02   0.774    0.439
## Water_Resistance_Rating^6 -2.791e-02  3.774e-02  -0.739    0.460
## Connectivity_Features.L  -7.256e-01  3.200e-02 -22.678 < 2e-16 ***
## Connectivity_Features.Q  -6.917e-05  2.789e-02  -0.002    0.998
## Connectivity_Features.C   4.350e-01  3.167e-02  13.736 < 2e-16 ***
## App_Ecosystem_Support.L   8.235e-01  3.457e-02  23.822 < 2e-16 ***
## App_Ecosystem_Support.Q  -5.142e-01  2.817e-02 -18.255 < 2e-16 ***
## CategoryFitness Tracker   1.165e+00  6.465e-02  18.023 < 2e-16 ***
## CategorySmart Ring        6.146e-01  3.511e-02  17.505 < 2e-16 ***
## CategorySmartwatch        2.131e+00  3.392e-02  62.824 < 2e-16 ***
## CategorySports Watch     1.226e+00  6.641e-02  18.458 < 2e-16 ***
## BrandApple                1.191e+00  2.701e-02  44.073 < 2e-16 ***
## BrandFitbit               5.635e-02  5.693e-02   0.990    0.322
## BrandGarmin               1.307e-01  8.602e-02   1.520    0.129
```

```
## BrandHuawei      -3.982e-02  6.617e-02  -0.602    0.547
## BrandOura        6.146e-01  3.511e-02  17.505 < 2e-16 ***
## BrandPolar       2.049e-02  6.399e-02   0.320    0.749
## BrandSamsung     2.750e-02  7.400e-02   0.372    0.710
## BrandWHOO        8.123e-01  1.202e-01   6.757 1.91e-11 ***
## BrandWithings    -2.331e-03  6.422e-02  -0.036    0.971
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 1.300  1.551   3.647   0.124
## s(Price_USD)                   8.613  8.940 185.313 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 39/43
## R-sq.(adj) =  0.664  Deviance explained =  67%
## GCV = 0.23911  Scale est. = 0.23496    n = 1782
```

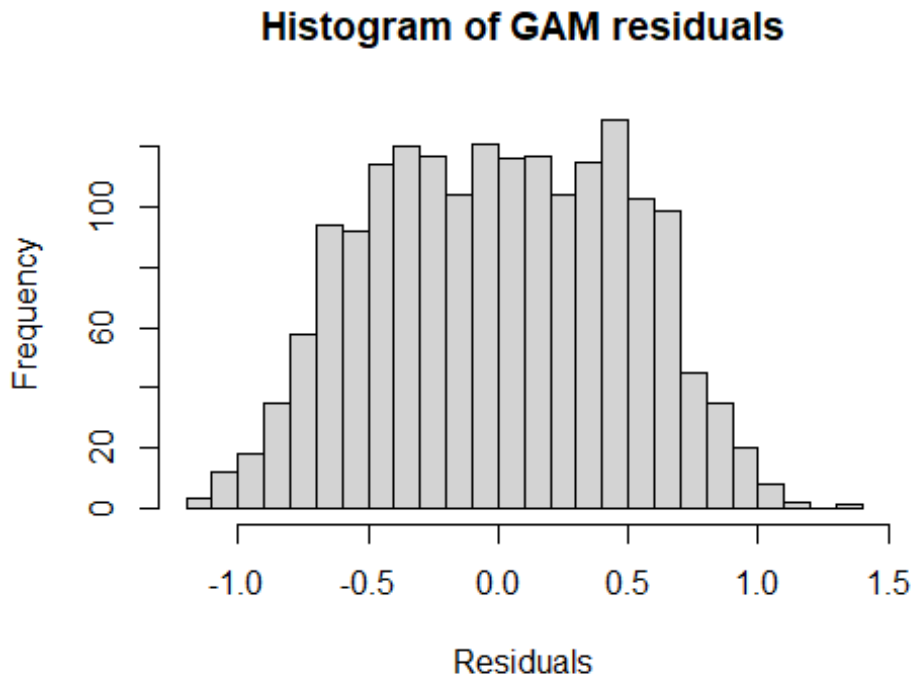
```
gam.check(gam_fit)
```



```
##
## Method: GCV  Optimizer: magic
## Smoothing parameter selection converged after 11 iterations.
## The RMS GCV score gradient at convergence was 1.10948e-06 .
## The Hessian was positive definite.
## Model rank =  39 / 43
```

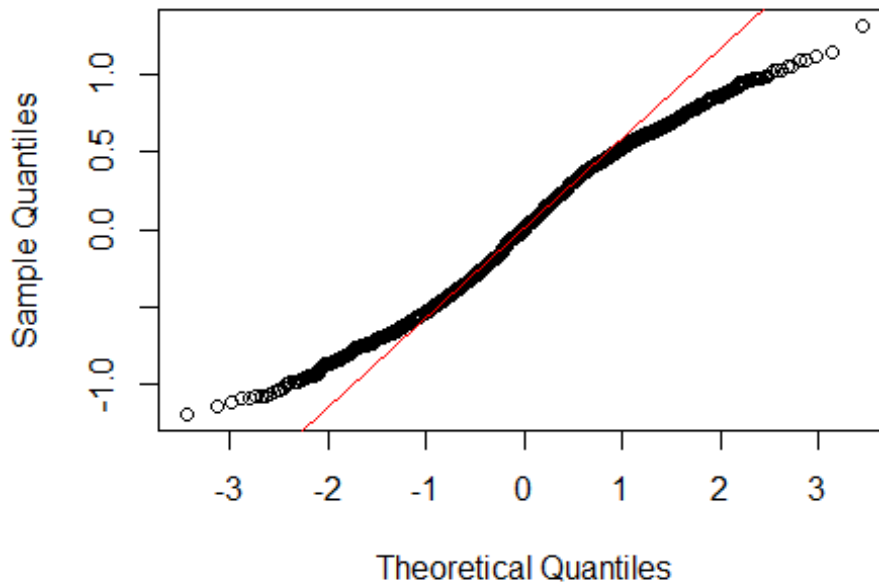
```
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##               k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.00 1.30    0.99  0.350
## s(Price_USD)                   9.00 8.61    0.96  0.025 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```



```
qqnorm(res)
qqline(res, col = "red")
```

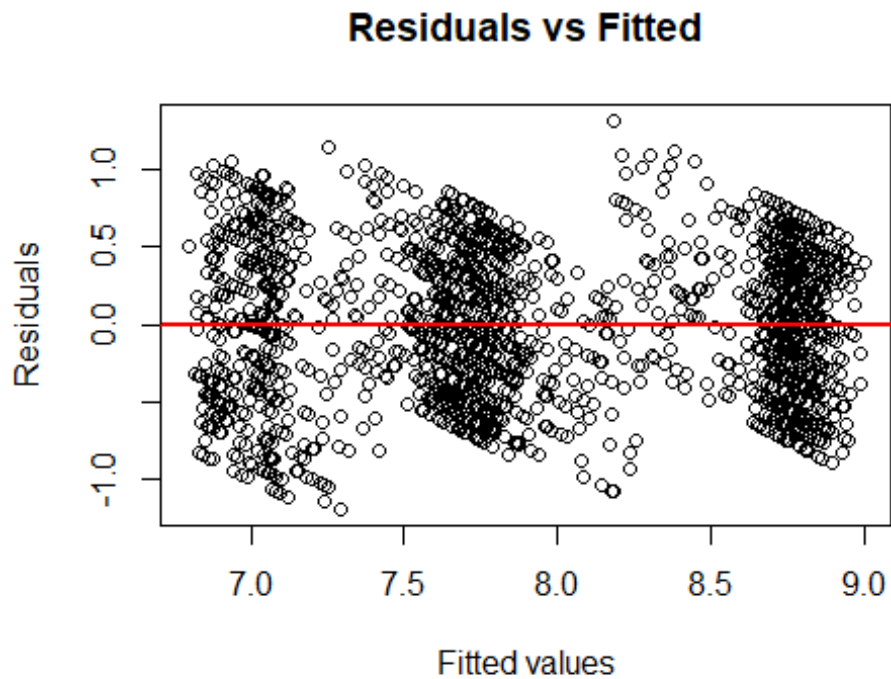
Normal Q-Q Plot



```
fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# permutation test for GAM
# H0: Water_Resistance_Rating = 0

# Observed statistic for Water_Resistance_Rating
T0 = max(abs(summary(gam_fit)$p.table[2:7, "t value"]))

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

res.H0 = gam_H0$residuals

wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      Water_Resistance_Rating +
```

```

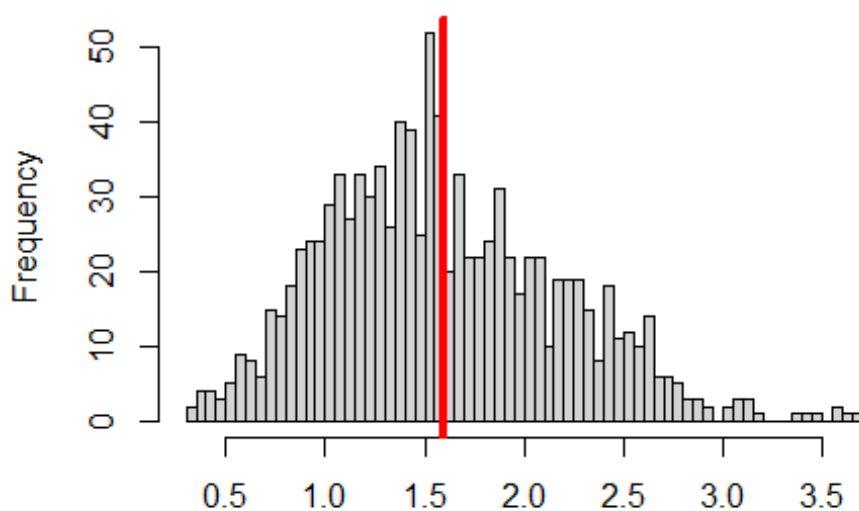
Connectivity_Features +
App_Ecosystem_Support +
s(Price_USD, bs="cr") +
Category +
Brand,
data = df_train
)
return(max(abs(summary(gam.perm)$p.table[2:7, "t value"])))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

Permutational distribution of test statistics

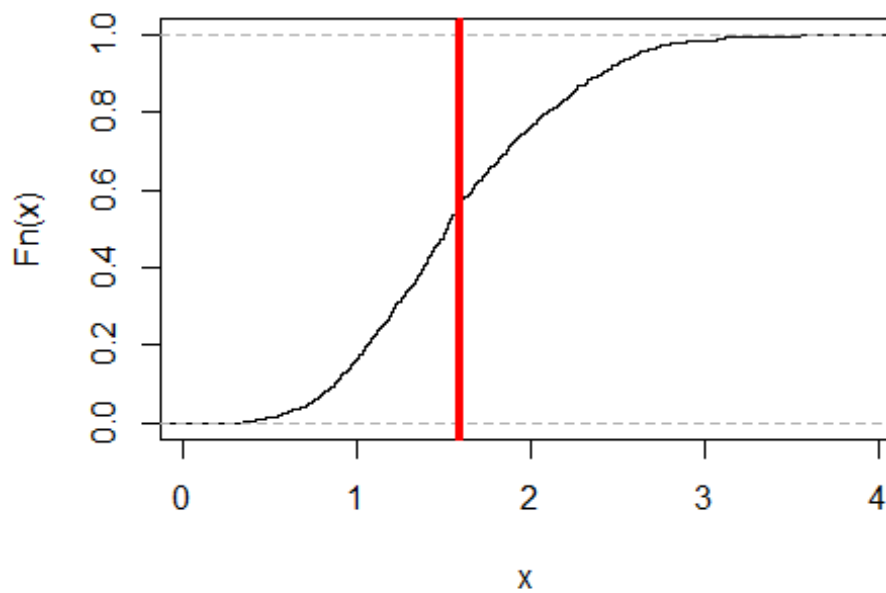


```

plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)

```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P

## [1] 0.436

# we remove Water_Resistance_Rating

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

summary(gam_fit)

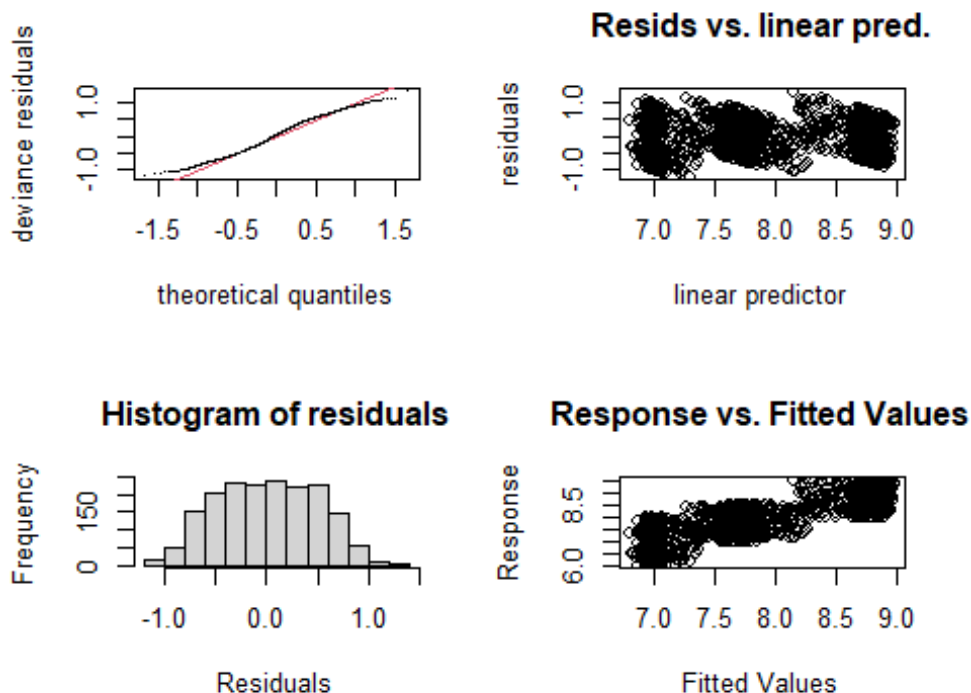
##
## Family: gaussian
## Link function: identity
##
## Formula:
```



```

## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
##   Connectivity_Features + App_Ecosystem_Support + s(Price_USD,
##   bs = "cr") + Category + Brand
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.946164   0.024289 244.804 < 2e-16 ***
## Connectivity_Features.L -0.759410   0.023201 -32.731 < 2e-16 ***
## Connectivity_Features.Q  0.003268   0.027841   0.117   0.907
## Connectivity_Features.C  0.448608   0.029681  15.114 < 2e-16 ***
## App_Ecosystem_Support.L  0.821963   0.034563  23.781 < 2e-16 ***
## App_Ecosystem_Support.Q -0.513874   0.028151 -18.254 < 2e-16 ***
## CategoryFitness Tracker  1.231016   0.052300  23.537 < 2e-16 ***
## CategorySmart Ring      0.580892   0.033077  17.562 < 2e-16 ***
## CategorySmartwatch      2.093222   0.022808  91.776 < 2e-16 ***
## CategorySports Watch    1.150512   0.044535  25.834 < 2e-16 ***
## BrandApple             1.191051   0.026993  44.124 < 2e-16 ***
## BrandFitbit            0.055552   0.056942   0.976   0.329
## BrandGarmin            0.127604   0.085748   1.488   0.137
## BrandHuawei            -0.041684   0.066176  -0.630   0.529
## BrandOura              0.580892   0.033077  17.562 < 2e-16 ***
## BrandPolar            0.020607   0.064023   0.322   0.748
## BrandSamsung           0.028489   0.074019   0.385   0.700
## BrandWHOOP            0.890522   0.113494   7.846 7.38e-15 ***
## BrandWithings          0.001572   0.064205   0.024   0.980
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 1.351  1.636   3.923   0.105
## s(Price_USD)                   8.607  8.938 185.687 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 33/37
## R-sq.(adj) = 0.664   Deviance explained = 66.8%
## GCV = 0.23872   Scale est. = 0.23538   n = 1782
gam.check(gam_fit)

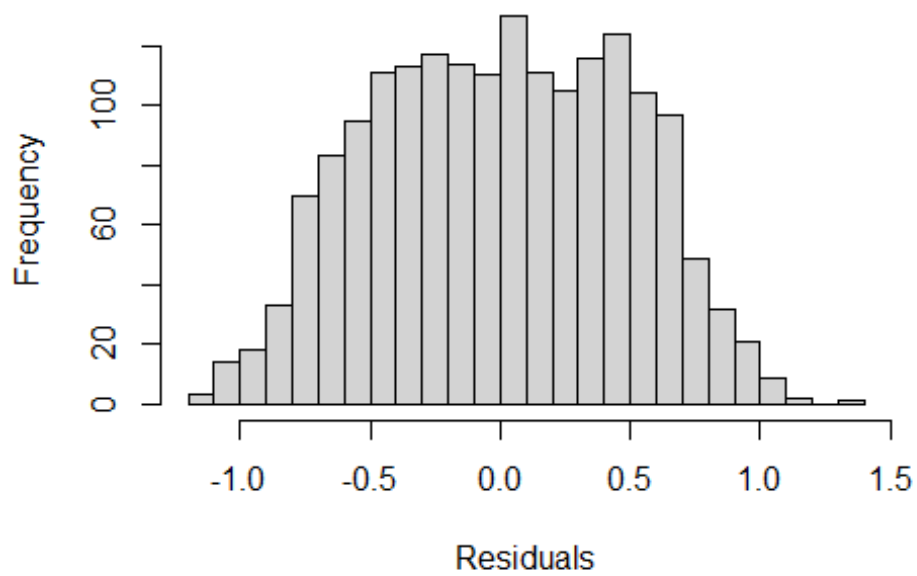
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 12 iterations.
## The RMS GCV score gradient at convergence was 2.774756e-07 .
## The Hessian was positive definite.
## Model rank = 33 / 37
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##          k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.00 1.35    0.99  0.295
## s(Price_USD)                   9.00 8.61    0.95  0.015 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

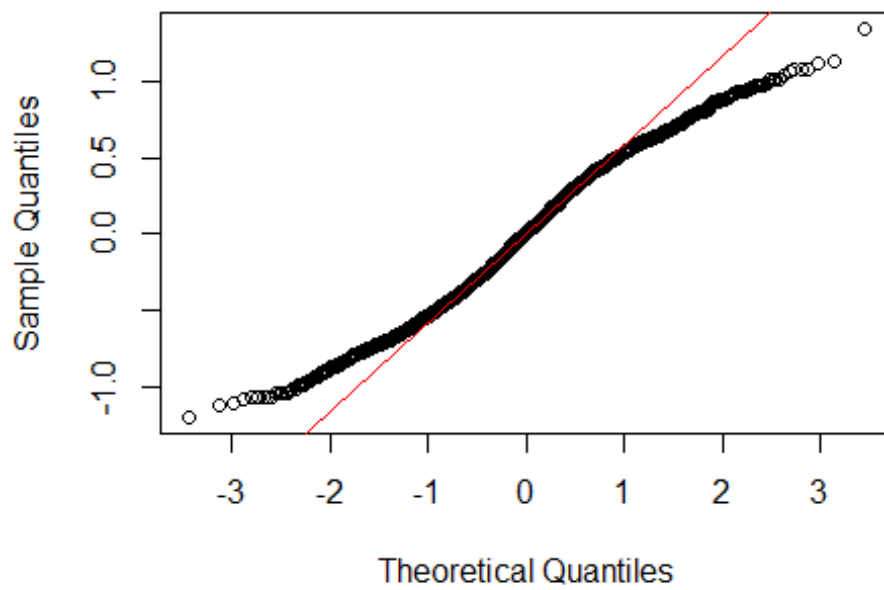
# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```

Histogram of GAM residuals



```
qqnorm(res)  
qqline(res, col = "red")
```

Normal Q-Q Plot



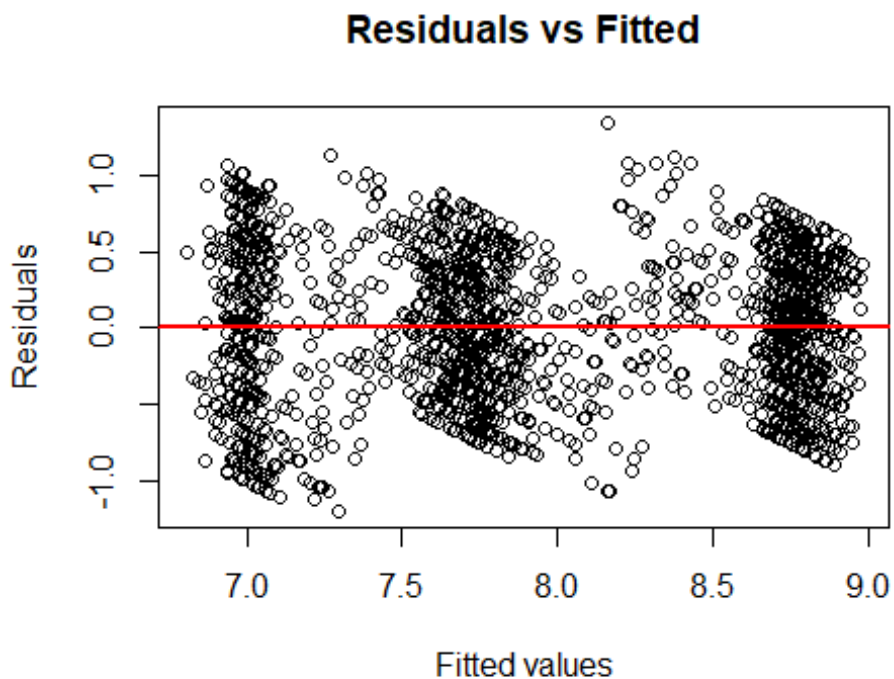
```

fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)

```



```

# permutation test for GAM
# H0: Step_Count_Accuracy_Percent = 0

# Observed statistic for Step_Count_Accuracy_Percent
T0 = abs(summary(gam_fit)$s.table[1, 3])

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

```

```

res.H0 = gam_H0$residuals

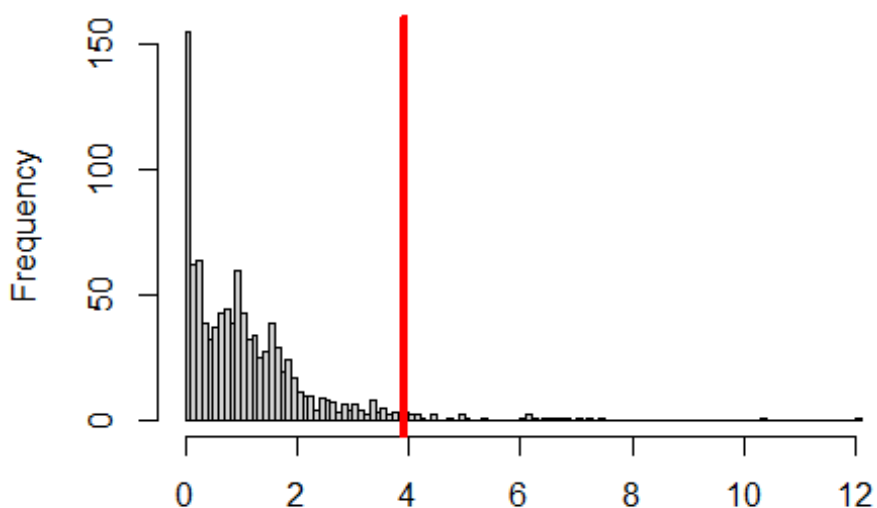
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      Connectivity_Features +
      App_Ecosystem_Support +
      s(Price_USD, bs="cr") +
      Category +
      Brand,
    data = df_train
  )
  return(abs(summary(gam.perm)$s.table[1, 3]))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

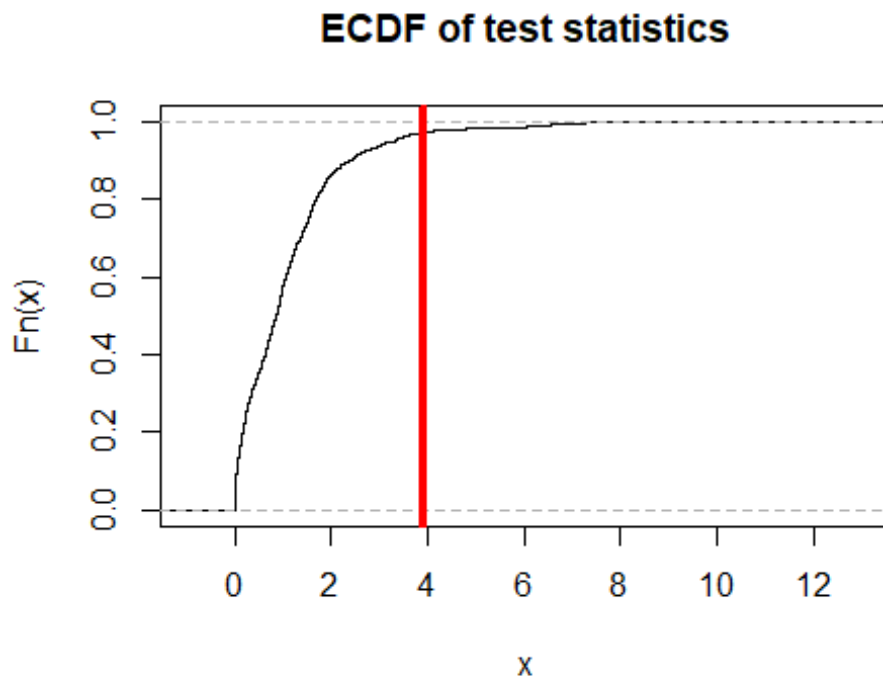
hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

Permutational distribution of test statistics



```
plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)
```



```
P = sum(T_H0 >= T0) / B
P

## [1] 0.029

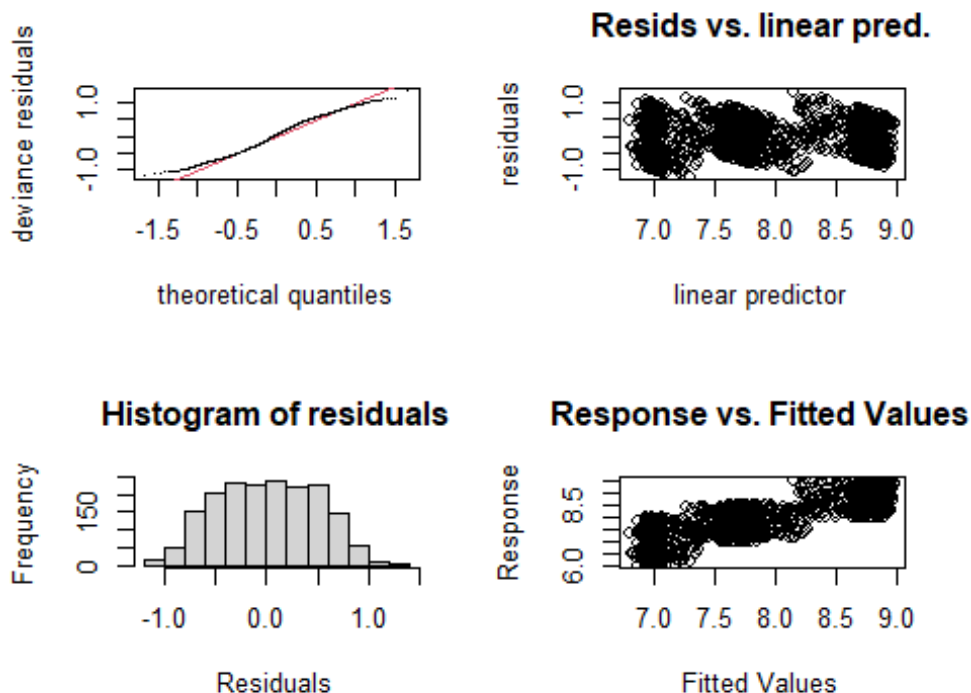
# we keep Step_Count_Accuracy_Percent

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category +
    Brand,
  data = df_train
)

summary(gam_fit)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
##   Connectivity_Features + App_Ecosystem_Support + s(Price_USD,
##   bs = "cr") + Category + Brand
##
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.946164   0.024289 244.804 < 2e-16 ***
## Connectivity_Features.L -0.759410   0.023201 -32.731 < 2e-16 ***
## Connectivity_Features.Q  0.003268   0.027841   0.117   0.907
## Connectivity_Features.C  0.448608   0.029681  15.114 < 2e-16 ***
## App_Ecosystem_Support.L  0.821963   0.034563  23.781 < 2e-16 ***
## App_Ecosystem_Support.Q -0.513874   0.028151 -18.254 < 2e-16 ***
## CategoryFitness Tracker  1.231016   0.052300  23.537 < 2e-16 ***
## CategorySmart Ring      0.580892   0.033077  17.562 < 2e-16 ***
## CategorySmartwatch      2.093222   0.022808  91.776 < 2e-16 ***
## CategorySports Watch    1.150512   0.044535  25.834 < 2e-16 ***
## BrandApple             1.191051   0.026993  44.124 < 2e-16 ***
## BrandFitbit            0.055552   0.056942   0.976   0.329
## BrandGarmin            0.127604   0.085748   1.488   0.137
## BrandHuawei            -0.041684   0.066176  -0.630   0.529
## BrandOura              0.580892   0.033077  17.562 < 2e-16 ***
## BrandPolar             0.020607   0.064023   0.322   0.748
## BrandSamsung           0.028489   0.074019   0.385   0.700
## BrandWHOOP            0.890522   0.113494   7.846 7.38e-15 ***
## BrandWithings          0.001572   0.064205   0.024   0.980
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 1.351  1.636   3.923   0.105
## s(Price_USD)                   8.607  8.938 185.687 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 33/37
## R-sq.(adj) = 0.664   Deviance explained = 66.8%
## GCV = 0.23872   Scale est. = 0.23538   n = 1782

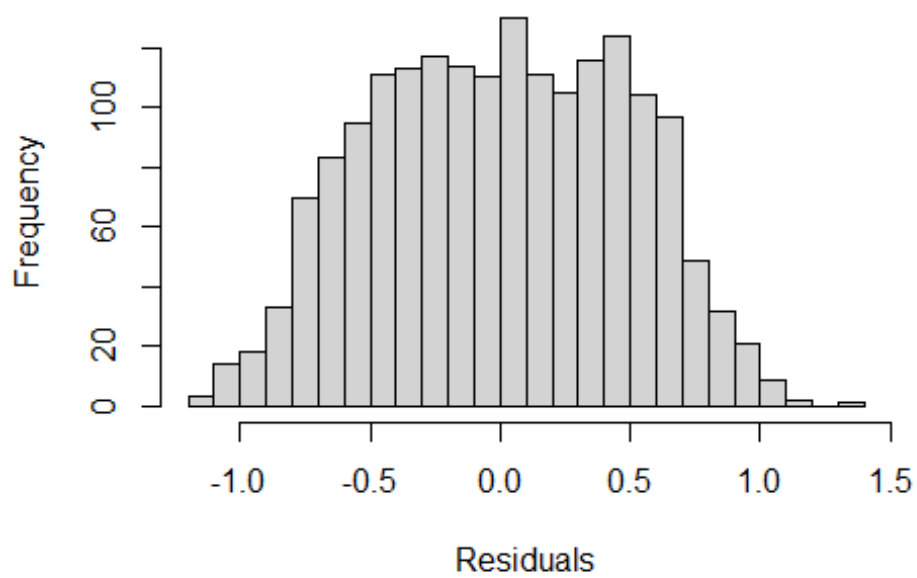
gam.check(gam_fit)
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 12 iterations.
## The RMS GCV score gradient at convergence was 2.774756e-07 .
## The Hessian was positive definite.
## Model rank = 33 / 37
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##          k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.00 1.35    0.99    0.270
## s(Price_USD)                   9.00 8.61    0.95    0.005 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

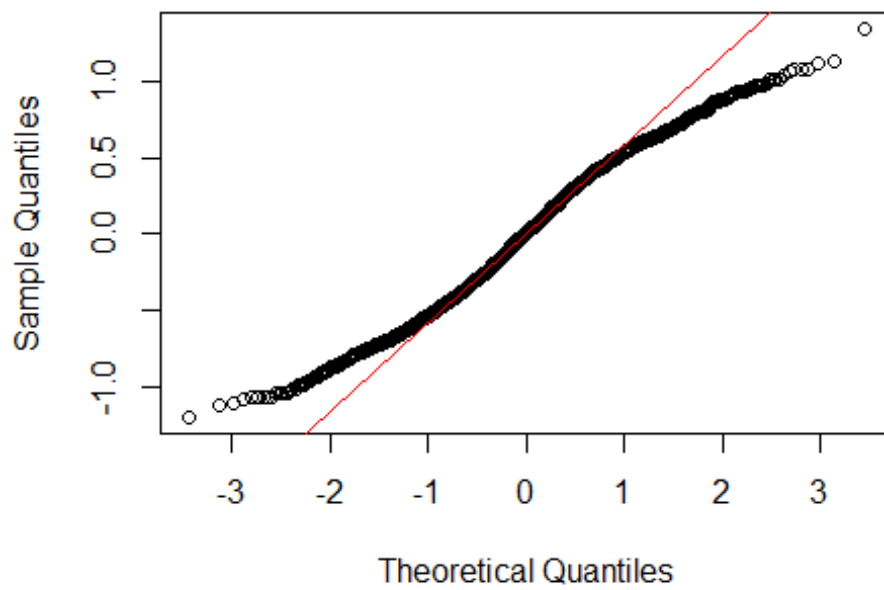
# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```


Histogram of GAM residuals



```
qqnorm(res)  
qqline(res, col = "red")
```

Normal Q-Q Plot



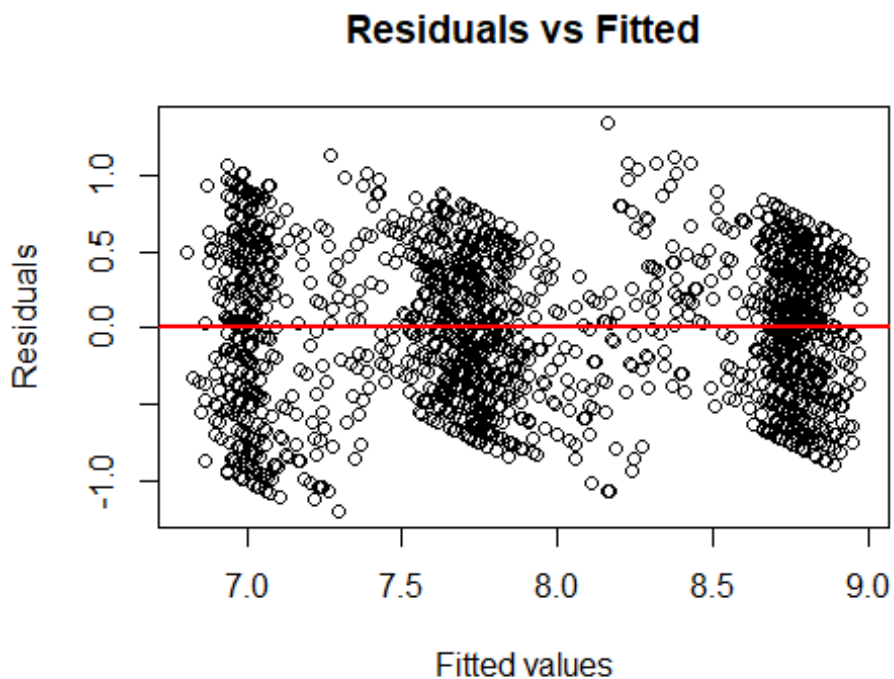
```

fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)

```



```

# Permutation test for GAM
# H0: Brand = 0

# Observed statistic for Brand
T0 = max(abs(summary(gam_fit)$p.table[11:19, "t value"]))

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

```

```

res.H0 = gam_H0$residuals

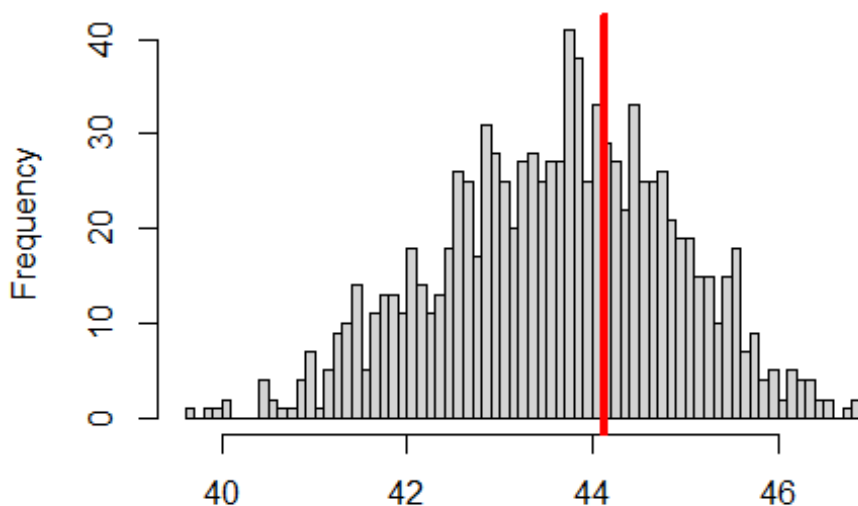
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      Connectivity_Features +
      App_Ecosystem_Support +
      s(Price_USD, bs="cr") +
      Category +
      Brand,
    data = df_train
  )
  return(max(abs(summary(gam.perm)$p.table[11:19, "t value"])))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

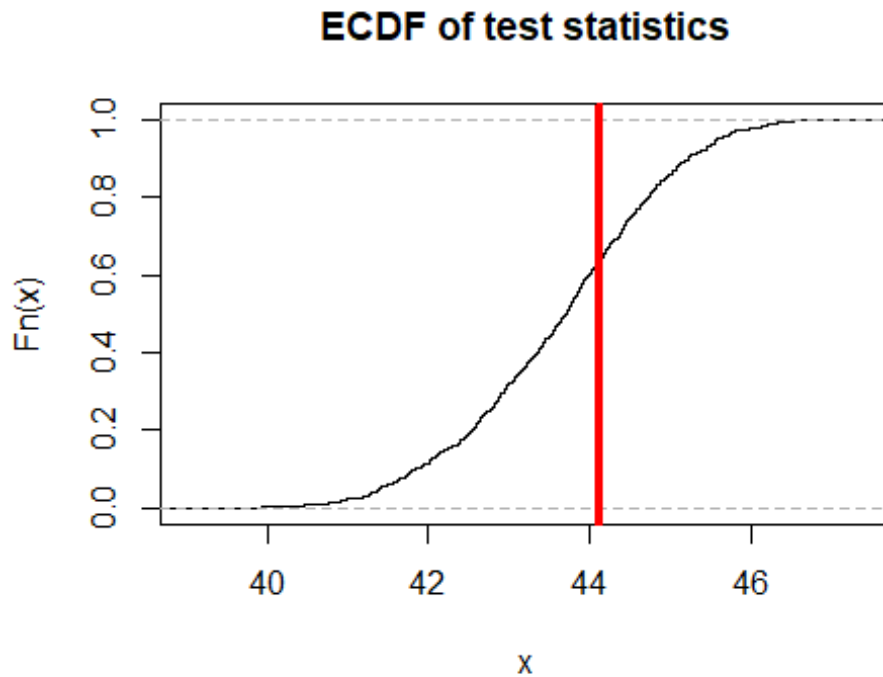
hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

Permutational distribution of test statistics



```
plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)
```



```
P = sum(T_H0 >= T0) / B
P

## [1] 0.357

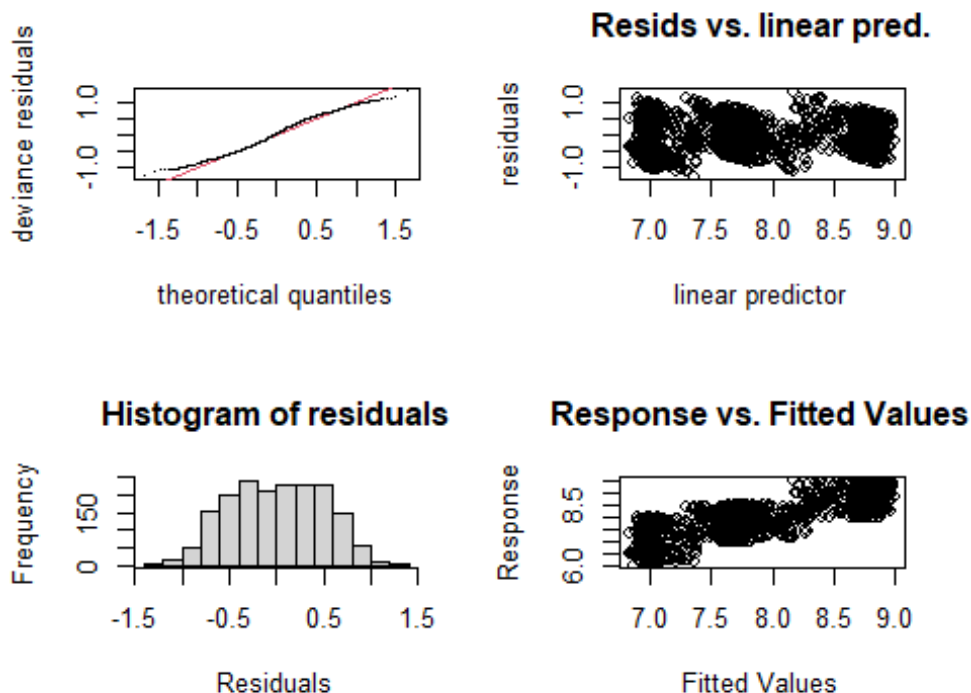
# We remove Brand

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Connectivity_Features +
    App_Ecosystem_Support +
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

summary(gam_fit)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
##   Connectivity_Features + App_Ecosystem_Support + s(Price_USD,
##   bs = "cr") + Category
##
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    6.900718   0.086773  79.526   <2e-16 ***
## Connectivity_Features.L -1.374515   0.084967 -16.177   <2e-16 ***
## Connectivity_Features.Q  0.003594   0.027777   0.129   0.8971
## Connectivity_Features.C  0.754673   0.052594  14.349   <2e-16 ***
## App_Ecosystem_Support.L  0.011176   0.030973   0.361   0.7183
## App_Ecosystem_Support.Q -0.030178   0.023586  -1.280   0.2009
## CategoryFitness Tracker  0.362603   0.151367   2.396   0.0167 *
## CategorySmart Ring      0.266656   0.152624   1.747   0.0808 .
## CategorySmartwatch      1.883455   0.053311  35.329   <2e-16 ***
## CategorySports Watch    0.269332   0.149598   1.800   0.0720 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 2.009  2.515   2.186   0.108
## s(Price_USD)                   8.620  8.942 236.080 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 27/28
## R-sq.(adj) = 0.664   Deviance explained = 66.8%
## GCV = 0.23759   Scale est. = 0.23497   n = 1782
```

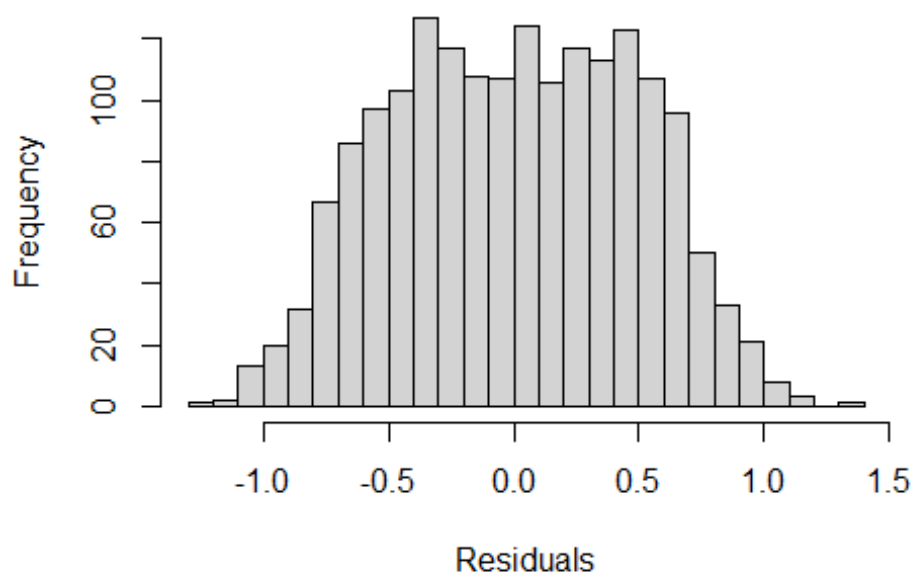
```
gam.check(gam_fit)
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 11 iterations.
## The RMS GCV score gradient at convergence was 4.101849e-08 .
## The Hessian was positive definite.
## Model rank = 27 / 28
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##          k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.00 2.01    0.99  0.355
## s(Price_USD)                   9.00 8.62    0.95  0.015 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

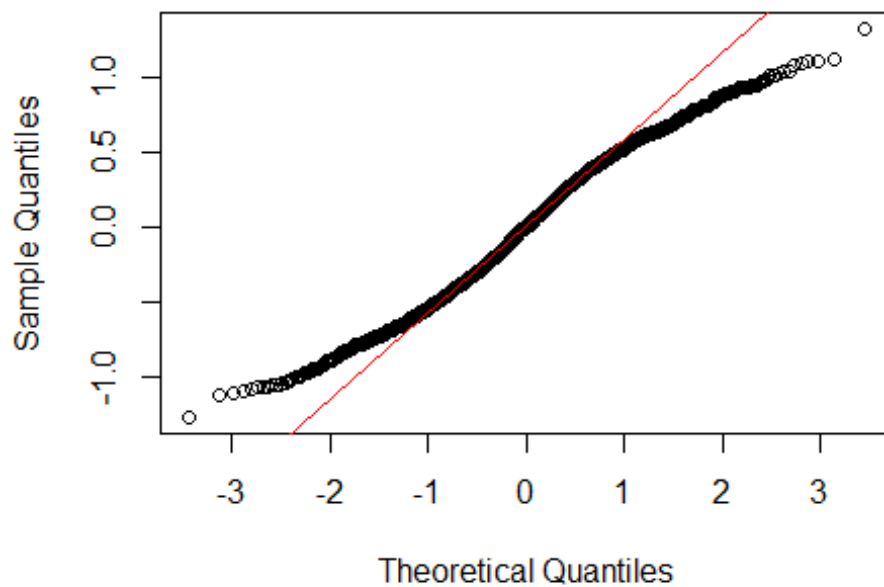
# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```

Histogram of GAM residuals



```
qqnorm(res)  
qqline(res, col = "red")
```

Normal Q-Q Plot



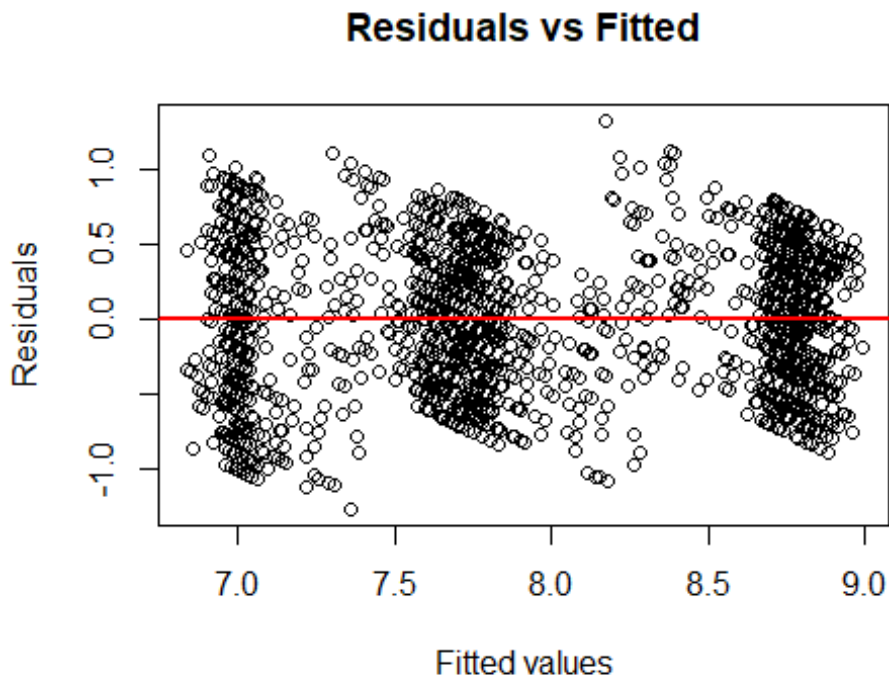
```

fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)

```



```

# permutation test for GAM
# H0: App_Ecosystem_Support = 0

# Observed statistic for App_Ecosystem_Support
T0 = max(abs(summary(gam_fit)$p.table[5:6, "t value"]))

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Connectivity_Features +
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

res.H0 = gam_H0$residuals

```



```

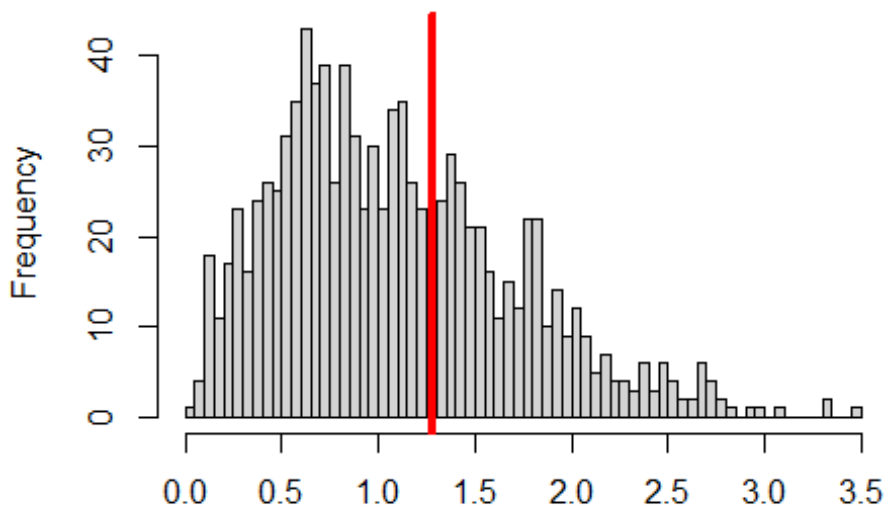
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      Connectivity_Features +
      App_Ecosystem_Support +
      s(Price_USD, bs="cr") +
      Category,
    data = df_train
  )
  return(max(abs(summary(gam.perm)$p.table[5:6, "t value"])))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

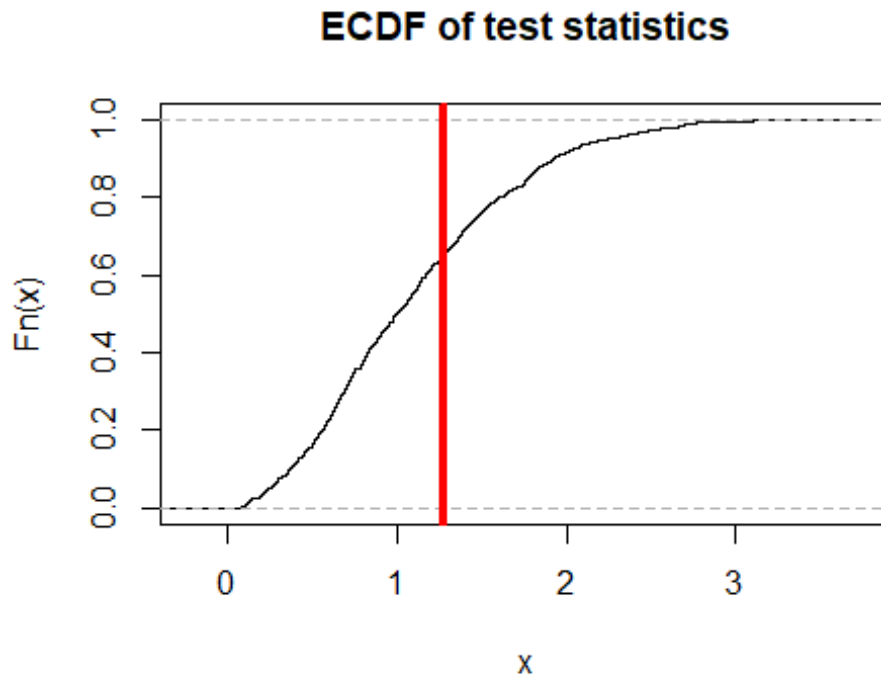
hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

Permutational distribution of test statistics



```
plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)
```



```
P = sum(T_H0 >= T0) / B
P

## [1] 0.349

# we remove App_Ecosystem_Support

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Connectivity_Features +
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

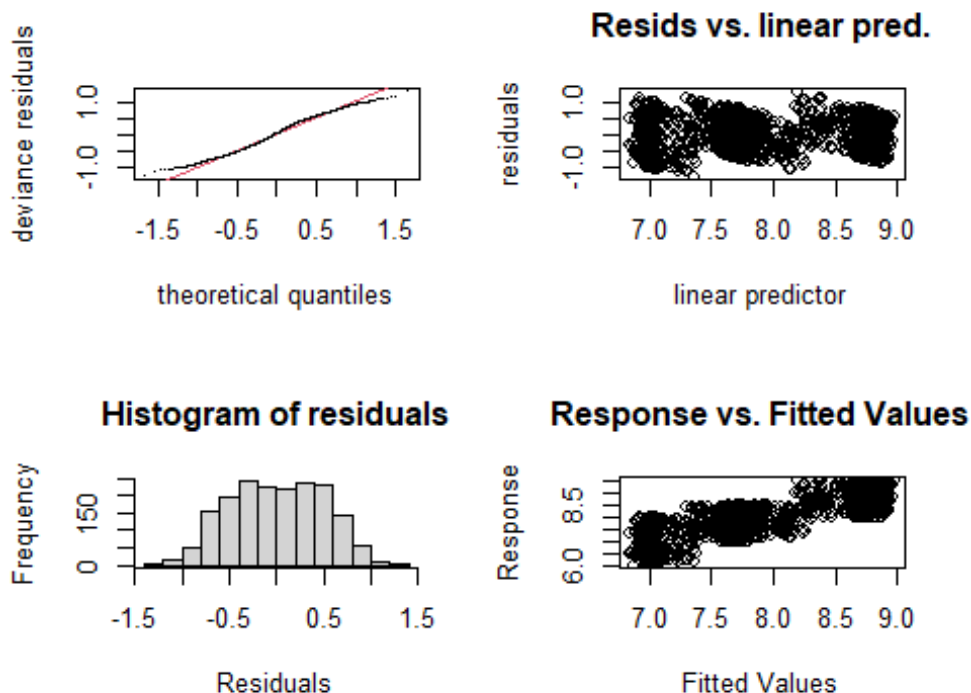
summary(gam_fit)

##
## Family: gaussian
## Link function: identity
##
```

```

## Formula:
## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
##   Connectivity_Features + s(Price_USD, bs = "cr") + Category
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      6.909604    0.086315  80.051   <2e-16 ***
## Connectivity_Features.L -1.380153    0.084853 -16.265   <2e-16 ***
## Connectivity_Features.Q  0.003351    0.027775   0.121   0.9040
## Connectivity_Features.C  0.758293    0.052514  14.440   <2e-16 ***
## CategoryFitness Tracker  0.349401    0.150876   2.316   0.0207 *
## CategorySmart Ring      0.262157    0.152528   1.719   0.0858 .
## CategorySmartwatch      1.881237    0.053270  35.315   <2e-16 ***
## CategorySports Watch    0.259019    0.149147   1.737   0.0826 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 1.990  2.495   2.179   0.102
## s(Price_USD)                   8.622  8.942 239.624   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 25/26
## R-sq.(adj) = 0.664   Deviance explained = 66.7%
## GCV = 0.23731   Scale est. = 0.23496   n = 1782
gam.check(gam_fit)

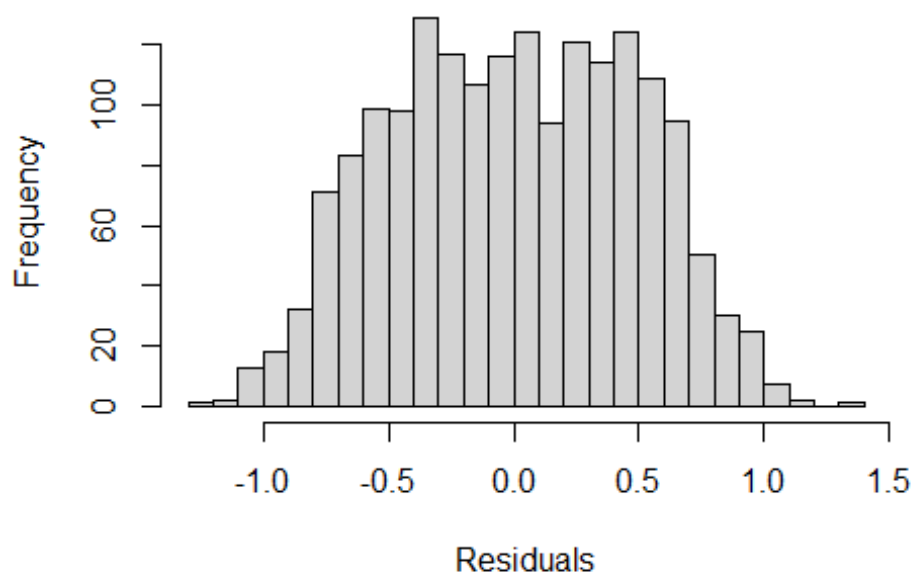
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 10 iterations.
## The RMS GCV score gradient at convergence was 2.839488e-06 .
## The Hessian was positive definite.
## Model rank = 25 / 26
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##          k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.00 1.99    0.99    0.285
## s(Price_USD)                   9.00 8.62    0.95    0.005 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

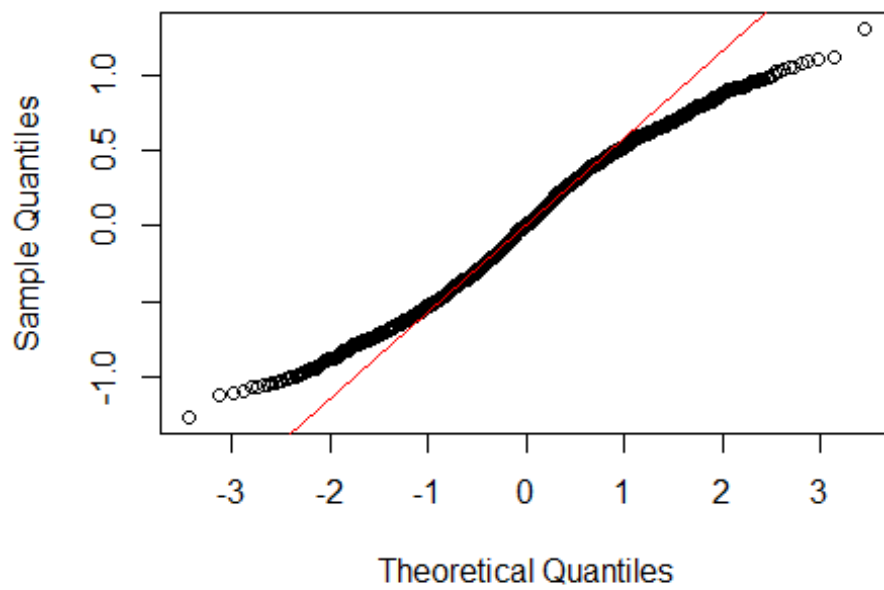
# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```

Histogram of GAM residuals



```
qqnorm(res)  
qqline(res, col = "red")
```

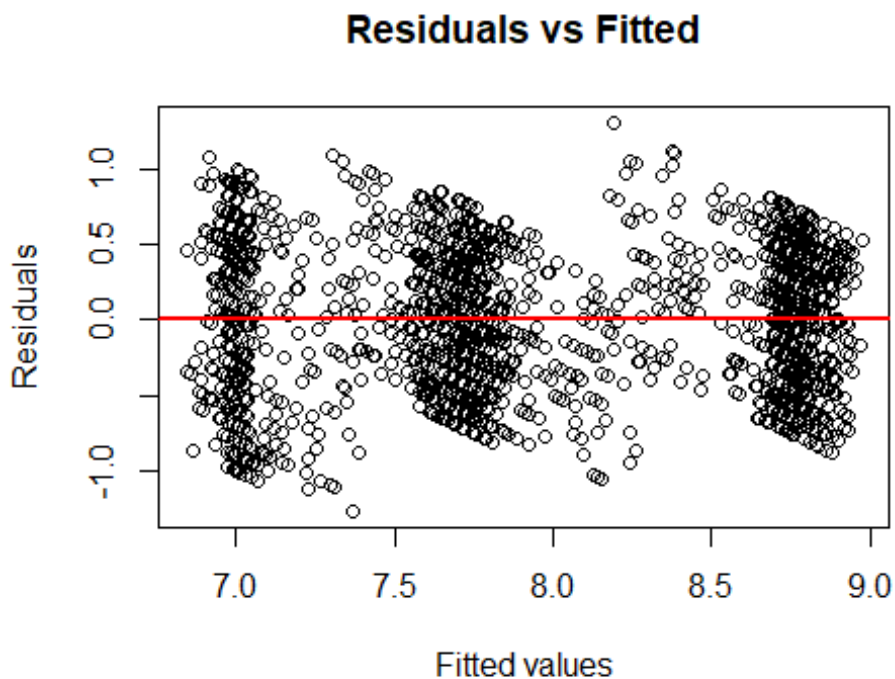
Normal Q-Q Plot



```
fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# permutation test for GAM
# H0: Connectivity_Features = 0

# Observed statistic for Connectivity_Features
T0 = max(abs(summary(gam_fit)$p.table[2:4, "t value"]))

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

res.H0 = gam_H0$residuals
```

```

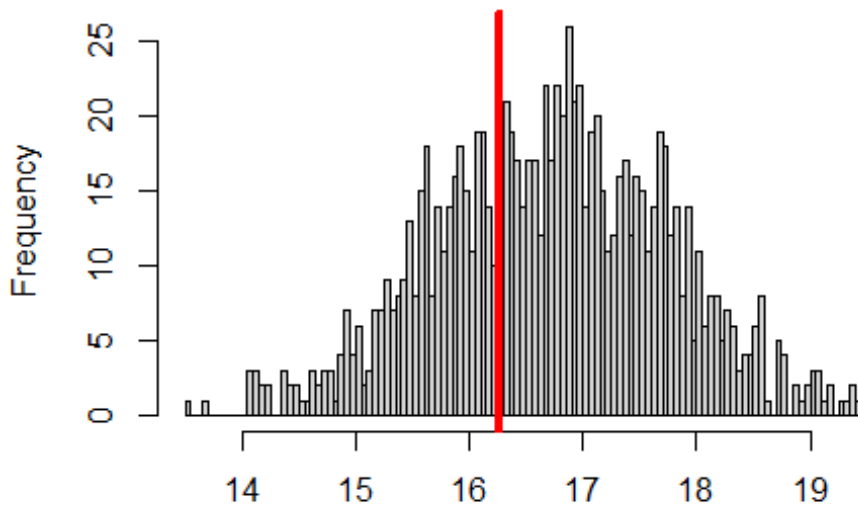
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      Connectivity_Features +
      s(Price_USD, bs="cr") +
      Category,
    data = df_train
  )
  return(max(abs(summary(gam.perm)$p.table[2:4, "t value"])))
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

Permutational distribution of test statistics

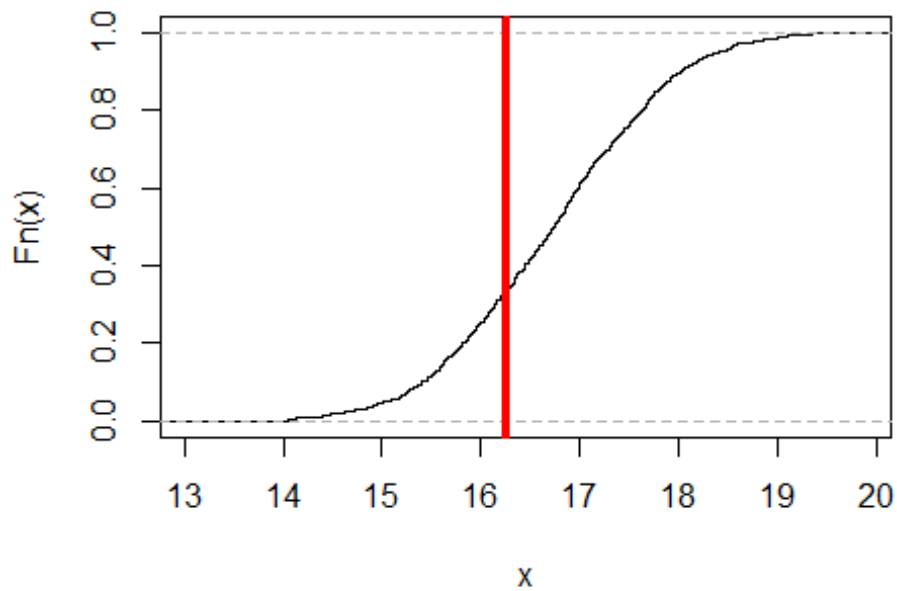


```

plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)

```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P

## [1] 0.665

# We remove Connectivity_Features

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

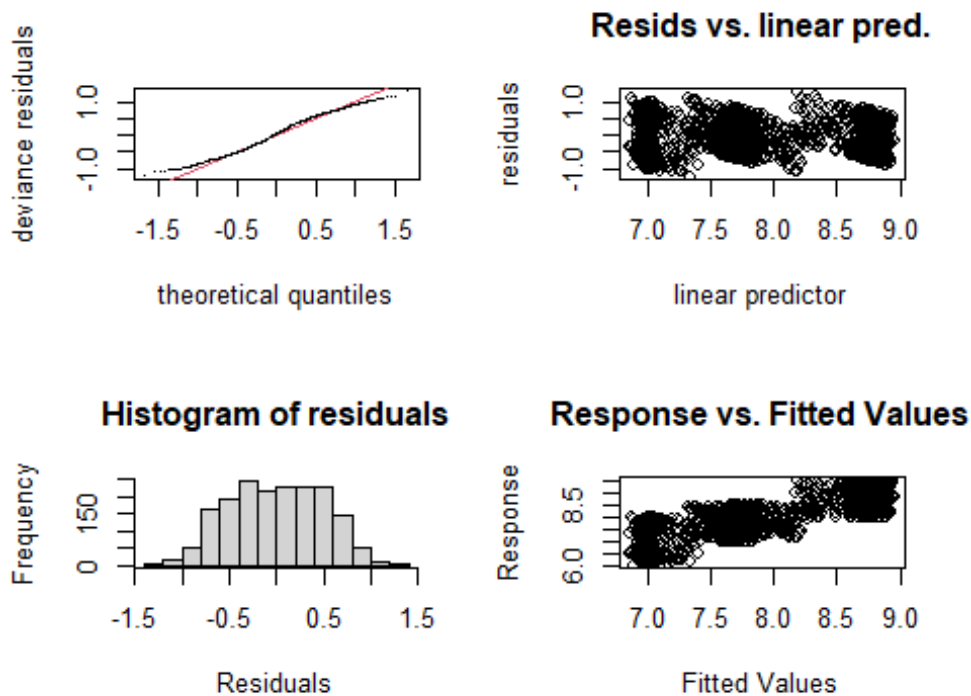
summary(gam_fit)

##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
##   s(Price_USD, bs = "cr") + Category
##
```



```
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)      7.6675     0.1316  58.284  <2e-16 ***
## CategoryFitness Tracker  0.3684     0.1498   2.459   0.0140 *
## CategorySmart Ring      0.2636     0.1526   1.728   0.0842 .
## CategorySmartwatch      0.3370     0.1467   2.297   0.0217 *
## CategorySports Watch    0.2794     0.1479   1.889   0.0591 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 2.005  2.513   2.14   0.106
## s(Price_USD)                   8.614  8.940 238.82  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.664   Deviance explained = 66.6%
## GCV = 0.23753   Scale est. = 0.23545   n = 1782
```

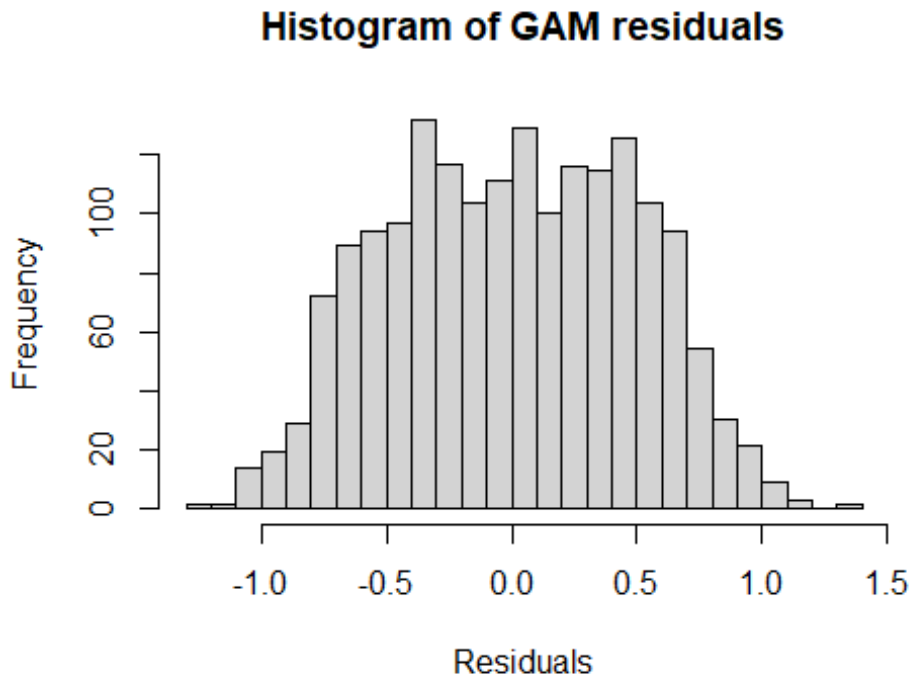
```
gam.check(gam_fit)
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 11 iterations.
## The RMS GCV score gradient at convergence was 3.428304e-08 .
## The Hessian was positive definite.
## Model rank = 23 / 23
```

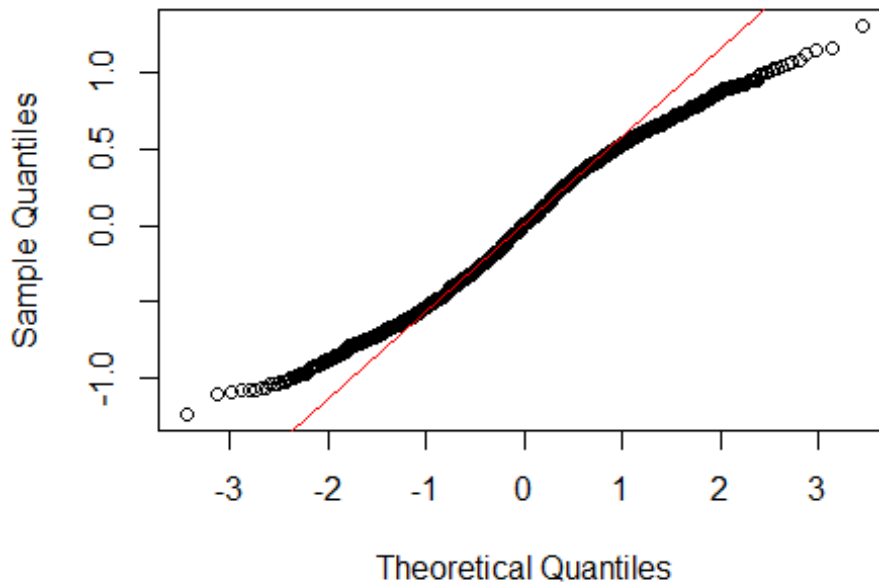
```
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##               k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.00 2.00    0.99    0.33
## s(Price_USD)                   9.00 8.61    0.95    0.02 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```



```
qqnorm(res)
qqline(res, col = "red")
```

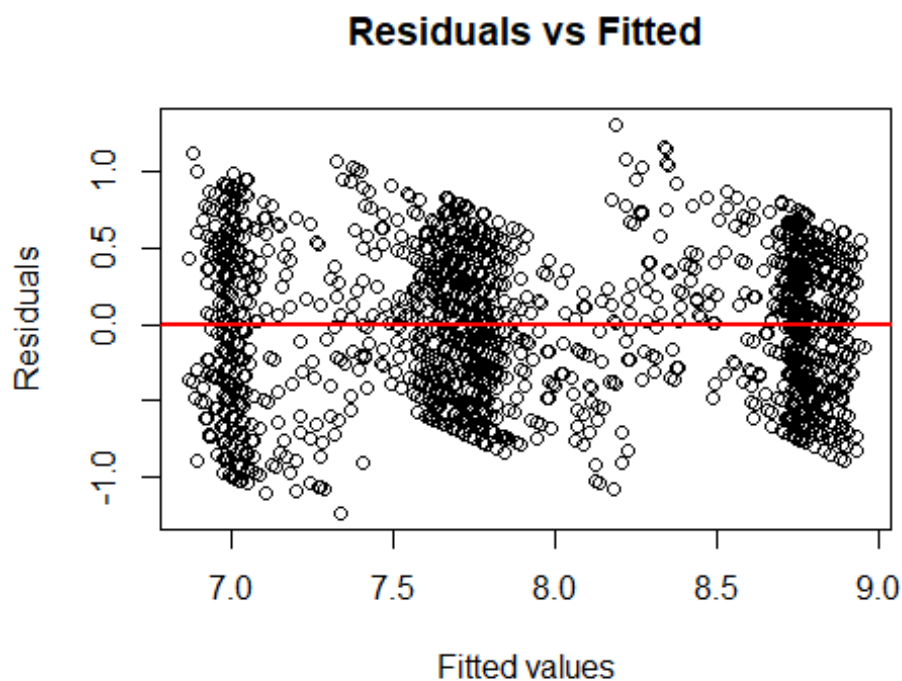
Normal Q-Q Plot



```
fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# Permutation test for GAM
# H0: Category = 0

# Observed statistic for Category
T0 = max(abs(summary(gam_fit)$p.table[2:5, "t value"])))

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Price_USD, bs="cr"),
  data = df_train
)

res.H0 = gam_H0$residuals

wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      s(Price_USD, bs="cr") +
      Category,
    data = df_train
  )
  return(max(abs(summary(gam.perm)$p.table[2:5, "t value"])))
}
```

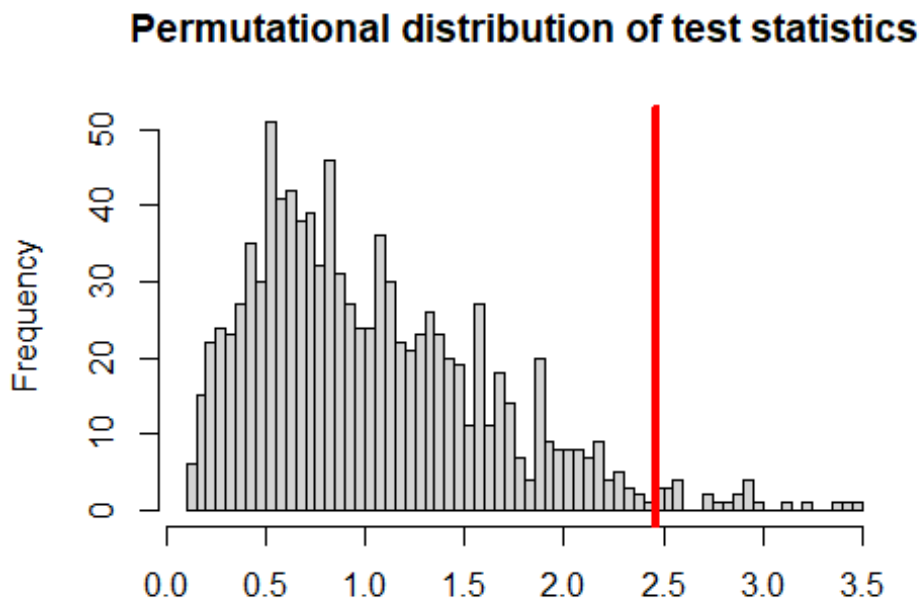
```

}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

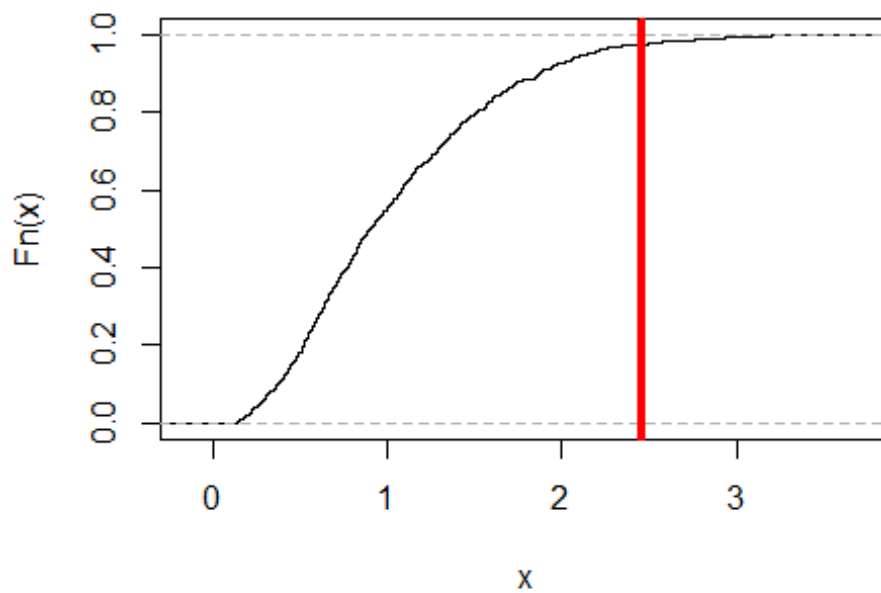


```

plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)

```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
```

```
P #0.013
```

```
## [1] 0.026
```

```
# We keep Category
```

```
# GAM model
```

```
gam_fit <- gam(  
  User_Satisfaction_Rating ~  
    s(Step_Count_Accuracy_Percent, bs="cr") +  
    s(Price_USD, bs="cr") +  
    Category,  
  data = df_train  
)
```

```
summary(gam_fit)
```

```
##
```

```
## Family: gaussian
```

```
## Link function: identity
```

```
##
```

```
## Formula:
```

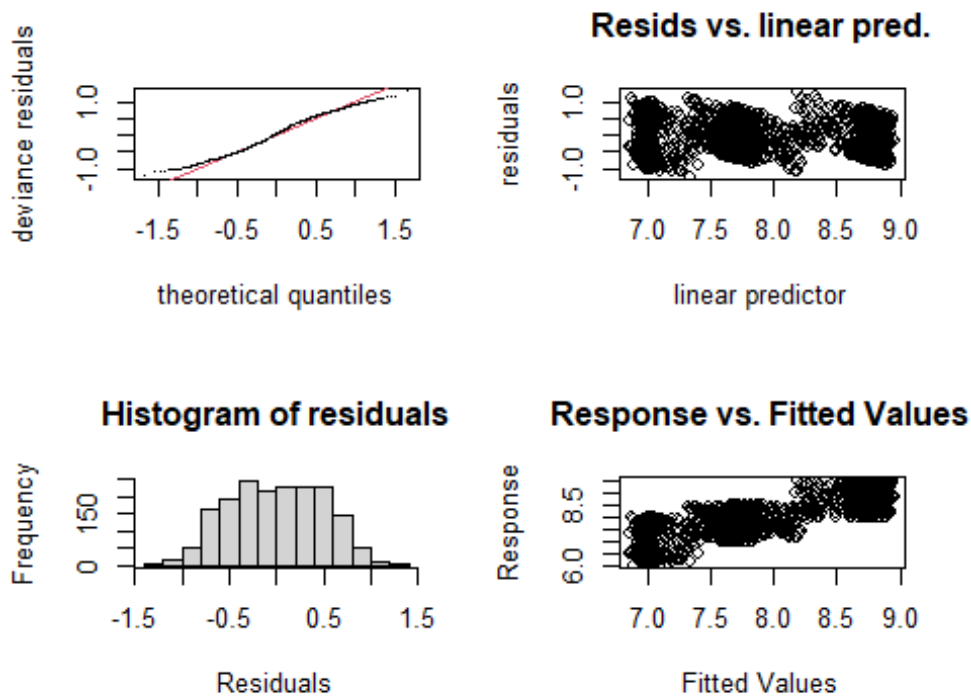
```
## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
```

```
##   s(Price_USD, bs = "cr") + Category
```

```
##
```

```
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)      7.6675     0.1316  58.284  <2e-16 ***
## CategoryFitness Tracker  0.3684     0.1498   2.459   0.0140 *
## CategorySmart Ring      0.2636     0.1526   1.728   0.0842 .
## CategorySmartwatch      0.3370     0.1467   2.297   0.0217 *
## CategorySports Watch    0.2794     0.1479   1.889   0.0591 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 2.005  2.513   2.14   0.106
## s(Price_USD)                   8.614  8.940 238.82 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.664   Deviance explained = 66.6%
## GCV = 0.23753   Scale est. = 0.23545   n = 1782
```

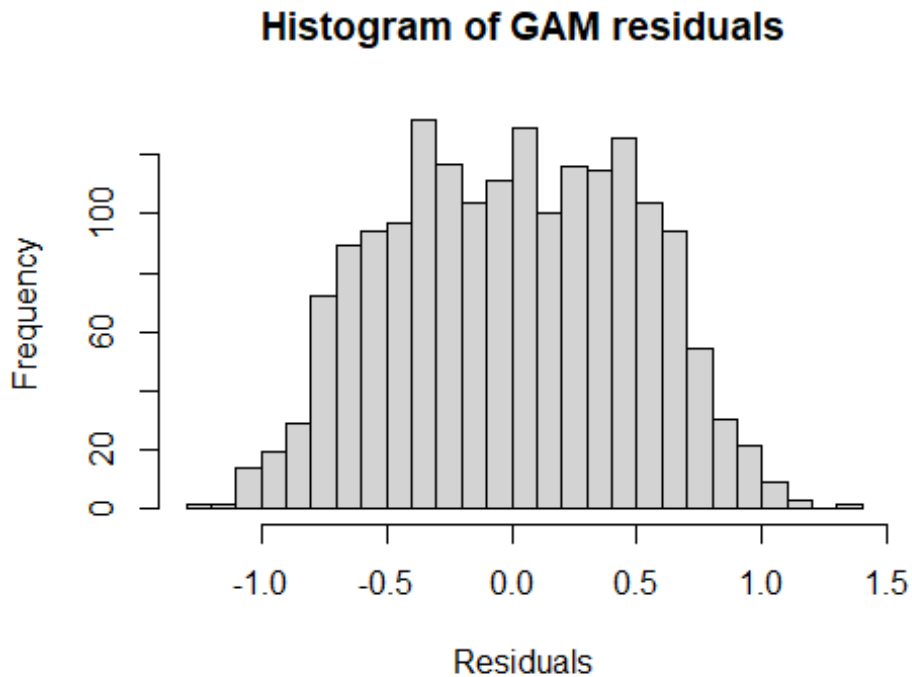
```
gam.check(gam_fit)
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 11 iterations.
## The RMS GCV score gradient at convergence was 3.428304e-08 .
## The Hessian was positive definite.
## Model rank = 23 / 23
```

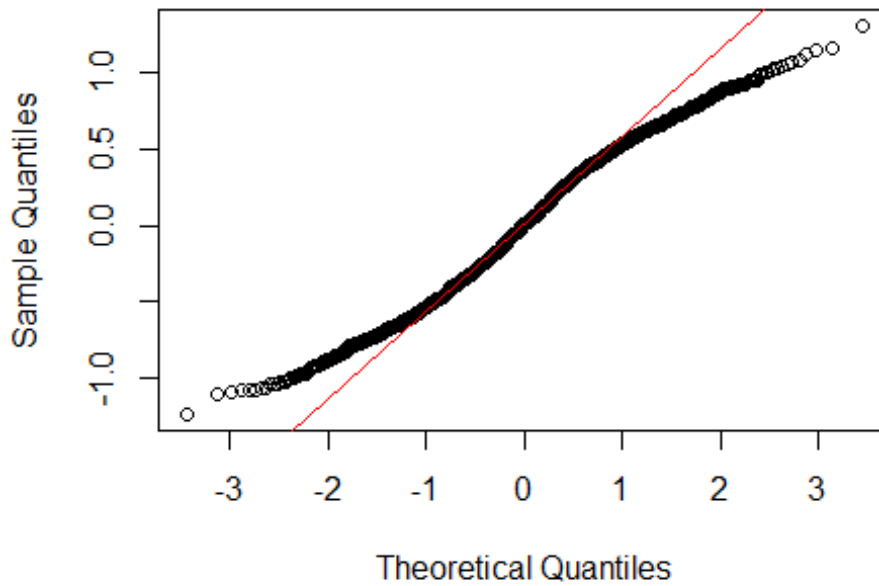
```
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##               k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.00 2.00    0.99    0.34
## s(Price_USD)                   9.00 8.61    0.95    0.02 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```



```
qqnorm(res)
qqline(res, col = "red")
```

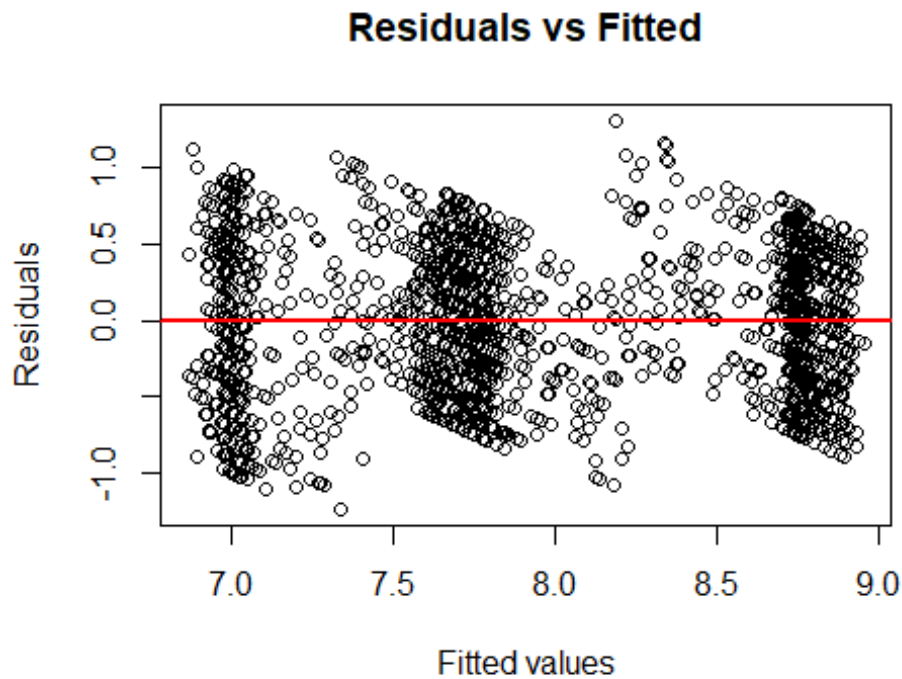

Normal Q-Q Plot



```
fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# Permutation test for GAM
# H0: Price_USD = 0

#Observed statistic for Price_USD
T0 = abs(summary(gam_fit)$s.table[2, 3])

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    Category,
  data = df_train
)

res.H0 = gam_H0$residuals

wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      s(Price_USD, bs="cr") +
      Category,
    data = df_train
  )
  return(abs(summary(gam.perm)$s.table[2, 3]))
}
```

```

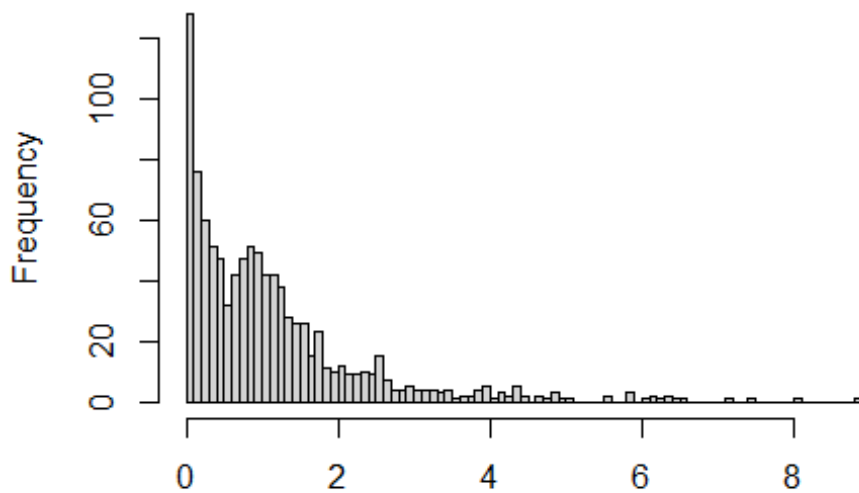
}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

Permutational distribution of test statistics

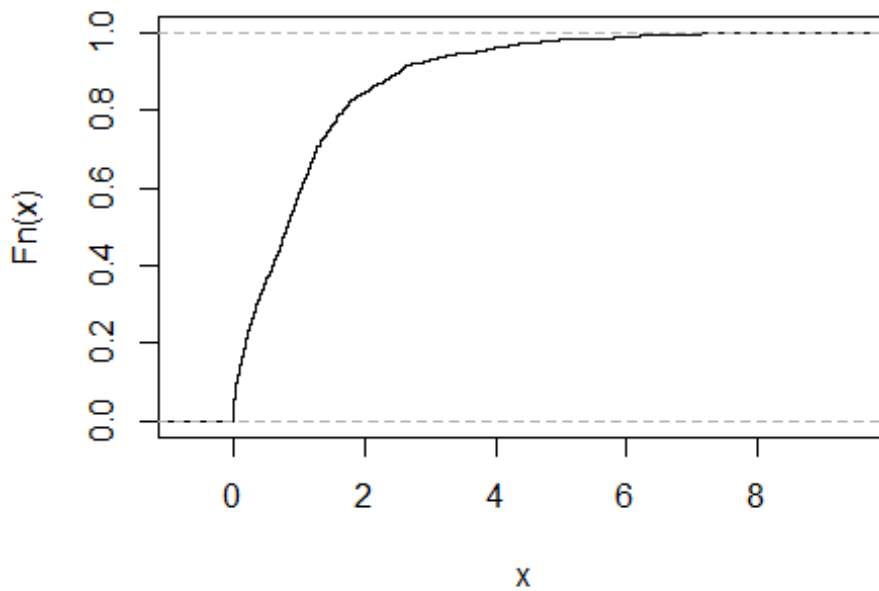


```

plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)

```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P

## [1] 0

# We keep Price_USD

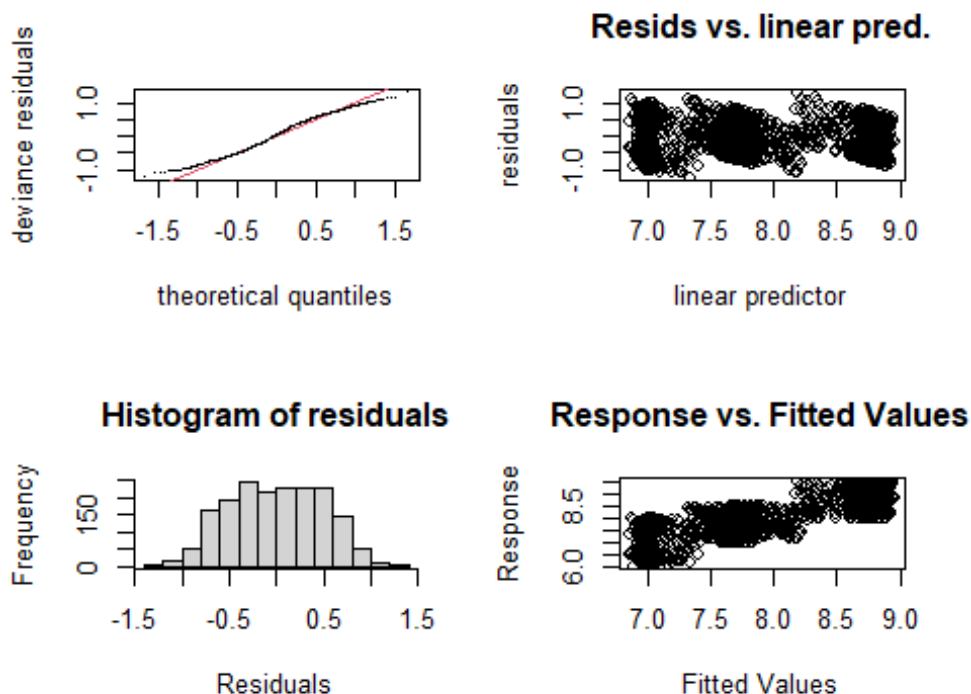
# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Step_Count_Accuracy_Percent, bs="cr") +
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

summary(gam_fit)

##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Step_Count_Accuracy_Percent, bs = "cr") +
##   s(Price_USD, bs = "cr") + Category
```

```
##
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)      7.6675     0.1316  58.284  <2e-16 ***
## CategoryFitness Tracker  0.3684     0.1498   2.459   0.0140 *
## CategorySmart Ring      0.2636     0.1526   1.728   0.0842 .
## CategorySmartwatch      0.3370     0.1467   2.297   0.0217 *
## CategorySports Watch    0.2794     0.1479   1.889   0.0591 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(Step_Count_Accuracy_Percent) 2.005  2.513   2.14   0.106
## s(Price_USD)                   8.614  8.940 238.82 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.664   Deviance explained = 66.6%
## GCV = 0.23753   Scale est. = 0.23545   n = 1782
```

```
gam.check(gam_fit)
```

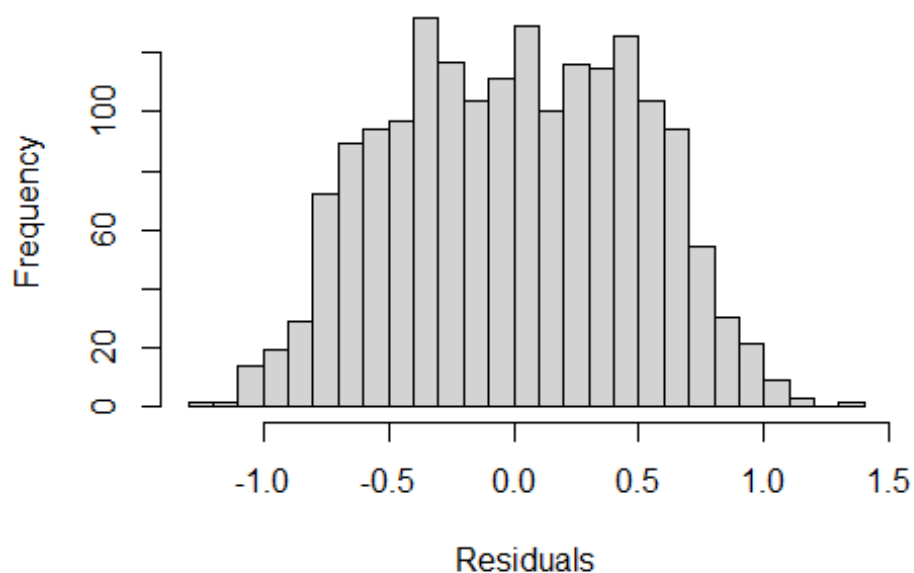


```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 11 iterations.
## The RMS GCV score gradient at convergence was 3.428304e-08 .
## The Hessian was positive definite.
```

```
## Model rank = 23 / 23
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##               k'   edf k-index p-value
## s(Step_Count_Accuracy_Percent) 9.00 2.00    0.99    0.33
## s(Price_USD)                   9.00 8.61    0.95    0.02 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

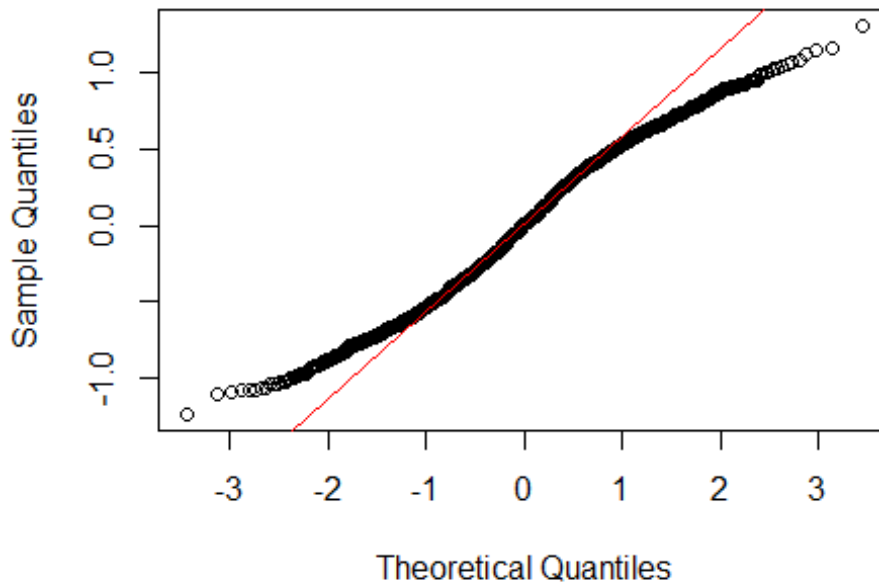
# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```

Histogram of GAM residuals



```
qqnorm(res)
qqline(res, col = "red")
```

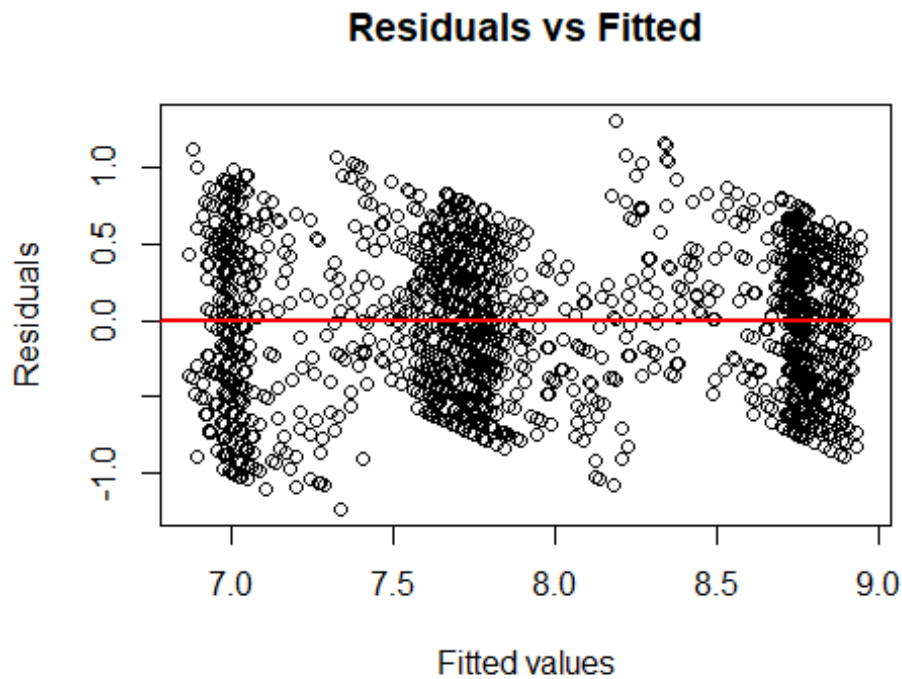
Normal Q-Q Plot



```
fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# Permutation test for GAM
# H0: Step_Count_Accuracy_Percent = 0

# Observed statistic for Step_Count_Accuracy_Percent
T0 = abs(summary(gam_fit)$s.table[1, 3])

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

res.H0 = gam_H0$residuals

wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Step_Count_Accuracy_Percent, bs="cr") +
      s(Price_USD, bs="cr") +
      Category,
    data = df_train
  )
  return(abs(summary(gam.perm)$s.table[1, 3]))
}
```



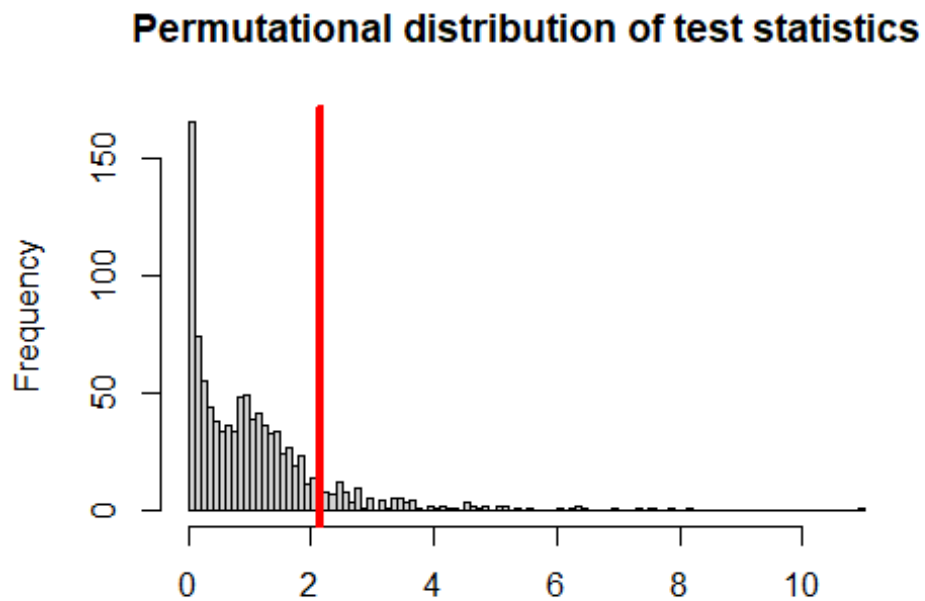
```

}

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
      breaks = 100,
      main = 'Permutational distribution of test statistics',
      xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

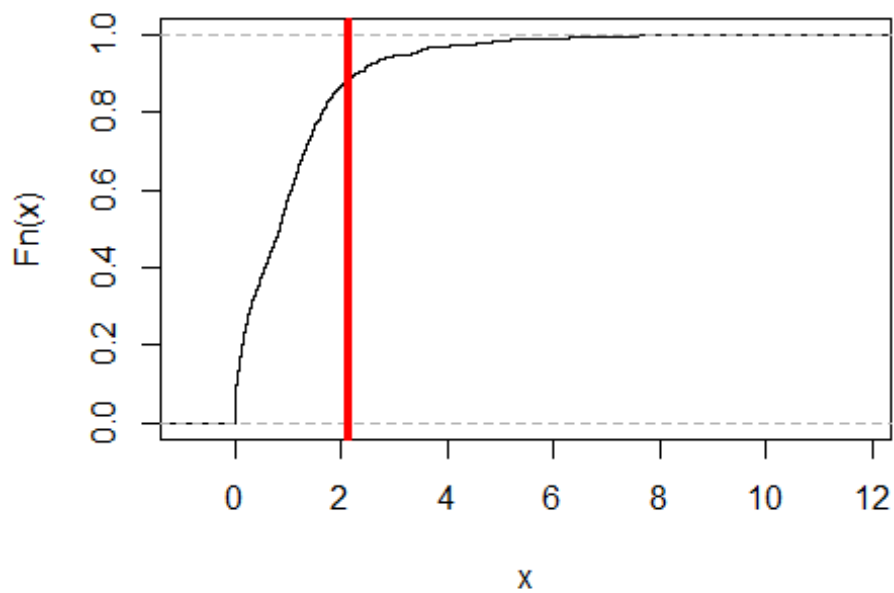


```

plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)

```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P

## [1] 0.115

# We remove Step_Count_Accuracy_Percent

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

summary(gam_fit)

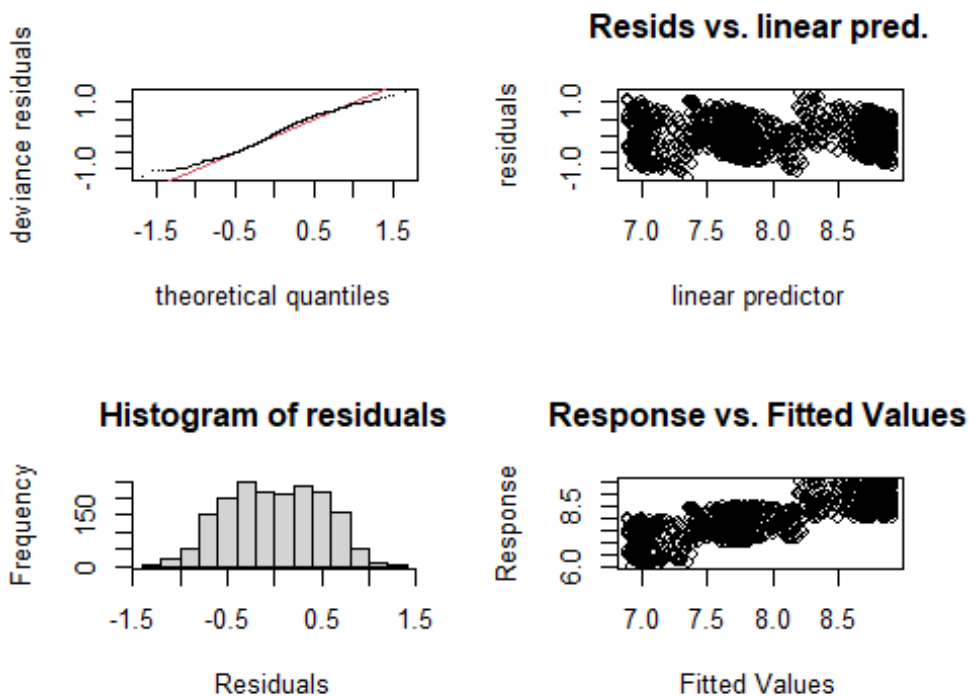
##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Price_USD, bs = "cr") + Category
##
## Parametric coefficients:
##
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.811	0.007	260.1	<.0001
Category	0.000	0.000	0.0	1.000

```
##
```

```
## (Intercept)          7.6752      0.1316  58.320   <2e-16 ***
## CategoryFitness Tracker 0.3615      0.1498   2.414   0.0159 *
## CategorySmart Ring     0.2592      0.1522   1.702   0.0888 .
## CategorySmartwatch     0.3253      0.1467   2.217   0.0268 *
## CategorySports Watch   0.2760      0.1480   1.865   0.0624 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Price_USD) 8.607  8.938 262.9 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.663   Deviance explained = 66.5%
## GCV = 0.23791   Scale est. = 0.23609    n = 1782

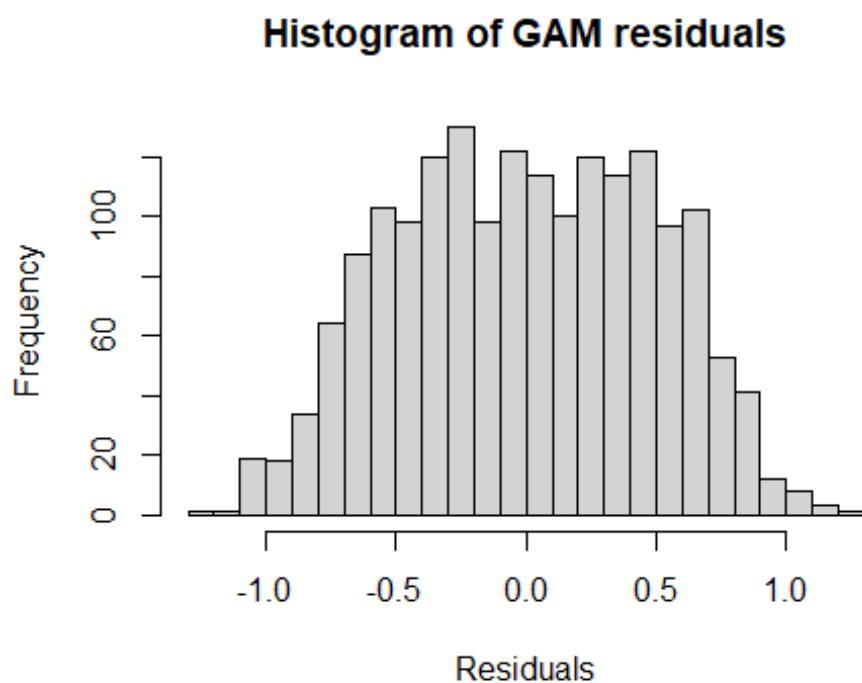
gam.check(gam_fit)
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 7 iterations.
## The RMS GCV score gradient at convergence was 6.625446e-07 .
## The Hessian was positive definite.
## Model rank =  14 / 14
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
```

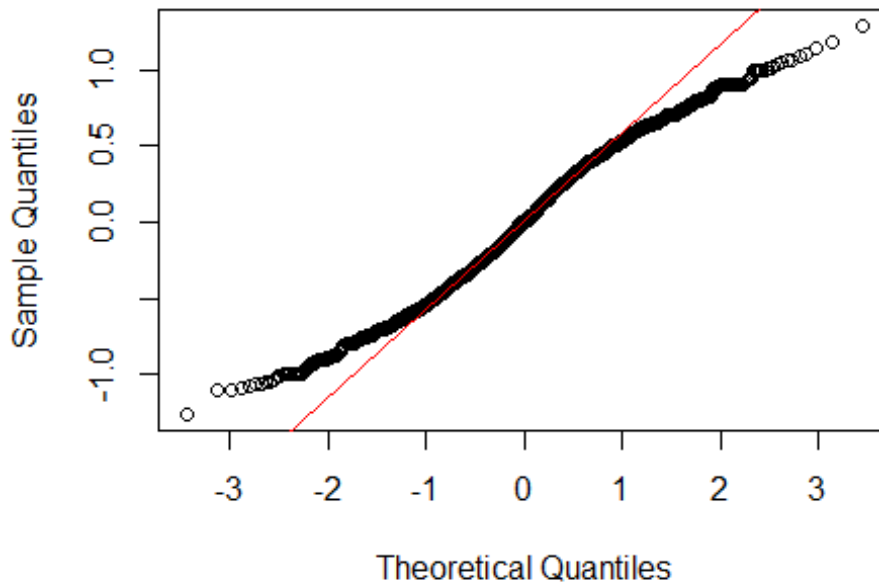
```
##
##           k'   edf k-index p-value
## s(Price_USD) 9.00 8.61    0.95  0.005 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```



```
qqnorm(res)
qqline(res, col = "red")
```

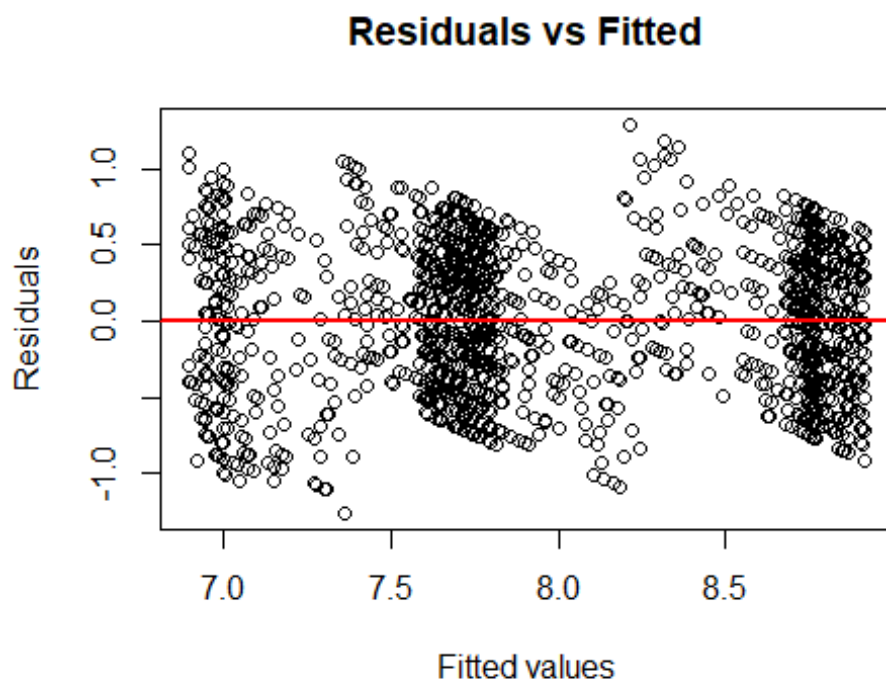
Normal Q-Q Plot



```
fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# Permutation test for GAM
# H0: Price_USD = 0

# Observed statistic for Price_USD
T0 = abs(summary(gam_fit)$s.table[1, 3])

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    Category,
  data = df_train
)

res.H0 = gam_H0$residuals

wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Price_USD, bs="cr") +
      Category,
    data = df_train
  )
  return(abs(summary(gam.perm)$s.table[1, 3]))
}
```

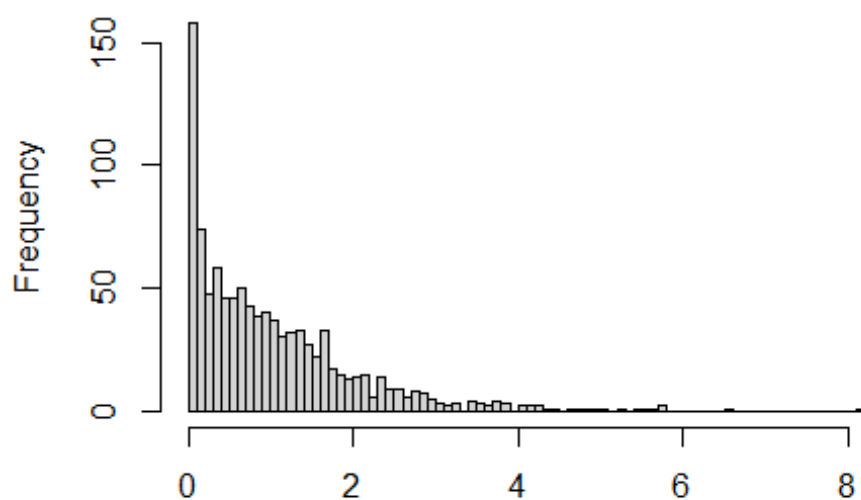
```

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
     breaks = 100,
     main = 'Permutational distribution of test statistics',
     xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

Permutational distribution of test statistics

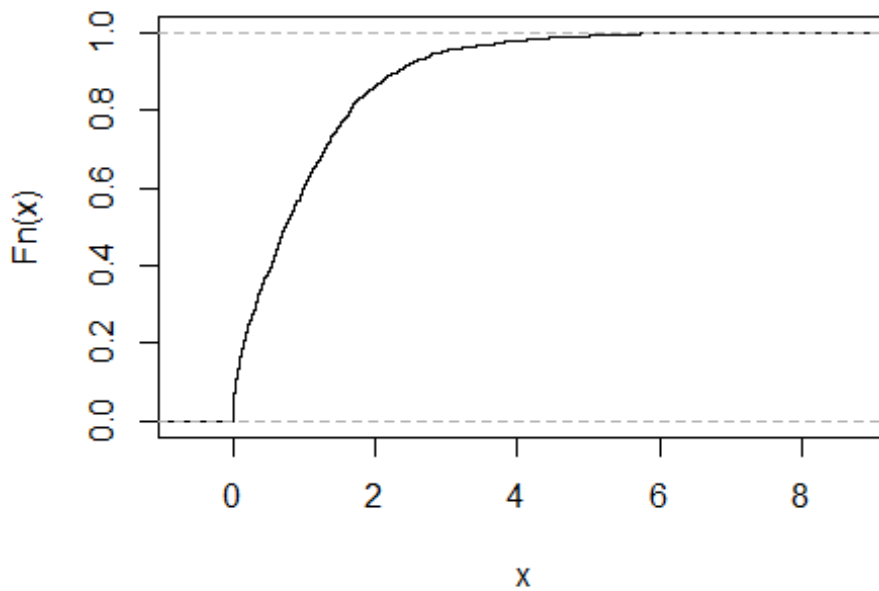


```

plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)

```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P

## [1] 0

# We keep Price_USD

# GAM model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

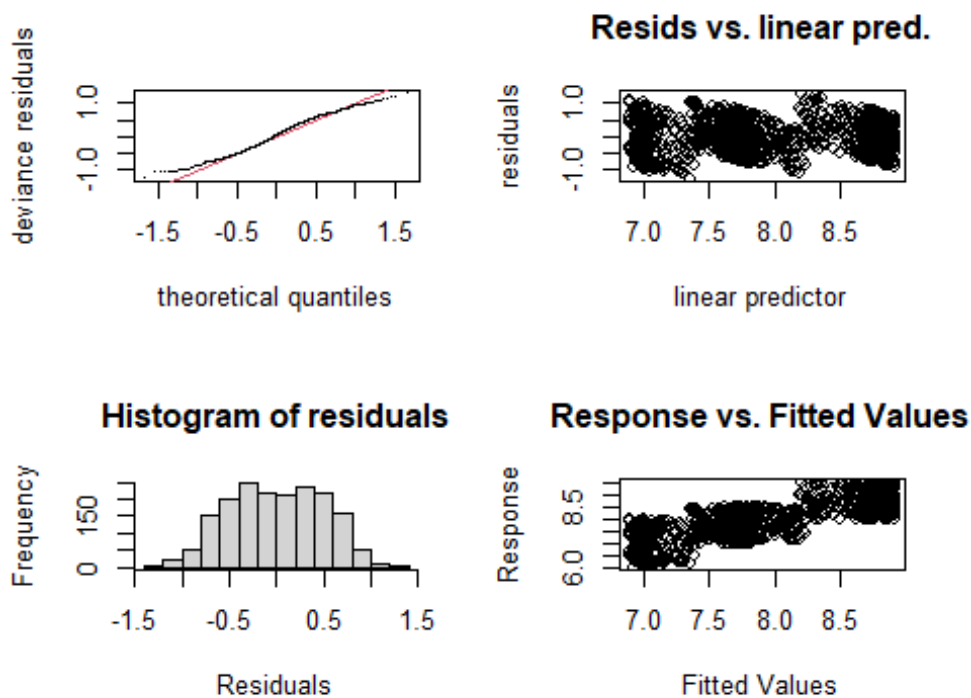
summary(gam_fit)

##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Price_USD, bs = "cr") + Category
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    7.6752     0.1316  58.320   <2e-16 ***
```



```
## CategoryFitness Tracker    0.3615    0.1498    2.414    0.0159 *
## CategorySmart Ring        0.2592    0.1522    1.702    0.0888 .
## CategorySmartwatch        0.3253    0.1467    2.217    0.0268 *
## CategorySports Watch      0.2760    0.1480    1.865    0.0624 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##          edf Ref.df      F p-value
## s(Price_USD) 8.607  8.938 262.9 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.663   Deviance explained = 66.5%
## GCV = 0.23791   Scale est. = 0.23609    n = 1782

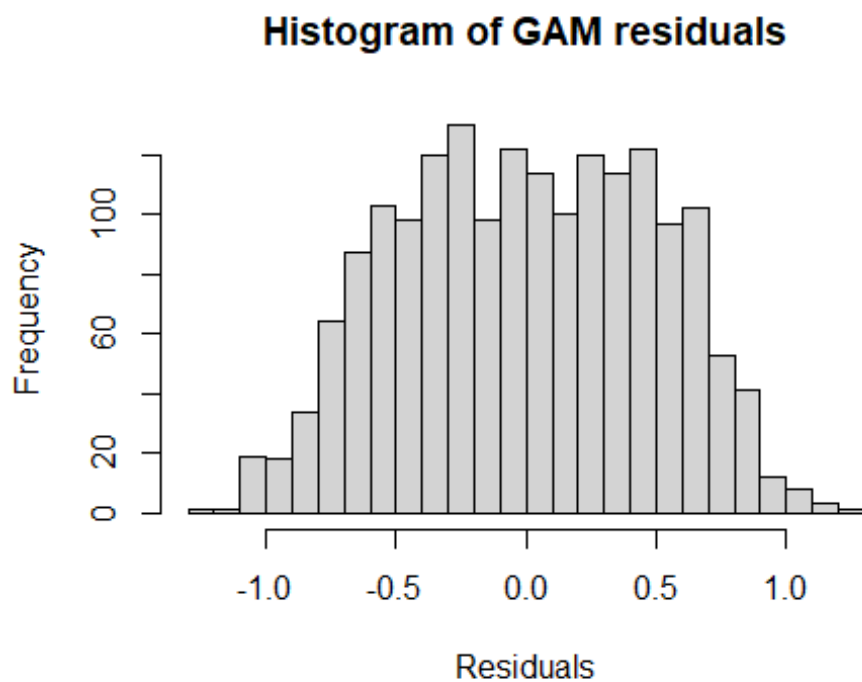
gam.check(gam_fit)
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 7 iterations.
## The RMS GCV score gradient at convergence was 6.625446e-07 .
## The Hessian was positive definite.
## Model rank =  14 / 14
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
```

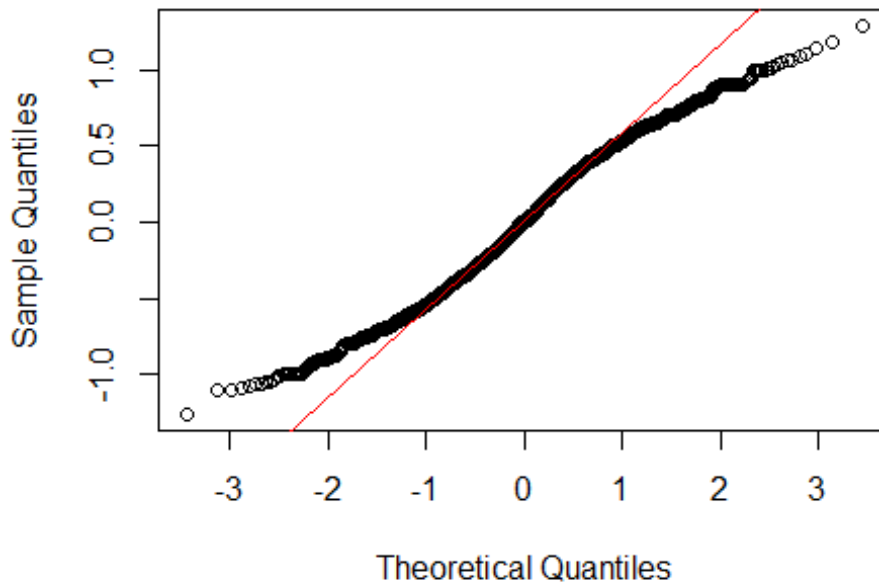
```
##          k'   edf k-index p-value
## s(Price_USD) 9.00 8.61    0.95 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Residual diagnostics
res <- residuals(gam_fit)
hist(res, breaks = 30, main = "Histogram of GAM residuals",
      xlab = "Residuals")
```



```
qqnorm(res)
qqline(res, col = "red")
```

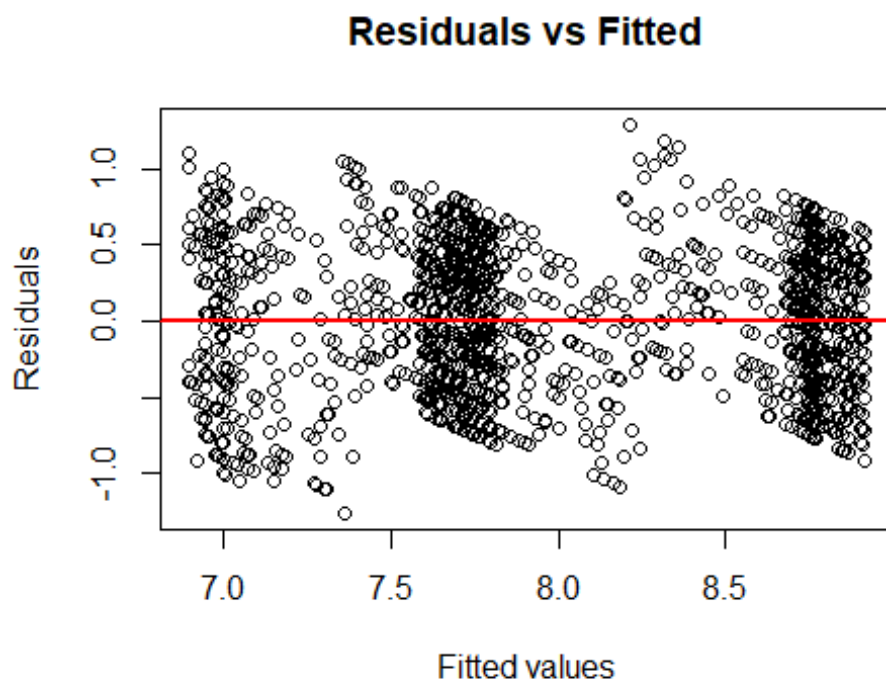
Normal Q-Q Plot



```
fit <- fitted(gam_fit)

plot(
  fit, res,
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red", lwd = 2)
```



```
# Permutation test for GAM
# H0: Category = 0

# Observed statistic for Category
T0 = max(abs(summary(gam_fit)$p.table[2:5, 3]))

gam_H0 <- gam(
  User_Satisfaction_Rating ~
    s(Price_USD, bs="cr"),
  data = df_train
)

res.H0 = gam_H0$residuals

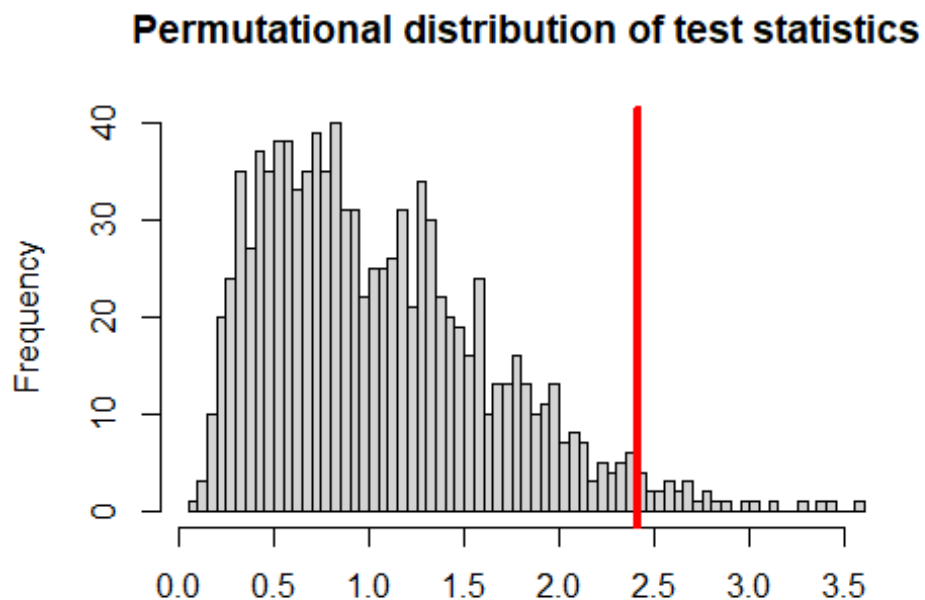
wrapper = function() {
  permutation = sample(n_train)
  res.H0.perm = res.H0[permutation]
  Y.perm.H0 = gam_H0$fitted + res.H0.perm
  gam.perm = gam(
    Y.perm.H0 ~
      s(Price_USD, bs="cr") +
      Category,
    data = df_train
  )
  return(max(abs(summary(gam.perm)$p.table[2:5, 3])))
}
```

```

T_H0 = pbreplicate(B, wrapper(), simplify = 'vector')

hist(sort(T_H0)[-1000],
     breaks = 100,
     main = 'Permutational distribution of test statistics',
     xlab = '')
abline(v = T0, col = 'red', lwd = 4)

```

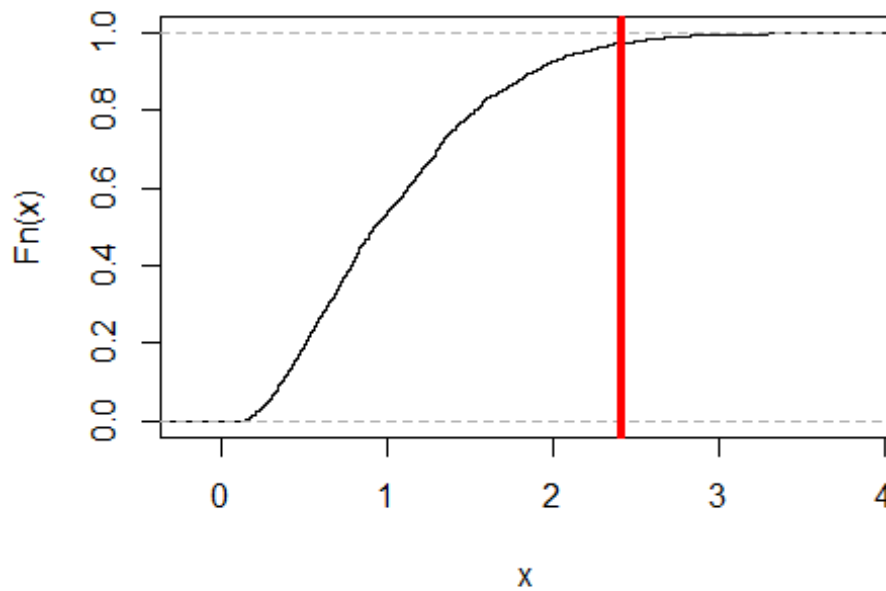


```

plot(ecdf(sort(T_H0)[-1000]), main = 'ECDF of test statistics')
abline(v = T0, col = 'red', lwd = 4)

```

ECDF of test statistics



```
P = sum(T_H0 >= T0) / B
P #0.021

## [1] 0.029

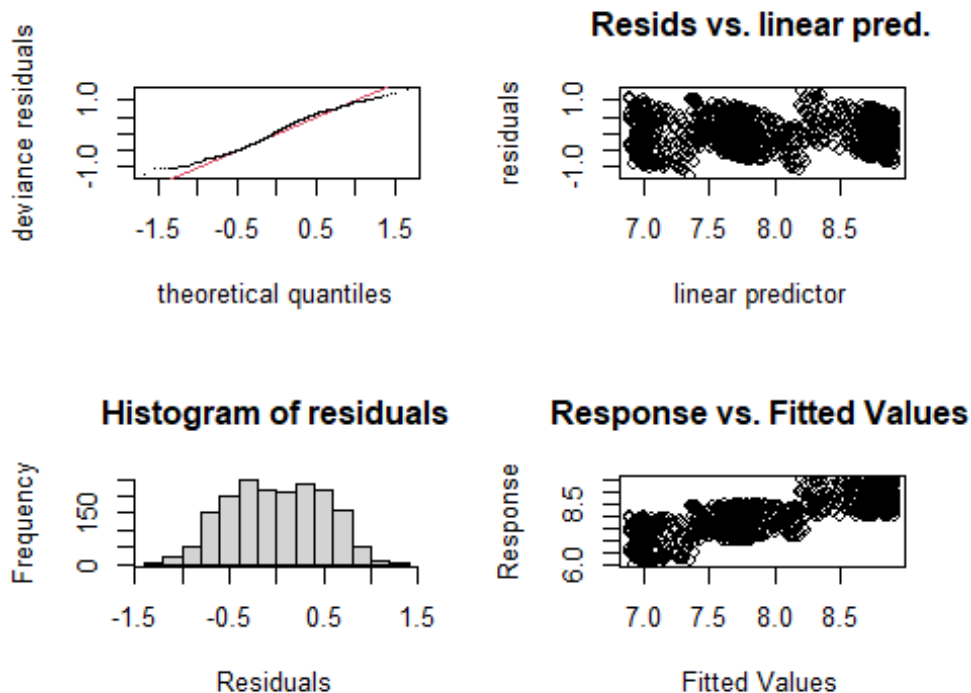
# We keep Category

### Final model
gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)
summary(gam_fit)

##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Price_USD, bs = "cr") + Category
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    7.6752    0.1316  58.320   <2e-16 ***
## CategoryFitness Tracker  0.3615    0.1498   2.414   0.0159 *
```

```
## CategorySmart Ring      0.2592      0.1522      1.702      0.0888 .
## CategorySmartwatch      0.3253      0.1467      2.217      0.0268 *
## CategorySports Watch    0.2760      0.1480      1.865      0.0624 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Price_USD) 8.607  8.938 262.9 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.663   Deviance explained = 66.5%
## GCV = 0.23791   Scale est. = 0.23609    n = 1782

gam.check(gam_fit)
```



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 7 iterations.
## The RMS GCV score gradient at convergence was 6.625446e-07 .
## The Hessian was positive definite.
## Model rank = 14 / 14
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##              k'   edf k-index p-value
```

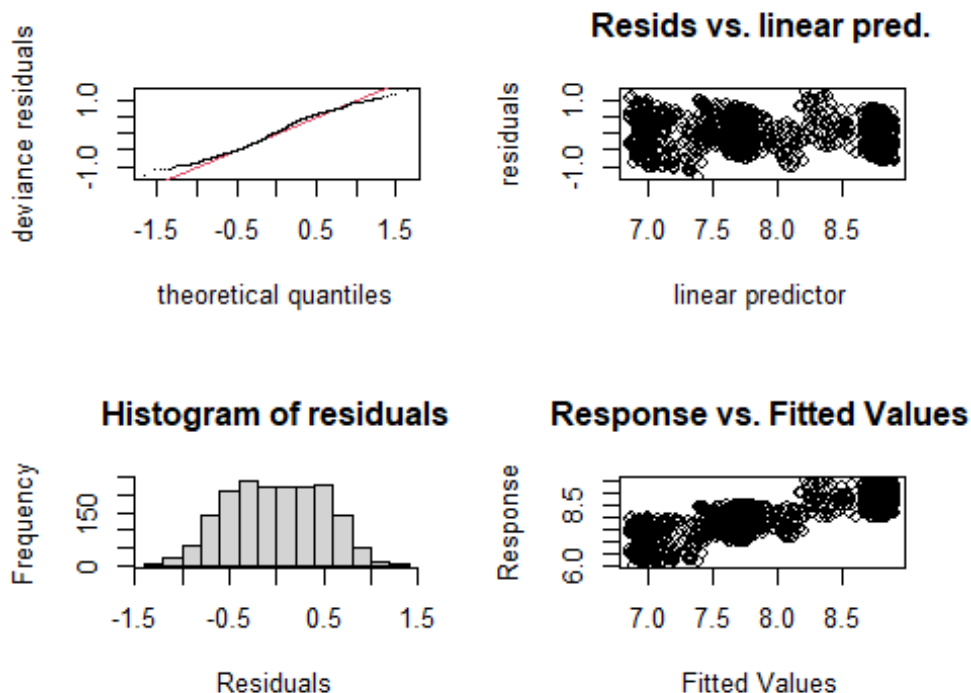
```

## s(Price_USD) 9.00 8.61    0.95    0.01 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

gam_fit <- gam(
  User_Satisfaction_Rating ~
    s(Price_USD, bs="cr", k=12) +
    Category,
  data = df_train
)
summary(gam_fit)

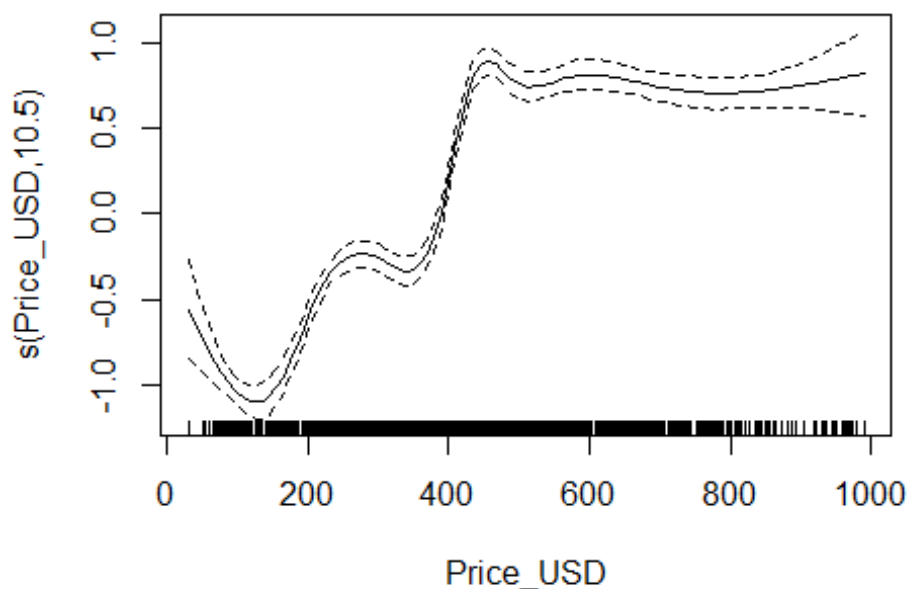
##
## Family: gaussian
## Link function: identity
##
## Formula:
## User_Satisfaction_Rating ~ s(Price_USD, bs = "cr", k = 12) +
##   Category
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      7.5598    0.1491  50.707 < 2e-16 ***
## CategoryFitness Tracker  0.4842    0.1685   2.873  0.00412 **
## CategorySmart Ring      0.4067    0.1710   2.378  0.01752 *
## CategorySmartwatch      0.4521    0.1663   2.719  0.00661 **
## CategorySports Watch    0.4022    0.1674   2.402  0.01640 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(Price_USD) 10.5  10.92 221.4 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.668   Deviance explained = 67.1%
## GCV = 0.23442   Scale est. = 0.23238    n = 1782
gam.check(gam_fit)

```

```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 7 iterations.
## The RMS GCV score gradient at convergence was 2.003019e-06 .
## The Hessian was positive definite.
## Model rank = 16 / 16
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##           k'   edf k-index p-value
## s(Price_USD) 11.0 10.5   0.96   0.07 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

plot(gam_fit)
```



```
# Conformal prediction interval for the GAM model ----
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
# Create prediction grid for Price vs Category
```

```
pred_grid <- expand_grid(
  Price_USD = seq(
    min(df_train$Price_USD),
    max(df_train$Price_USD),
    length.out = 200
  ),
  Category = levels(df_train$Category)
)
```

```
pred_grid$fit <- predict(gam_fit, newdata = pred_grid)
```

```
# Prediction on the calibration set
```

```
cal_pred <- predict(gam_fit, newdata = df_cal)
```

```
# Compute nonconformity scores
```

```
scores <- abs(df_cal$User_Satisfaction - cal_pred)
```

```
# Compute conformal quantiles
```

```
q_hat <- quantile(scores, probs = 1 - alpha)
```

```
pred_grid <- pred_grid %>%
```

```

mutate(
  lower = fit - q_hat,
  upper = fit + q_hat
)

ggplot(pred_grid, aes(x = Price_USD, y = fit, color = Category, fill = Category))
+

# Conformal prediction intervals
geom_ribbon(aes(ymin = lower, ymax = upper),
  alpha = 0.15, color = NA) +

# GAM fitted curves
geom_line(size = 1.1) +

# Observed data points
geom_point(
  data = df_train,
  aes(x = Price_USD, y = User_Satisfaction_Rating, color = Category),
  alpha = 0.4,
  size = 1
) +

labs(
  title = "Conformal prediction intervals across device categories",
  x = "Price (USD)",
  y = "User Satisfaction Rating"
) +

theme_minimal()

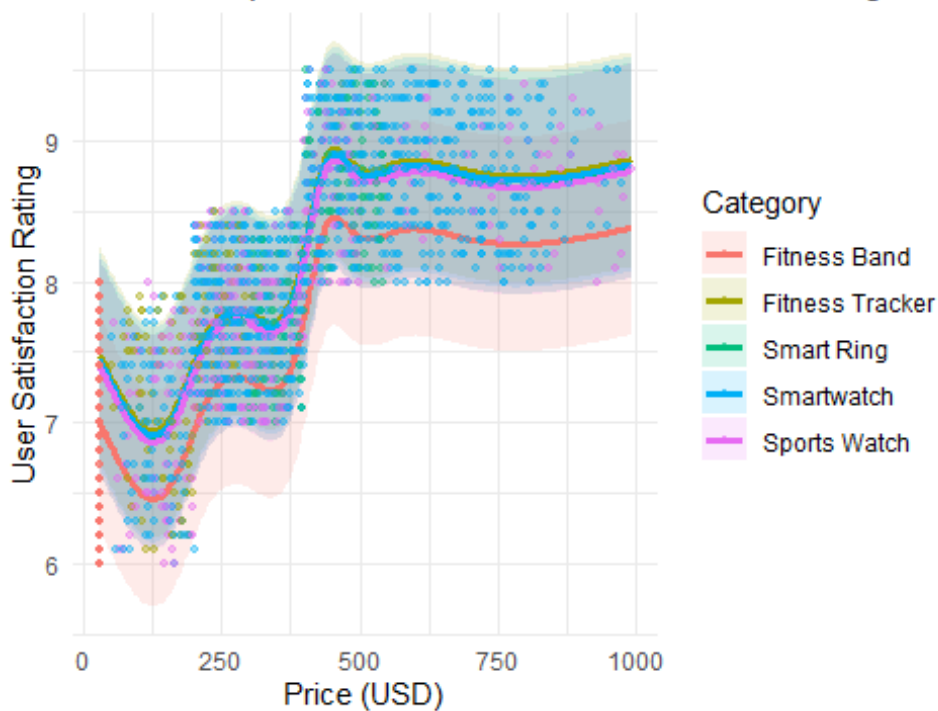
```

```

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```

Conformal prediction intervals across device categories



```
# Normalized conformal prediction ----

# Absolute residuals on calibration set
residuals_cal <- abs(df_cal$User_Satisfaction_Rating - cal_pred)

# Fit GAM on log residuals from training set
train_pred <- predict(gam_fit, newdata = df_train)
residuals_train <- abs(df_train$User_Satisfaction_Rating - train_pred)
log_residuals <- log(residuals_train + 1e-6)
log_sigma_fit <- gam(
  log_residuals ~
    s(Price_USD, bs="cr") +
    Category,
  data = df_train
)

# Predict scale for calibration set
sigma_cal <- exp(predict(log_sigma_fit, newdata = df_cal))

# Normalized scores for calibration
cal_pred <- predict(gam_fit, newdata = df_cal)
residuals_cal <- abs(df_cal$User_Satisfaction_Rating - cal_pred)
norm_scores <- residuals_cal / sigma_cal

# Quantile for normalized conformal interval
q_hat_norm <- quantile(norm_scores, probs = 1 - alpha)
```

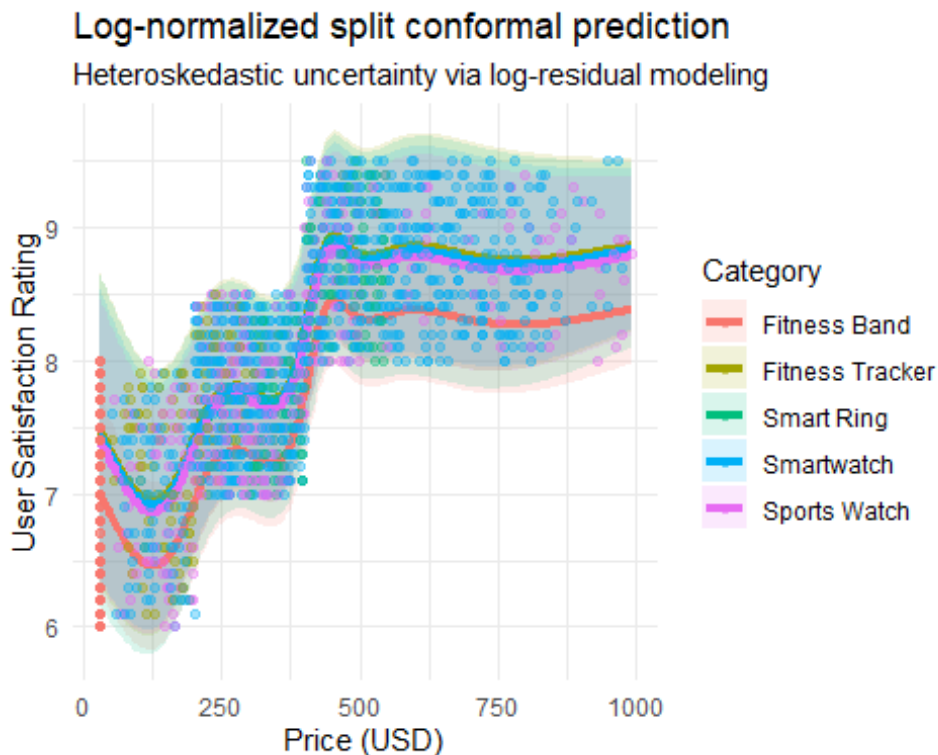
```

# Normalized conformal prediction interval
pred_grid$sigma <- exp(predict(log_sigma_fit, newdata = pred_grid))

pred_grid <- pred_grid %>%
  mutate(
    lower_norm = fit - q_hat_norm * sigma,
    upper_norm = fit + q_hat_norm * sigma
  )

ggplot(pred_grid, aes(x = Price_USD, y = fit, color = Category, fill = Category))
+
  geom_ribbon(aes(ymin = lower_norm, ymax = upper_norm),
    alpha = 0.15, color = NA) +
  geom_line(size = 1.5) +
  geom_point(
    data = df_train,
    aes(x = Price_USD, y = User_Satisfaction_Rating, color = Category),
    alpha = 0.35,
    size = 1.5
  ) +
  labs(
    title = "Log-normalized split conformal prediction",
    subtitle = "Heteroskedastic uncertainty via log-residual modeling",
    x = "Price (USD)",
    y = "User Satisfaction Rating"
  ) +
  theme_minimal()

```



```

ggplot(pred_grid, aes(x = Price_USD, y = fit)) +

  # Normalized conformal prediction intervals
  geom_ribbon(
    aes(ymin = lower_norm, ymax = upper_norm),
    fill = "steelblue",
    alpha = 0.2
  ) +

  # GAM fitted curve
  geom_line(
    color = "steelblue",
    size = 1.2
  ) +

  # Observed data points
  geom_point(
    data = df_train,
    aes(x = Price_USD, y = User_Satisfaction_Rating),
    alpha = 0.4,
    size = 1
  ) +

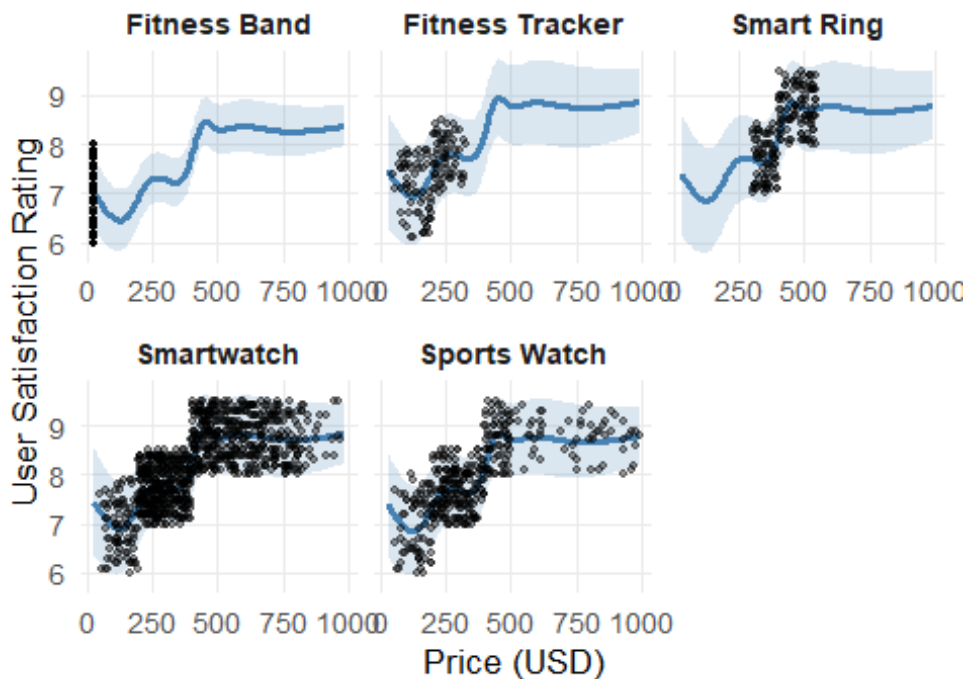
  # One panel per category
  facet_wrap(~ Category, scales = "free_x") +

  labs(
    title = "Normalized conformal prediction intervals by device category",
    x = "Price (USD)",
    y = "User Satisfaction Rating"
  ) +

  theme_minimal(base_size = 13) +
  theme(
    strip.text = element_text(face = "bold"),
    plot.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )

```

Normalized conformal prediction intervals b



```
# Predict new device (Smart Ring, 700$)
new_device <- data.frame(
  Battery_Life_Hours = 48,
  Heart_Rate_Accuracy_Percent = 96,
  Step_Count_Accuracy_Percent = 95,
  Sleep_Tracking_Accuracy_Percent = 87,
  GPS_Accuracy_Meters = 8,
  Health_Sensors_Count = 6,
  Water_Resistance_Score = 5,
  Connectivity_Features_Score = 3,
  App_Ecosystem_Support_Score = 1,
  Price_USD = 700,
  Category = factor("Smart Ring", levels = levels(df_model$Category))
)

pred_new <- predict(gam_fit, newdata = new_device)
sigma_new <- exp(predict(log_sigma_fit, newdata = new_device))

lower_norm <- pred_new - q_hat_norm * sigma_new
upper_norm <- pred_new + q_hat_norm * sigma_new

c(
  lower_bound = lower_norm,
  prediction = pred_new,
  upper_bound = upper_norm
)
```

```
## lower_bound.1 prediction.1 upper_bound.1
##      7.802199      8.708763      9.615327

# Predict new device (Smartwatch, 700$)
new_device <- data.frame(
  Battery_Life_Hours = 48,
  Heart_Rate_Accuracy_Percent = 96,
  Step_Count_Accuracy_Percent = 95,
  Sleep_Tracking_Accuracy_Percent = 87,
  GPS_Accuracy_Meters = 8,
  Health_Sensors_Count = 6,
  Water_Resistance_Score = 5,
  Connectivity_Features_Score = 3,
  App_Ecosystem_Support_Score = 1,
  Price_USD = 700,
  Category = factor("Smartwatch", levels = levels(df_model$Category))
)

pred_new <- predict(gam_fit, newdata = new_device)
sigma_new <- exp(predict(log_sigma_fit, newdata = new_device))

lower_norm <- pred_new - q_hat_norm * sigma_new
upper_norm <- pred_new + q_hat_norm * sigma_new

c(
  lower_bound = lower_norm,
  prediction = pred_new,
  upper_bound = upper_norm
)

## lower_bound.1 prediction.1 upper_bound.1
##      7.964423      8.754145      9.543866
```